

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

Кафедра «Информационные системы»

Дисциплина «Информационная безопасность»

Лабораторная работа № 3

Изучение криптографических алгоритмов.
Алгоритмы шифрования.

Выполнил:
студент гр. ПИБд-41
Белянин Н.Н.
Проверил:
Доцент, к.т.н.
Мытарев П. В.

Ульяновск, 2025г.

Лабораторная работа № 3

Изучение криптографических алгоритмов. Алгоритмы шифрования.

Содержание задания

1. Реализовать указанный преподавателем криптографический алгоритм.
2. Программа должна иметь интерфейс, в котором содержатся данные студента, вариант, справочная информация – алгоритм и его описание, возможности для сохранения и выбора обрабатываемого файла.
3. В случае шифрования пароль должен вводиться как с клавиатуры, так и из файла по выбору.
4. Отчет должен содержать вариант задания, блок-схему алгоритма, реализованного в программе, листинг программы с комментариями, расчет данных по одному циклу алгоритма.
5. Отчет и код программы с исполняемым кодом дополнительно представляются в электронном виде.
6. Обрабатываться должен файл любого формата(word,excel, графические форматы и т.д.), длиной не менее 1 кБ.
7. В программе должна быть возможность выбора исходного файла и файла назначения как для шифрования, так и для расшифрования.
8. Ответить на вопросы преподавателя.

Вопросы:

1. Симметричные алгоритмы шифрования и их характеристики.
2. Асимметричные алгоритмы шифрования и их характеристики.
3. Реализация хэш-функций.
4. Реализация MAC.
5. Реализация ЭЦП.
6. Понятие слоя, раунда, S-box.
7. Как в ГОСТ Р 34.12-2018 Магма (ГОСТ 28147) осуществляется преобразование с помощью S-boxes. Что подается на вход и получается на выходе.

Вариант – 6

6. Алгоритм ГОСТ Р 34.12-2018 Магма (ГОСТ 28147). Режим гаммирования с ОС.

Ссылка на репозиторий: <https://github.com/Nikbeli/Information-Security>

Оглавление

Ход выполнения лабораторной работы.....	5
<i>Теоретическая часть выполнения</i>	5
Энигма	6
Шифрование	6
Области применения шифров	9
Типы шифров	11
Ассиметричные шифры.....	14
Симметричные шифры	16
Параметры симметричных алгоритмов.....	18
Виды симметричных шифров	20
Ответы на вопросы.....	27
<i>Практическая часть выполнения</i>	41
Режимы алгоритма	43
Алгоритм ГОСТ Р 34.12-2018 Магма (ГОСТ 28147). Режим гаммирования с обратной связью (ОС).	48
Структура шифра	50
Криптоанализ.....	52
Сложность алгоритма (асимптотика).....	53
Схемы шифрования Магма (ГОСТ 28147-89).....	53
Схема структуры раунда выполнения алгоритма	53
Схема структуры раунда алгоритма с таблицей замен	54
Схема получения итерационных ключей.	55
Схема алгоритма при шифровании по раундам	56
Схема работы алгоритма при расшифровке по раундам	57
Блок-схема	58
Архитектура	60
Стек технологий	60

Реализация на С.....	61
main.c	61
cipher.c	61
magma.c	62
fileIO.c.....	64
kdf.c и sha256.c	66
ui.c	68
Заголовочные файлы.....	73
Реализация на Java	75
Main.java.....	75
MagmaCFB.java	78
HashUtils.java	80
FileUtils.java	82
StudentInfo.java	82
Демонстрация работы программной системы	82
Трассировка	92
Уязвимости режима гаммирования ОС Магмы	95
Верификация и борьба с уязвимостями	96
Достоинства стандарта	97
Критика стандарта.....	97
Заключение.....	98
Приложение. Листинг кода	100

Ход выполнения лабораторной работы

Теоретическая часть выполнения

Начиная с 2ого века до н. э. начались создаваться различные шифры. Знаменитый шифр Цезаря и шифр виженера, который плавно перерос в механические устройства, тем самым, став прародителем энигмы¹. Объединяет все эти шифры только одно: все они без исключения были взломаны. Вплоть до середины 20 века криптоаналитикам удавалось брать верх над любым «невзламываемым» шифром, который десятилетиями разрабатывался величайшими криптографами в надежде обеспечить безопасность передачи сообщения или данных. Благодаря криптоанализу за многие века была собрана колоссальная база методик по взлому ручных шифров, что в дальнейшем позволило расшифровать многие древние произведения человечества (египетские иероглифы, манускрипты), которые дали нам новые представления о событиях и жизни людей в те времена. От которых остались лишь слухи, легенды и чудеса света, которые люди не могут объяснить до сих пор.

Шифр (от фр. Chiffre «цифра») – это система обратимых преобразований, зависящая от некоторого секретного параметра (ключа) и предназначенная для обеспечения секретности передаваемой информации. Он может представлять собой совокупность условных знаков (условную азбуку из цифр, букв или определённых знаков) либо алгоритм преобразования обычных цифр или букв.

Процесс засекречивания сообщения с помощью шифра называется шифрованием. Наука о создании и использовании шифров называется *криптографией*. Шифр может представлять собой совокупность условных знаков (условная азбука из цифр, букв или определённых знаков) либо алгоритм преобразования обычных цифр и букв.

¹ «Энигма» (от нем. Enigma — загадка) — это переносная шифровальная машина, использовавшаяся для шифрования и дешифрования секретных сообщений. Трёхроторная военная немецкая шифровальная машина

Энигма

Первую версию роторной шифровальной машины запатентовал в 1918 году Артур Шербиус. На основе конструкции первоначальной модели «Энигмы» было создано целое семейство электромеханических роторных машин под тем же названием, применявшихся с 1920-х годов в сфере коммерческой и военной связи во многих странах мира. Впервые шифр «Энигмы» удалось дешифровать в польском «Бюро шифров» в декабре 1932 года.

Как и другие роторные машины, «Энигма» состояла из комбинации механических и электрических систем. Механическая часть включала в себя клавиатуру, набор вращающихся дисков (роторов), которые были расположены вдоль вала и прилегали к нему, и ступенчатого механизма,двигающего один или несколько роторов при каждом нажатии на клавишу. Электрическая часть, в свою очередь, состояла из электрической схемы, соединяющей между собой клавиатуру, коммутационную панель, лампочки и роторы (для соединения роторов использовались скользящие контакты). Конкретный механизм работы мог быть разным, но общий принцип был таков: при каждом нажатии на клавишу самый правый ротор сдвигается на одну позицию, а при определённых условиях сдвигаются и другие роторы. Движение роторов приводит к различным криптографическим преобразованиям при каждом следующем нажатии на клавишу на клавиатуре.

Шифрование

Ручная попытка создать тот шифр, который никогда не будет взломан, заключался в использовании ключа шифрования равным размеру исходного сообщения. Это решало проблему с взломом и на поиск ключа методом полного перебора могло уйти очень много времени. Однако и на само шифрование и расшифрование так же уходило много времени, что делало алгоритм бесполезным. Поиск ключа составил бы **1000!** перестановок ключа.

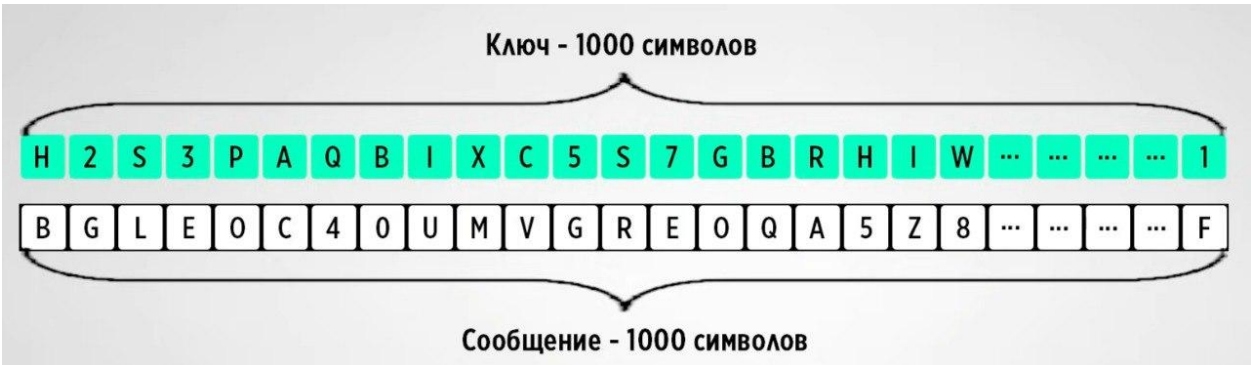


Рисунок 1. Алгоритм на основе ключа равного размеру сообщения.

Особо сильная потребность в шифрах появилась в XX веке, поскольку половину мира была погружена в войну и информация беспрепятственно передавалась по радиовышкам в виде радиоволн, что легко позволяло их перехватывать. Более того сейчас человечество практически всегда находится в некоторой информационной войне. Государства пытаются перехватить важные засекреченные данные или злоумышленники, которые пытаются украсть личные данные обычных людей, чтобы в дальнейшем закомпроментировать. Компьютер стал прорывом в области шифрования.

Отличие человека от компьютера в том, что люди оперируют буквами и символами, а компьютер оперирует двоичными цифрами.

65

СТАНДАРТ КОДИРОВАНИЯ СИМВОЛОВ

01000001	A
01000010	B
01000011	C
01000100	D
01000101	E
01000110	F
01000111	G
01001000	H
01001001	I
01001010	J
01001011	K
01001100	L
01001101	M
01001110	N
01001111	O
01010000	P

Рисунок 2. Сопоставление ASCII.

Любая информация при шифровании будет переведена в двоичный код, а затем преобразуется. Чтобы это стало возможным, каждому текстовому символу были придуманы числовые значения. Так был введен **важный стандарт кодирования символов ASCII**. Английская буква А хранится как 65 в таблице. Какое-то сообщение в ходе преобразования станет длинной строкой единиц и нулей.

Стандартная часть таблицы ASCII											
№ п/п	символ	двоичный код	№ п/п	символ	двоичный код	№ п/п	символ	двоичный код	№ п/п	символ	двоичный код
32	пробел	00100000	56	8	00111000	80	P	01010000	104	h	01101000
33	!	00100001	57	9	00111001	81	Q	01010001	105	i	01101001
34	"	00100010	58	:	00111010	82	R	01010010	106	j	01101010
35	#	00100011	59	;	00111011	83	S	01010011	107	k	01101011
36	\$	00100100	60	<	00111100	84	T	01010100	108	l	01101100
37	%	00100101	61		00111101	85	U	01010101	109	m	01101101
38	&	00100110	62	>	00111110	86	V	01010110	110	n	01101110
39	'	00100111	63	?	00111111	87	W	01010111	111	o	01101111
40	(	00101000	64	@	01000000	88	X	01011000	112	p	01110000
41	)	00101001	65	A	01000001	89	Y	01011001	113	q	01110001
42	*	00101010	66	B	01000010	90	Z	01011010	114	r	01110010
43	+	00101011	67	C	01000011	91	[	01011011	115	s	01110011
44	,	00101100	68	D	01000100	92	\	01011100	116	t	01110100
45	-	00101101	69	E	01000101	93	]	01011101	117	u	01110101
46	.	00101110	70	F	01000110	94	^	01011110	118	v	01110110
47	/	00101111	71	G	01000111	95	_	01011111	119	w	01110111
48	0	00110000	72	H	01001000	96	'	01100000	120	x	01111000
49	1	00110001	73	I	01001001	97	a	01100001	121	y	01111001
50	2	00110010	74	J	01001010	98	b	01100010	122	z	01111010
51	3	00110011	75	K	01001011	99	c	01100011	123	{	01111011
52	4	00110100	76	L	01001100	100	d	01100100	124		01111100
53	5	00110101	77	M	01001101	101	e	01100101	125	}	01111101
54	6	00110110	78	N	01001110	102	f	01100110	126	~	01111110
55	7	00110111	79	O	01001111	103	g	01100111	127		01111111

Рисунок 3. Таблица ASCII.

Несмотря на то, что произошла цифровизация технологий, методы шифрования несколько не поменялись. **Метод замены, перестановки и их комбинация** – это по-прежнему единственные способы, что-либо зашифровать. Только, если раньше переставляли символы, то сейчас переставляются биты в букве или в двух отдельных буквах. А метод замены стал примитивнее, ведь мы меняем единицу на ноль и ноль на единицу. Всё это напоминает битовую операцию **XOR**. Этим не ограничимся и мы можем для

двоичных чисел применять любые математические битовые операции типа сдвига, которые будут приводить к полному изменению исходной информации, что открывает новый необъятный мир криптографии.

$$\begin{array}{cccc} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ \hline 1 & 1 & 0 & 0 \end{array} \quad \text{или} \quad 1010 \ll 10100$$

Важным параметром любого шифра является **ключ** — параметр криптографического алгоритма, обеспечивающий выбор одного преобразования из совокупности преобразований, возможных для этого алгоритма. В современной криптографии предполагается, что вся секретность криптографического алгоритма сосредоточена в ключе, но не деталях самого алгоритма (*принцип Керкгоффса*).

Не стоит путать шифр с кодированием — фиксированным преобразованием информации из одного вида в другой. В последнем отсутствует понятие ключа и не выполняется принцип Керкгоффса. В наше время кодирование практически не используется для защиты информации от несанкционированного доступа, а лишь от ошибок при передаче данных (помехоустойчивое кодирование) и других целях, не связанных с защитой.

Области применения шифров

1. Шифры применяются для *тайной переписки* дипломатических представителей со своими правительствами.
2. В государственной и военной сфере шифры исторически играли ключевую роль в защите государственной тайны, дипломатической переписки.
3. В вооружённых силах для передачи текста секретных документов по техническим средствам связи и для военных коммуникаций. Современные армии используют криптографию для защиты систем управления войсками, разведывательных данных и беспилотных аппаратов.

4. В экономике и банковской системе шифры используются для обеспечения безопасности транзакций, а также некоторыми интернет-сервисами для защиты денежных средств клиентов при производстве онлайн-платежей и банковских операций. Банковские карты в том числе подлежат шифрованию и защите информации и данных.

5. Мобильная связь также зависит от шифрования, поскольку стандарты GSM/LTE используют криптоалгоритмы для защиты голосовой связи и мобильного интернета.

6. В компьютерной (информационной) безопасности шифры используются для защиты данных на дисках (шифрование SSD, жестких дисков).

7. Шифры используются в науке, которые изучают создание и использование шифров – это *криптографией*. *Криптоанализ* — это наука о методах получения исходного значения зашифрованной информации.

8. Обеспечение конфиденциальности переписки в мессенджерах и социальных сетях.

9. Интернет-коммуникации полностью на шифрах: протокол HTTPS защищает соединения с сайтами, VPN создают зашифрованные туннели для удаленной работы, а SSL/TLS сертификаты обеспечивают безопасность электронной коммерции.

10. Цифровая идентификация и электронные подписи основаны на асимметричном шифровании, обеспечивая юридическую значимость электронных документов.

11. В криптовалютах и блокчейне шифры лежат в основе создания кошельков, подписи транзакций и обеспечения неизменности распределенного реестра.

12. В системах «умного дома», где используются роботы-пылесосы, чайники, стиральные машины и другие девайсы с дистанционным управлением – все обеспечиваются криптографическими алгоритмами для

защиты от несанкционированного доступа и обеспечения безопасного функционирования.

13. Сигнализация автомобиля или какой-то системы безопасности

Области применения шифров охватывают практически все сферы современной жизни, где требуется защита информации.

Типы шифров

Шифры разделяют по количеству ключей шифрования и расшифрования. Могут использовать один ключ для шифрования и расшифрования или два различных ключа. **По типу ключа:**

- ***Симметричный шифр*** использует один ключ для шифрования и расшифрования. Для кодирования и расшифровки информации используется один и тот же открытый ключ, доступ к которому может получить любой пользователь. Такой способ достаточно уязвим с точки зрения безопасности данных, поэтому он чаще применяется не для передачи, а для хранения информации.

- ***Асимметричный шифр*** использует два различных ключа. Для кодирования и дешифровки используются разные ключи. При этом ключ, который нужен для разгадывания кода, а закрытый, им владеет только нужный получатель. Такой вид шифрования информации считается более надёжным, и его алгоритмы применяются, например, в системе цифровых подписей и блокчейне.

- ***Гибридный шифр***, который включает в себя оба вышеописанных метода. Сначала сообщение шифруется с помощью открытого ключа, затем этот ключ скрывается с помощью ещё одного открытого ключа, но для его расшифровки необходим закрытый, известный только получателю

Шифры могут быть сконструированы так, чтобы либо шифровать сразу весь текст, либо шифровать его по мере поступления. **По типу блочности:**

- **Блочный шифр** шифрует сразу целый блок текста, выдавая шифротекст после получения всей информации.

- **Поточный шифр** шифрует информацию и выдаёт шифротекст по мере поступления, таким образом, имея возможность обрабатывать текст неограниченного размера, используя фиксированный объём памяти.

Блочный шифр можно превратить в поточный, разбивая входные данные на отдельные блоки и шифруя их по отдельности.

Также существуют неиспользуемые сейчас *подстановочные шифры*, обладающие (в своём большинстве) слабой криптостойкостью.

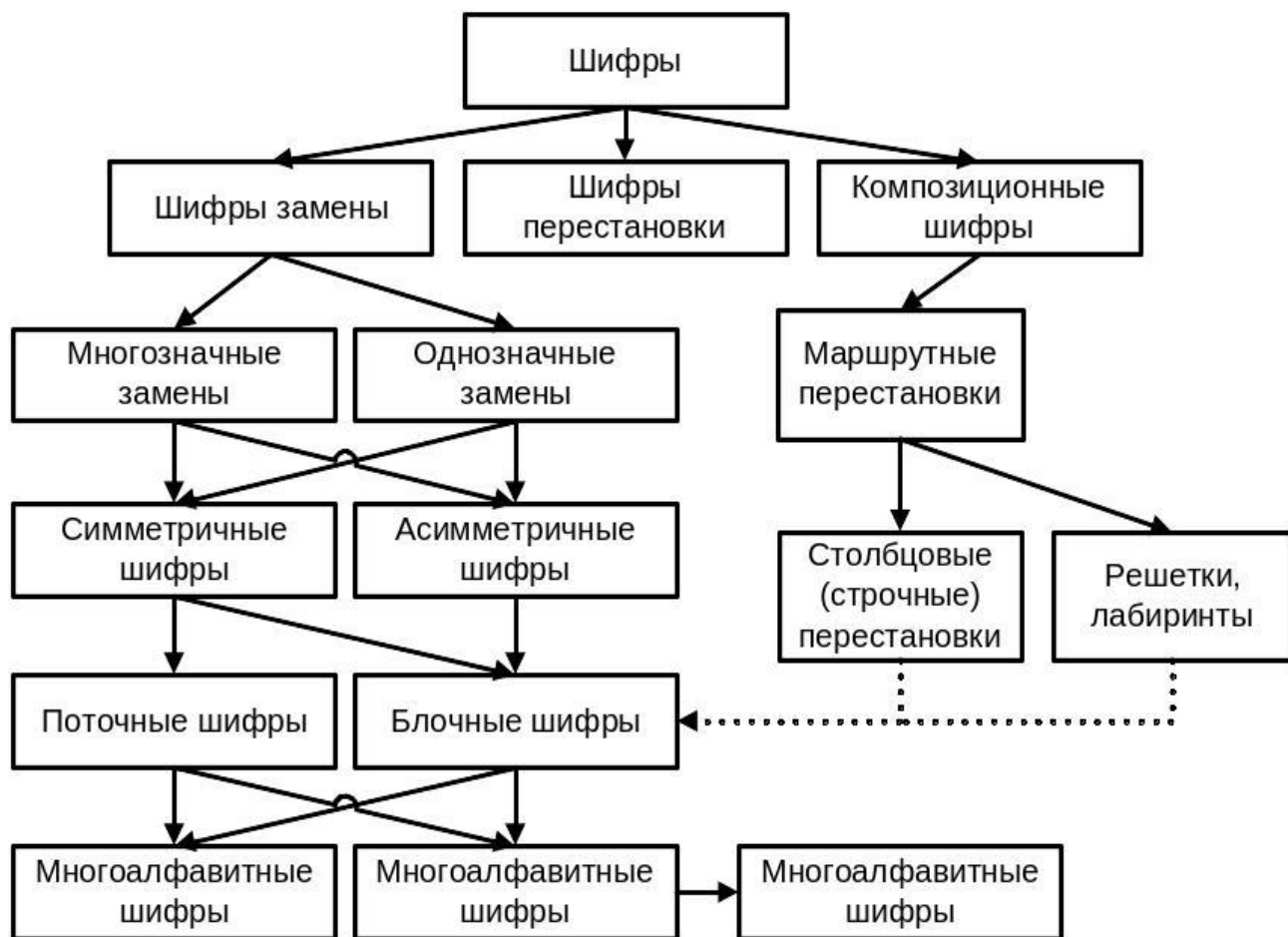


Рисунок 4. Типы шифров.

Шифры замены – этот тип шифров заменяет символы в открытом тексте на другие символы. Данный способ разделяют на следующие типы:

- Многозначные замены – каждый символ заменяется на несколько символов, что позволяет повысить безопасность.

- Однозначные замены – каждый символ заменяется на единственный другой символ (примером служит Шифр Цезаря, где каждый символ сдвигается на фиксированное количество позиций в алфавите).

Шифры перестановки – этот тип шифрования изменяет порядок символов в сообщении, но не меняют сами символы (транспозиционные шифры, где меняются местами буквы, но их содержание остаётся тем же).

- Маршрутные перестановки – это шифры, в которых используются определённые маршруты перестановки (столбцовые шифры или лабиринты).

1. Столбцовые (строчные) перестановки – символы текста записываются в столбцы и затем читаются по строкам.

2. Решётки, лабиринты – это шифры используют решётки или лабиринты для того, чтобы переставлять символы и скрывать сообщение.

Композиционные шифры – это шифры, которые используют комбинацию нескольких шифров для повышения безопасности. Комбинируются разные методы шифрования, чтобы создать более сложную систему защиты.

- Многоалфавитные шифры – это метод, в котором используется несколько алфавитов для шифрования, что делает шифр более сложным для взлома, так как невозможно найти единственный способ подбора ключа.

Поточные и блочные шифры мы рассмотрели выше, а симметричные и асимметричные шифры мы рассмотрим далее, поскольку в них присутствует много особенностей и зачастую при работе с шифрованием, рассматриваются именно эти типы.

Классификация шифров

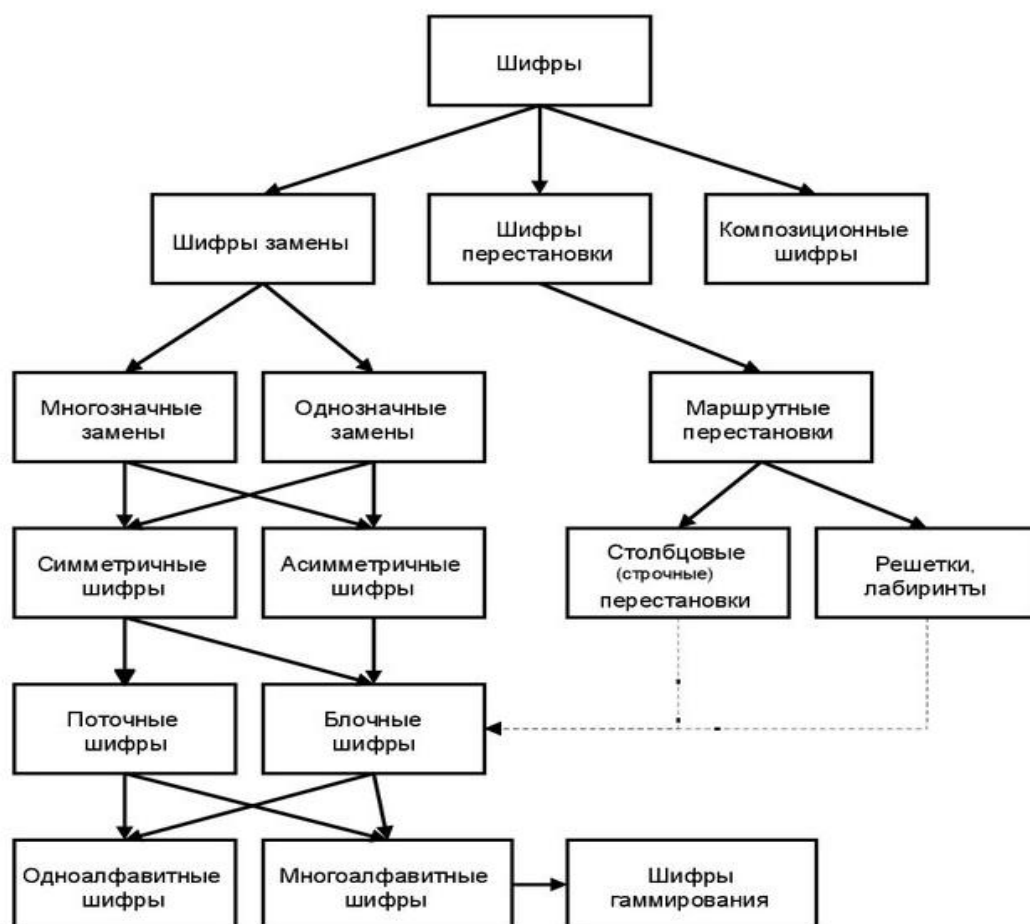


Рисунок 5. Классификация шифров.

Асимметричные шифры

Асимметричный шифр – это метод шифрования, при котором предполагается использование сразу двух ключей шифрования. Также это система шифрования и/или *электронной цифровой подписи (ЭЦП)*², при которой открытый ключ передаётся по открытому (незащищённому, доступному для наблюдению) каналу, и используется для проверки ЭЦП и для шифрования сообщения. Для генерации ЭЦП и для расшифровки сообщения используется **секретный ключ**. Криптографические системы с

²Электронная цифровая подпись (ЭЦП) – это реквизит электронного документа, полученный в результате криптографического преобразования информации (набор знаков и паролей) с использованием закрытого ключа подписи, который позволяет подтвердить подлинность и авторство электронного документа. Подпись связана и с автором, и с самим документом с помощью криптографических методов и не может быть подделана с помощью обычного копирования.

открытым ключом в настоящее время широко применяются в различных сетевых протоколах: в протоколах TLS и его предшественнике SSL (лежащих в основе HTTPS), в SSH. Также используются в PGP, S/MIME

- **RSA** (абб. от фамилий Rivest, Shamir и Adleman) — это криптографический алгоритм с открытым ключом, основывающийся на вычислительной сложности задачи факторизации больших полупростых чисел. Криптосистема RSA стала первой системой, пригодной и для шифрования и цифровой подписи. Алгоритм используется в большом числе криптографических приложений, включая PGP, S/MIME, TLS/SSL, IPsec/IKE.

Система RSA используется для защиты программного обеспечения и в схемах цифровой подписи. Используется в открытой системе шифрования PGP в сочетании с симметричными алгоритмами. Обладает низкой скоростью шифрования сообщения обычно шифруют с помощью более производительных симметричных алгоритмов со случайным сеансовым ключом (AES, IDEA, Serpent, Twofish), а с помощью RSA шифруют лишь этот ключ, таким образом реализуется гибридная криптосистема. Такой механизм содержит потенциальные уязвимости ввиду необходимости использовать криптографически стойкий генератор псевдослучайных чисел для формирования случайного сеансового ключа симметричного шифрования.

- **Elgamal** – это криптосистема с открытым ключом, основанная на трудности вычисления дискретных логарифмов в конечном поле. Криптосистема включает в себя алгоритм шифрования и алгоритм цифровой подписи. Схема Эль-Гамала лежит в основе бывших стандартов электронной цифровой подписи в США (DSA) и России (ГОСТ Р 34.10-94)

- **Elliptic curve cryptography, ECC** — (криптосистема на основе эллиптических кривых) – это форма криптографии с открытым ключом, основанная на математике эллиптических кривых. Это один из видов

асимметричного шифрования, который опирается на два отдельных ключа: открытый, которым можно делиться с другими, и закрытый, который нужно держать в секрете.

Эллиптическая криптография — это раздел криптографии, который изучает асимметричные криптосистемы, основанные на эллиптических кривых над конечными полями. Основное преимущество эллиптической криптографии заключается в том, что на сегодняшний день неизвестны субэкспоненциальные алгоритмы дискретного логарифмирования.

Симметричные шифры

Симметричные криптосистемы (симметричное шифрование, симметричные шифры) (англ. symmetric-key algorithm) — это способ шифрования, в котором для шифрования и расшифрования применяется один и тот же криптографический ключ. До изобретения схемы асимметричного шифрования единственным существовавшим способом являлось симметричное шифрование. Ключ алгоритма должен сохраняться в тайне обеими сторонами, должны осуществляться меры по защите доступа к каналу на всем пути следования криптограммы или сторонами взаимодействия посредством криптообъектов, сообщений, если данный канал взаимодействия под грифом «Не для использования третьими лицами». Алгоритм шифрования выбирается сторонами до начала обмена сообщениями.

Классическими примерами таких алгоритмов являются симметричные криптографические алгоритмы, перечисленные ниже:

- **Простая перестановка без ключа** — это один из простых методов шифрования. Сообщения записывается в таблицу по столбцам. После того, как открытый текст записан колонками, для образования шифртекста он считывается по строкам. Для использования этого шифра отправителю и получателю нужно договориться об общем ключе в виде размера таблицы.

Объединение букв в группы не входит в ключ шифра и используется лишь для удобства записи не смыслового текста.

- **Одиночная перестановка по ключу** – это более практический метод шифрования, называемый одиночной перестановкой по ключу, очень похож на предыдущий. Он отличается лишь тем, что колонки таблицы переставляются по ключевому слову, фразе или набору чисел длиной в строку таблицы.

- **Двойная перестановка** – метод, при котором для дополнительной скрытности можно повторно шифровать сообщение, которое уже было зашифровано. Для этого размер второй таблицы подбирают так, чтобы длины её строк и столбцов отличались от длин в первой таблице. Лучше всего, если они будут взаимно простыми и, кроме того, в первой таблице можно переставлять столбцы, а во второй строки. Наконец, можно заполнять таблицу зигзагом, змейкой, по спирали или каким-то другим способом. Такой метод делает процесс дешифрования чуть дольше и сложнее.

- **Перестановка «Магический квадрат»** – это метод при котором используются квадратные таблицы со вписанными в их клетки последовательными натуральными числами от 1, которые дают в сумме по каждому столбцу, каждой строке и каждой диагонали одно и то же число. Подобные квадраты широко применяются для вписывания шифруемого текста по приведённой в них нумерации. Если потом выписать содержимое таблицы по строкам, то получается шифровка перестановкой букв. На первый взгляд кажется, будто магических квадратов очень мало. Тем не менее, их число очень быстро возрастает с увеличением размера квадрата.

Так, существует лишь один магический квадрат размером 3 x 3. Магических квадратов 4 x 4 насчитывается уже 880, а число магических квадратов размером 5 x 5 около 250000. Поэтому магические квадраты больших размеров могли *быть хорошей основой для надежной системы шифрования*, потому что ручной перебор всех вариантов ключа для этого

шифра огромен. В квадрат размером 4 на 4 вписываются числа от 1 до 16. Его магия состоит в том, что сумма чисел по строкам, столбцам и полным диагоналям равняется одному и тому же числу — 34. Впервые эти квадраты появились в Китае, где им и была приписана некоторая «магическая сила».

В настоящее время симметричные шифры — это:

1. **Блочные шифры.** Обработывают информацию блоками определённой длины (обычно 64, 128 бит), применяя к блоку ключ в установленном порядке, как правило, несколькими циклами перемешивания и подстановки, называемыми раундами. Результатом повторения раундов является лавинный эффект — это нарастающая потеря соответствия битов между блоками открытых и зашифрованных данных.

2. **Поточные шифры,** в которых шифрование проводится над каждым битом либо байтом исходного (открытого) текста с использованием гаммирования³. Поточный шифр может быть легко создан на основе блочного (например, ГОСТ 28147-89 в режиме гаммирования), запущенного в специальном режиме.

Параметры симметричных алгоритмов

Существует много алгоритмов симметричных шифров и у них есть свои характеристики, которые как-то классифицируют шифрование, существенными параметрами которых являются:

- Стойкость — самый главный параметр, который определяет, насколько способен алгоритм противостоять различным атакам.
- Длина ключа — количественный параметр, который напрямую влияет на стойкость к атаке полного перебора (brute-force). Размер

³ Гаммирование, или Шифр XOR, — метод симметричного шифрования, заключающийся в «наложении» последовательности, состоящей из случайных чисел, на открытый текст. Последовательность случайных чисел называется *гамма-последовательностью* и используется для зашифровывания и расшифровывания данных.

секретного ключа в битах. Эффективная длина ключа может быть меньше номинальной, у DES номинальная длина ключа 64 бита, но из-за «битов чётности» эффективная длина – 56 бит.

- Число раундов – параметр обеспечивающий «перемешивание» данных и ключа до необходимого уровня. Блочные шифры (AES, DES) работают циклически, повторяя одну и ту же последовательность операций (раунд) несколько раз. Многократное повторение обеспечивает лавинный эффект и устойчивость к специализированным атакам.

- Длина обрабатываемого блока – определяет единицу данных, которую алгоритм шифрует за одну операцию. Размер блока входных/выходных данных в битах. Исторически стандартом были 64 бита (DES). Современный стандарт — 128 бит (AES). При шифровании большого количества данных для некоторых режимов возрастает вероятность найти 2 одинаковых блока шифротекста⁴, что может раскрыть информацию (64-битного блока (DES) = 2^{32} , для 128-битного блока (AES) = 2^{64}). Сложность реализации – буферы памяти и разрядность процессора.

- Сложность аппаратной/программной реализации – практический параметр, определяющий скорость работы и стоимость использования алгоритма. Оценка ресурсов, необходимых для выполнения алгоритма (времени, объёма памяти, энергопотребления). Метрики аппаратной реализации (параметры "железа": микросхемы, кристаллы, частота)

- Сложность преобразования – качественный параметр, описывающий внутреннее устройство алгоритма. Математическая и структурная сложность базовых операций, из которых состоит каждый раунд шифрования. Напрямую определяет стойкость и сложность реализации. Простые и элегантные преобразования (как в AES) предпочтительнее, так как их легче анализировать на предмет уязвимостей.

⁴ **Шифротекст, шифртекст** — результат операции шифрования. Часто также используется вместо термина «криптограмма», хотя последний подчёркивает сам факт передачи сообщения, а не шифрования. Процесс применения операции шифрования к шифротексту называется перешифровкой.

Виды симметричных шифров

блочные шифры:

1. AES (англ. Advanced Encryption Standard) — американский стандарт шифрования, симметричный алгоритм блочного шифрования (размер блока 128 бит, ключ 128/192/256 бит), принятый в качестве стандарта шифрования правительством США по результатам конкурса AES.

2. ГОСТ 28147-89 (Магма) — советский и российский стандарт шифрования, также является стандартом СНГ

3. DES (англ. Data Encryption Standard) — официальный стандарт (FIPS 46-3). шифрования данных в США. Алгоритм для симметричного шифрования, разработанный фирмой IBM и утверждённый в 1977 году. Размер блока для DES равен 64 битам. В основе алгоритма лежит сеть Фейстеля с 16 циклами (раундами) и ключом, имеющим длину 56 бит. Алгоритм использует комбинацию нелинейных (S-блоки) и линейных (перестановки E, IP, IP-1) преобразований.

4. 3DES (Triple-DES, тройной DES) – симметричный блочный шифр, созданный в 1978 году на основе алгоритма DES с целью устранения главного недостатка последнего — малой длины ключа (56 бит), который может быть взломан методом полного перебора ключа.

5. RC2 (Шифр Ривеста (Rivest Cipher или Ron's Cipher)) – блочный шифр с длиной блока 64 бита и переменной длиной ключа, разработанный Роном Ривестом в конце 1980-х годов. Алгоритм является более быстрым, чем алгоритм DES.

6. RC5 – блочный шифр, разработанный Роном Ривестом с переменным количеством раундов, длиной блока и длиной ключа. Это расширяет сферу использования и упрощает переход на более сильный вариант алгоритма.

7. RC6 – симметричный блочный криптографический алгоритм, производный от алгоритма RC5. Шифр RC6 был заявлен на конкурс AES. Поддерживает блоки длиной 128 бит и ключи длиной 128, 192 и 256 бит, но сам алгоритм, как и RC5, может быть сконфигурирован для поддержки более

широкого диапазона длин как блоков, так и ключей (от 0 до 2040 бит). RC6 похож на RC5 по своей структуре и также довольно прост в реализации.

8. Blowfish – это криптографический алгоритм, реализующий блочное симметричное шифрование с переменной длиной ключа. Разработан Брюсом Шнайером в 1993 году. Представляет собой сеть Фейстеля. Выполнен на простых и быстрых операциях: XOR, подстановка, сложение. Является незапатентованным и свободно распространяемым.

9. Twofish – это симметричный алгоритм блочного шифрования с размером блока 128 бит и длиной ключа до 256 бит. Число раундов 16. Разработан группой специалистов во главе с Брюсом Шнайером.

10. NUSH – блочный алгоритм симметричного шифрования, разработанный Анатолием Лебедевым и Алексеем Волчковым для российской компании LAN Crypto. Алгоритм не использует S-блоки, а только такие операции, как AND, OR, XOR, сложение по модулю и циклические сдвиги. Перед первым и после последнего раунда проводится «отбеливание» ключа.

11. IDEA (International Data Encryption Algorithm, международный алгоритм шифрования данных) – это симметричный блочный алгоритм шифрования данных, запатентованный швейцарской фирмой Ascom. IDEA использует 128-битный ключ и 64-битный размер блока, открытый текст разбивается на блоки по 64 бит.

12. CAST (по инициалам разработчиков Carlisle Adams и Stafford Tavares) – это блочный алгоритм симметричного шифрования на основе сети Фейстеля.

13. CAST-128 – блочный алгоритм симметричного шифрования на основе сети Фейстеля, который используется в целом ряде продуктов криптографической защиты. CAST-128 состоит из 12 или 16 раундов сети Фейстеля с размером блока 64 бита и длиной ключа от 40 до 128 бит (но только с инкрементацией по 8 бит).

14. CAST-256 – блочный алгоритм симметричного шифрования на основе сети Фейстеля, опубликованный в июне 1998 года. CAST-256

построен из тех же элементов, что и CAST-128, включая S-боксы, но размер блока увеличен вдвое и равен 128 битам. Это влияет на диффузионные свойства и защиту шифра. Алгоритм шифрует информацию 128-битными блоками и использует несколько фиксированных размеров ключа шифрования: 128, 160, 192, 224 или 256 битов.

15. CRAB – это блочный шифр, разработанный Бартом Калиски и Мэттом Робшоу из лаборатории RSA на первом семинаре FSE в 1993. Crab был разработан, чтобы продемонстрировать, как идеи хэш-функций могут быть использованы для создания быстрого шифрования. Алгоритм шифрования очень похож на MD5.

16. 3-WAY – симметричный блочный шифр с закрытым ключом, разработанный Йоаном Дайменом, одним из авторов алгоритма Rijndael (иногда называемого AES). Алгоритм 3-Way является 11-шаговой SP-сетью. Используются блок и ключ длиной 96 бит. Схема шифрования, как и характерно для алгоритмов типа SP-сеть, предполагает эффективную аппаратную реализацию. Вскоре после опубликования был проведён успешный криптоанализ алгоритма 3-Way, показавший его уязвимость для атаки на основе связанных ключей. Алгоритм не запатентован.

17. Khufu и Khafre – два блочных шифра в криптографии, разработанные Ральфом Мерклом в 1989 году, названные в честь египетских фараонов.

18. Khufu — 64-битный блочный шифр, в котором, что необычно, используются ключи размером 512 бит. Блочные шифры обычно используют ключи гораздо меньшего размера, редко превышающие 256 бит. Поскольку настройка ключа занимает много времени, Khufu не подходит для ситуаций, когда обрабатывается много небольших сообщений. Khufu – это шифр Фейстеля с 16 раундами по умолчанию (допускаются значения, кратные восьми, от 8 до 64).

19. Khafre – похож на Хуфу, но использует стандартный набор S-блоков и не вычисляет их на основе ключа. (они генерируются из таблиц RAND, которые используются в качестве источника "случайных чисел")

Преимущество Хефрена в том, что он может очень быстро зашифровать небольшой объём данных (хорошая *гибкость ключа*). Однако для достижения того же уровня безопасности, что и у Хуфу, Хефрену, вероятно, требуется большее количество раундов, что делает его более медленным при массовом шифровании. Хафре использует ключ, размер которого кратен 64 битам. Поскольку S-блоки не зависят от ключа, Хафре выполняет операцию XOR с подключами каждые восемь раундов.

20. Кузнечик (Kuznechik) – симметричный алгоритм блочного шифрования с размером блока 128 бит и длиной ключа 256 бит, использующий для генерации раундовых ключей SP-сеть. Шифр утверждён ГОСТом наряду с алгоритмом "Магма".

21. DESX – симметричный алгоритм шифрования, разработанный на основе блочного шифра DES в 1984 году. Данный алгоритм использует метод отбеливания ключа с целью усиления устойчивости к атакам на основе полного перебора. Хотя длина ключа алгоритма DESX — 184 бита, эффективная длина ключа составляет $56 + 64 - 1 - \log_2(M) = 119 - \log_2(M) = \sim 119$ бит, где M — число известных пар открытый текст/шифротекст.

22. Serpent (с лат. "змея") – симметричный блочный алгоритм шифрования. Имеет размер блока 128 бит и возможные длины ключа 128, 192 или 256 бит. Алгоритм представляет собой 32-раундовую SP-сеть, работающую с блоком из четырёх 32-битных слов. Serpent был разработан так, что все операции могут быть выполнены параллельно, используя 32 1-битных «потока».

23. SAFER (англ. Secure And Fast Encryption Routine — безопасная и быстрая процедура шифрования) — в криптографии семейство симметричных блочных криптоалгоритмов на основе подстановочно-перестановочной сети. Первый вариант шифра был создан и опубликован в 1993 году. Длина шифруемого блока и длина ключа равны 64 битам.

24. TEA (Tiny Encryption Algorithm) – блочный алгоритм шифрования типа «Сеть Фейстеля». Алгоритм был разработан в 1994 году. Шифр не патентован, широко используется в ряде криптографических приложений и

широком спектре аппаратного обеспечения благодаря крайне низким требованиям к памяти и простоте реализации. Алгоритм имеет программную реализацию на разных языках программирования и аппаратную реализацию на интегральных схемах типа FPGA. Алгоритм шифрования TEA основан на битовых операциях с 64-битным блоком, имеет 128-битный ключ шифрования. Стандартное количество раундов сети Фейстеля равно 64.

25. FROG – блочный шифр, разработанный Жоргудисом, Леру и Шаве. Алгоритм может работать с блоками любого размера от 8 до 128 байт и поддерживает ключи размером от 5 до 125 байт. Алгоритм состоит из 8 раундов и имеет очень сложное распределение ключей. Алгоритм симметричного блочного шифрования с неортодоксальной структурой, один из участников американского конкурса AES, разработка коста-риканской компании TecApro Internacional, был создан 1998 году.

26. MARS – является блочно-симметричным шифром с секретным ключом. Размер блока при шифровании 128 бита, размер ключа может варьироваться от 128 до 448 бит включительно (кратные 32 битам). Создатели стремились совместить в своём алгоритме быстроту кодирования и стойкость шифра. В результате получился один из самых криптостойких алгоритмов, участвовавших в конкурсе AES. В шифре используются работа с 32-битными словами, сеть Фейстеля и симметричность алгоритма. Алгоритм уникален тем, что использует все технологии: простейшие операции (сложение, вычитание, XOR), подстановки с использованием таблицы замен, фиксированный или зависимый от данных циклический сдвиг и ключевое отбеливание.

27. Skipjack – блочный шифр, разработанный Агентством национальной безопасности США в рамках проекта Capstone. После разработки сведения о шифре были засекречены. Изначально он предназначался для использования в чипе Clipper для защиты аудиоинформации, передаваемой по сетям правительственной телефонной связи, а также по сетям мобильной и беспроводной связи. Позже алгоритм был рассекречен. Skipjack использует

80-битный ключ для шифрования/дешифрования 64-битных блоков данных. Это несбалансированная сеть Фейстеля с 32 раундами.

потокосые шифры

1. RC4 (алгоритм шифрования с ключом переменной длины) – потоковый шифр, применяющийся в системах защиты информации в компьютерных сетях (например, в протоколах SSL и TLS, алгоритмах обеспечения безопасности беспроводных сетей WEP и WPA). Шифр разработан компанией RSA Security, и для его использования требуется лицензия. Алгоритм RC4 (потокосый шифр) строится на основе генератора псевдослучайных битов. На вход генератора записывается ключ, а на выходе читаются псевдослучайные биты. Длина ключа может составлять от 40 до 2048 бит. Генерируемые биты имеют равномерное распределение. Ядро алгоритма поточных шифров состоит из функции (генератора псевдослучайных битов или гаммы), который выдаёт поток битов ключа (ключевой поток, гамму, последовательность псевдослучайных битов).

2. SEAL (Software-optimized Encryption Algorithm, программно-эффективный алгоритм шифрования) — симметричный поточный алгоритм шифрования данных, оптимизированный для программной реализации. Алгоритм оптимизирован и рекомендован для 32-битных процессоров. Для работы ему требуется кэш-память на несколько килобайт и восемь 32-битовых регистров. Скорость шифрования – примерно 4 машинных такта на байт текста. Для кодирования и декодирования используется 160-битный ключ.

3. WAKE (World Auto Key Encryption algorithm, алгоритм шифрования на автоматическом ключе) – алгоритм, разработанный Дэвидом Уилером в 1993 году. Изначально проектировался как среднескоростная система шифрования (скорость в поточных шифрах измеряется в циклах на байт шифруемого открытого текста) блоков (минимального количества информации, которое может быть обработано алгоритмом; в данном случае блок составляет 32 бита), обладающая высокой безопасностью. По мнению

автора, является простым алгоритмом быстрого шифрования, подходящим для обработки больших массивов данных, а также коротких сообщений, где изменяется только секретный ключ. Подходит для использования хэшей на секретных ключах, используемых при верификации.

4. A5 – поточный алгоритм шифрования, используемый для обеспечения конфиденциальности передаваемых данных между телефоном и базовой станцией в европейской системе мобильной цифровой связи GSM. Шифр основан на побитовом сложении по модулю два (булева операция XOR) генерируемой псевдослучайной последовательности и шифруемой информации. В A5 псевдослучайная последовательность реализуется на основе трёх линейных регистров сдвига с обратной связью. Регистры имеют длины 19, 22 и 23 бита соответственно. Сдвигами управляет специальная схема, организующая на каждом шаге смещение как минимум двух регистров, что приводит к их неравномерному движению. Последовательность формируется путём операции XOR ("исключающее или") над выходными битами регистров. В этом алгоритме каждому символу открытого текста соответствует символ шифротекста. Текст не делится на блоки и не изменяется в размере.

5. Mosquito – аппаратно-ориентированный самосинхронизирующийся поточный шифр, разработанный в 2005 году Йоаном Дайменом и Парисом Китсосом. После взлома MOUSQUITO, была разработана вторая версия шифра, которая была представлена в проекте eSTREAM, где достигла третьего этапа отбора. В 2008 году вторая версия MOSQUITO — MOUSTIQUE, также была взломана. Общий смысл работы самосинхронизирующегося поточного шифра MOSQUITO аналогичен работе самосинхронизирующихся поточных шифров, в которых генерация потока ключей создаётся функцией от битов ключа и одного бита шифротекста, что, по сути, аналогично работе CFB с одним блоком перестановки. Особенности же шифра MOSQUITO заключаются в наличии 9 стадий конвейера, дополняющего условную зависимость регистра сдвига (Conditional Complementing Shift Registers – CCSR) и функциями перехода между стадиями конвейера особого вида.

Ответы на вопросы

1. Симметричные алгоритмы шифрования и их характеристики.

Симметричные алгоритмы шифрования – это алгоритмы, в которых используется один и тот же ключ для шифрования и для расшифрования данных. Главная цель – конфиденциальность больших объёмов данных. Также под симметричным шифрованием понимают криптографические системы, в которых так же для шифрования и расшифрования используется один и тот же ключ (**секретный ключ**).



Рисунок 6. Симметричное шифрование.

Симметричный ключ (секретный) — одна строка/блок битов, который нужно хранить в секрете. Хранение и безопасная передача — основная проблема. Утрата секретного ключа приводит к дыре в безопасности.

Это означает, что отправитель и получатель должны обладать одинаковым ключом, который должен храниться в секрете.

Основные характеристики:

1. Один ключ (должен быть секретным и известен обеим сторонам).

2. Высокая скорость (быстрее асимметричных алгоритмов и подходит для больших данных).

3. Типы симметричных шифров: **блочные** или **block cipher** (AES, ГОСТ-Магма, Кузнечик, DES) и **поточные** или **stream cipher** (RC4).

4. Режимы шифрования (для блочных) – ECB, CBC, CFB, OFB, CTR.

5. Масштабирование – проблема при распределении ключей для большого числа пользователей (N пользователей приводит к $O(N^2)$).

6. Аутентификация/целостность – симметричный шифр сам по себе не даёт целостность. Для этого используют аутентифицированные режимы или отдельный MAC.

7. Длина ключа – обычно 128, 192 или 256 бит.

8. Безопасность (обеспечивает конфиденциальность при условии, что ключ секретен, а также зависит от длины ключа и структуры алгоритма). Алгоритмы уязвимы, если ключ скомпрометирован. Главный недостаток в том, что нужна безопасная передача ключа между участниками. Требуется отдельный канал или асимметричной криптосистемы для обмена. Стойкость к известным атакам (коллизии/дифференциальный/линейный криптоанализ).



Рисунок 7. Работа шифрования относительно отправителя и получателя.

Применение: конфиденциальность объёмов данных (VPN, шифрование диска, файловая защита), быстрые каналы связи, шифрование больших сообщений. В закрытых системах с управлением секретами, где ключи безопасно распределены. В беспроводной связи и TLS (как часть гибридной схемы).

2. Ассиметричные алгоритмы шифрования и их характеристики.

Асимметричные алгоритмы шифрования (публично-ключевые, РКС) – это шифрование, при котором алгоритмы используют пару ключей — публичный/открытый (Public key) и приватный (Private key).

То, что зашифровано одним ключом, может быть расшифровано только другим ключом из пары.

- **Публичный ключ** – может быть открыт всем и свободно опубликован, используется для шифрования сообщений получателю или проверки подписи. Может передаваться по незащищённым каналам.
- **Приватный ключ** – должен храниться в секрете у владельца; используется для расшифровки сообщений, предназначенных для владельца, или для создания подписи.



Рисунок 8. Асимметричное шифрование.

Шифрование осуществляется открытым ключом, он шифрует и только закрытый ключ может расшифровать.

Подпись осуществляется закрытым ключом, подписывается хэш и открытым ключом осуществляется проверка этой подписи.

Основные характеристики:

1. Пара ключей (публичный/приватный – public/private)
2. Длина ключа значительно больше (RSA 2048-4096 бит, а для ECC в 256-512 бит)
3. Производительность и скорость. В разы медленнее симметричного шифрования и высокие вычислительные затраты, при больших ключах. В целом большие размеры ключей и подписей.
4. Безопасность – основана на математически сложных задачах (RSA – факторизация больших чисел, DSA – дискретном логарифме и ECC – эллиптических кривых). Подходит для безопасного обмена ключей. Однако уязвимы к квантовым атакам, не устойчивы к алгоритму Шора⁵.
5. Распределение ключей – не требует секретного канала, поскольку открытый ключ можно передавать по открытым каналам, решая проблему распределения ключей.

Применяется для обмена ключами, в TLS/HTTPS, а также для шифрования небольших сообщений. Пара публичный/приватный упрощают обмен (приватный хранится безопасно, а свободный свободно распространяется). Позволяет реализовывать цифровые подписи и аутентификацию.

Гибридное шифрование — компромисс между двумя предыдущими подходами. В этом случае сообщение шифруется симметрично, а ключ к

⁵ **Алгоритм Шора** — квантовый алгоритм факторизации (разложения числа на простые множители), позволяющий разложить число M за время $O(\log^3 M)$, используя $O(\log M)$ логических кубитов. Алгоритм разработан Питером Шором в 1994 году. Семь лет спустя, в 2001 году, его работоспособность была продемонстрирована группой специалистов IBM. Число 15 было разложено на множители 3 и 5 при помощи квантового компьютера с 7 кубитами.

нему – асимметрично. Получателю нужно сначала расшифровать симметричный ключ, а потом с его помощью – само сообщение.



Рисунок 9. Гибридное шифрование.

3. Реализация хэш-функций

Хэш-функция – это алгоритм или детерминированная функция, которая необратимо преобразует входные данные произвольной длины в фиксированную длину символов (хэш-код/хэш-сумма/дайджест). Такое преобразованные нельзя расшифровать. Хэш-код – результат хэширования тех или иных данных.

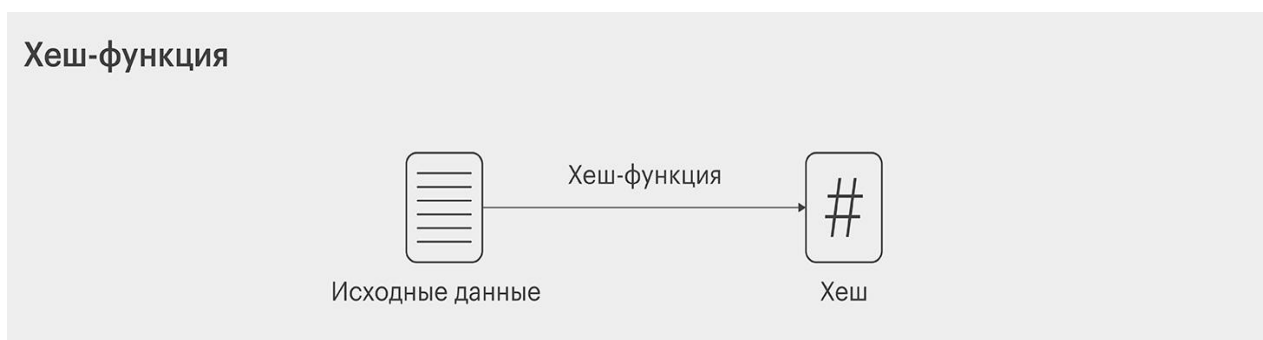


Рисунок 10. Хэширование.

Требования к криптографической хэш-функции:

1. Детерминированность – одинаковый вход и всегда одинаковый выход.
2. Быстрота вычисления – легко вычислить хэш для любого входа.
3. Односторонность и необратимость (pre-image resistance) – невозможность восстановить исходные данные по сгенерированному хэшу.
4. Устойчивость к коллизиям (collision resistance) – невозможно найти два разных входа с одинаковым хэшем.
5. Малое изменение входа (Avalanche effect) – малое изменение приводит к радикальному изменению хэша.
6. Хэш – всегда фиксированного размера вне зависимости от длины.

Примеры хэш-функций: SHA-256, SHA-384, SHA-512 (NIST), SHA-3 (Кессак), ГОСТ Р 34.11-2012. Устаревшие MD-5 и SHA-1 и ниже, не рекомендуются из-за найденных коллизий.

Основные характеристики:

1. Используют серию раундовых преобразований.
2. Основывается на следующих операциях: побитовые сдвиги, перестановки, нелинейные функции, сложение по модулю 2.
3. Имеет требования к устойчивости к коллизиям, односторонность (невозможность восстановить исходное сообщение).

Применение: цифровые подписи, целостность файлов, хранение паролей (с солью), SSL/TLS, блокчейн⁶.

4. Реализация MAC

Под MAC можно подразумевать разные вещи, которые так или иначе связаны с информационной безопасностью.

⁶ **Блокчейн** (англ. *blockchain*, изначально *block chain* — цепь из блоков) — выстроенная по определённым правилам непрерывная последовательная цепочка блоков (связный список), содержащих какую-либо информацию. Связь между блоками обеспечивается не только нумерацией, но и тем, что каждый блок содержит свою собственную хеш-сумму и хеш-сумму предыдущего блока.

Мандатное управление доступом (англ. Mandatory access control, MAC – принудительный контроль доступа) – это разграничение доступа субъектов к объектам, основанное на назначении *метки конфиденциальности* для информации, содержащейся в объектах, и выдаче официальных разрешений (допуска) субъектам на обращение к информации такого уровня конфиденциальности. Это способ, сочетающий защиту и ограничение прав, применяемый по отношению к компьютерным процессам, данным и системным устройствам и предназначенный для предотвращения их нежелательного использования.

Согласно требованиям ФСТЭК, мандатное управление доступом или «метки доступа» являются ключевым отличием систем защиты государственной тайны РФ старших классов 1В и 1Б от младших классов защитных систем на классическом разделении прав по матрице доступа.

Основные принципы MAC

- Объекты (файлы, процессы, устройства, сокет) помечаются метками безопасности (пример: «Секретно», «Конфиденциально»).
- Субъекты (пользователи, процессы) получают допуски разрешающие доступ к определённым уровням конфиденциальности.
- Доступ разрешён, только если допуск субъекта доминирует над меткой объекта (в рамках политики).
- Политика принудительна: пользователь не может изменить метки или обойти правила, даже имея права владельца.

Невозможно эффективно реализовать MAC с нуля в обычной Windows или стандартной Linux без специальных расширений. Необходимо использовать специальные расширения (SELinux, AppArmor, TrustedBSD)

Реализация архитектуры

1. Определяется политика безопасности, её уровни и категории и какие субъекты (роли/пользователи) и какие допуски они имеют.

2. Разметка объектов (labeling). Все файлы, процессы, устройства, сокетты имеют метки безопасности. Метки хранятся в расширенных атрибутах файловой системы.

3. Назначение допусков субъектам. При входе пользователя или запуске процесса ОС проверяет его допуск (сертификат, токен, профиль). Процесс наследует допуск пользователя.

4. Реализация ядра проверки доступа (reference monitor). Встроенный в ядро компонент, который перехватывает все попытки доступа. Сравнивает метку объекта и допуск субъекта по правилам выбранной модели. Разрешает или блокирует операции.

5. Аудит и мониторинг. Все попытки нарушения политики логируются.

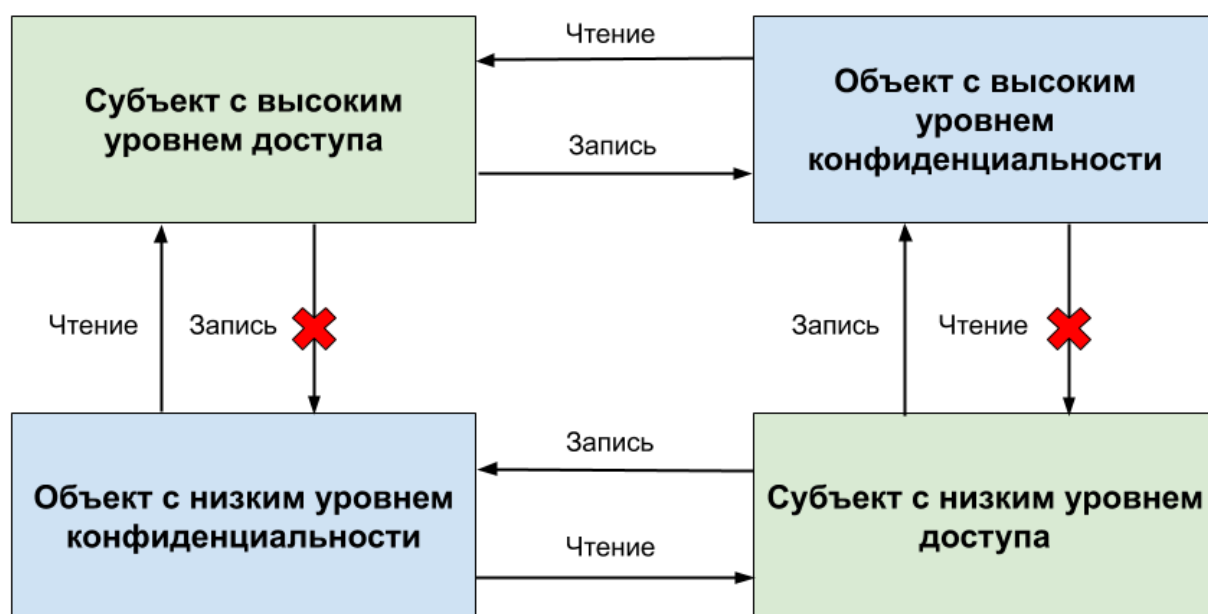


Рисунок 11. Схема работы.

Модель MAC по своей сути является «электронной» реализацией бумажного «секретного» документооборота. Это не программная функция в приложении, а системный уровень безопасности и архитектура встроенная в ОС. Использовать только на выделенных системах с контролируемой средой. Не совместимо с произвольным ПО (всегда тестировать каждое приложение). Лучше комбинировать с шифрованием, аудитом.

Имитовставка (англ. Message Authentication Code MAC – код аутентификации послания) – это криптографический код аутентичности сообщения, который защищает сообщение от подделки и обеспечивает целостность, а вычисляется на основе сообщения и секретного ключа. Средство обеспечения имитозащиты⁷ в протоколах аутентификации сообщений с доверяющими друг другу участниками — специальный набор символов, который добавляется к сообщению и предназначен для обеспечения его целостности и аутентичности источника данных. Для проверки целостности (но не аутентичности) сообщения на отправляющей стороне к сообщению добавляется значение *хеш-функции* от этого сообщения, на приёмной стороне также вырабатывается хэш от полученного сообщения. Выработанный на приёмной стороне и полученный хэш сравниваются. В случае равенства считается, что полученное сообщение дошло без изменений.

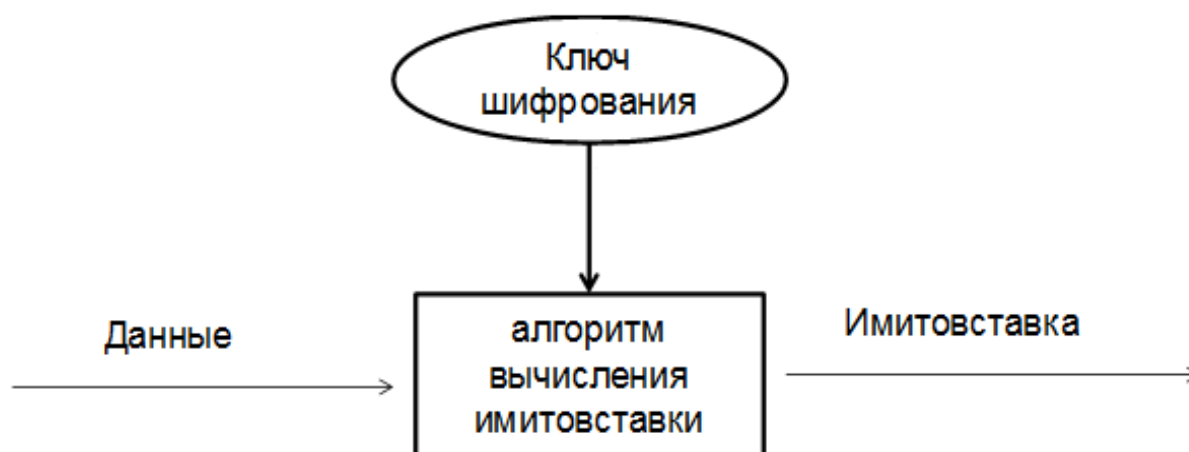


Рисунок 12. Имитовставка или MAC.

Основные принципы и требования имитовставки:

- Только обладатель ключа может сгенерировать/проверить MAC, либо секретный ключ должен быть известен только доверенным сторонам.

Используется общий секретный ключ!

⁷ **Имитозащита** — защита системы шифровальной связи или другой криптосистемы от навязывания ложных данных. Защита данных от внесения в них несанкционированных изменений, другими словами, защита целостности сообщения.

- При знании многих пар (сообщение, MAC), невозможно подделать MAC без знания ключа.

- Должен быть устойчив к атакам и коллизиям.
- MAC должен быть детерминированным (одно и то же сообщение + ключ = один и тот же тег.).

Для защиты от фальсификации (имитации) сообщения применяется имитовставка, выработанная с использованием секретного элемента (ключа), известного только отправителю и получателю.

Реализация MAC

1. Выбираем алгоритм блочного шифра AES-CMAC (для аппаратного AES, нет хэша) или хэша HMAC-SHA256.
2. Сгенерируйте случайный секретный ключ.
3. Вычисляем MAC от сообщения с этим ключом.
4. Передаём сообщение + MAC.
5. Получатель проверяет MAC, берёт тот же ключ, который использует постоянное время.
6. Храним ключ в безопасности (например, в защищённом хранилище, HSM, TPM).

Типы MAC: HMAC – построен на хэш-функции (HMAC-SHA256). Использует ключ и двойное хэширование. CMAC – построен на основе блочного шифра. GMAC – основан на режиме GCM.

Применение: Используется в SSH, TLS, IPsec, банковских протоколах и быстрее цифровой подписи, аутентификация API-запросов, безопасные протоколы обмена.

MAC-адрес (от англ. Media Access Control — контроль доступа к среде, Hardware Address, также физический адрес) — уникальный идентификатор, присваиваемый каждой единице сетевого оборудования или некоторым их интерфейсам в компьютерных сетях Ethernet. Присваивается производителем

или настраивается программно. Он используется на канальном уровне модели OSI для передачи данных.

При проектировании стандарта Ethernet было предусмотрено, что каждая сетевая карта должна иметь уникальный шестибайтный номер (MAC-адрес), «прошитый» в ней при изготовлении.

С помощью него можно ограничивать доступ.

- Фильтрация MAC-адресов (подключение только устройствам с разрешённым MAC).
- Слежение за тем, какие устройства появляются в сети, по их MAC-адресам.
- Привязка аккаунта по MAC.
- MAC-спуфинг – любой пользователь может изменить MAC-адрес своего интерфейса и выдать себя за другое устройство.

Сам по себе MAC-адрес не является механизмом защиты, как, например, шифрование или имитовставка (HMAC), но может использоваться в целях ИБ как часть систем аутентификации, фильтрации и контроля доступа.

5. Реализация ЭЦП

Электронная цифровая подпись (ЭЦП), цифровая подпись (ЦП) – это набор знаков и паролей, позволяющий подтвердить подлинность электронного документа (реальный человек или аккаунт в криптовалютной системе). Подпись связывается как с автором, так и с самим документом криптографическими методами и не может быть подделана путем обычного копирования.

Подпись генерируется с помощью закрытого ключа и проверяется с помощью открытого.

Электронная подпись – это дополнение к пересылаемому документу, созданное криптографическими методами. Она позволяет подтвердить

авторство сообщения и проверить данные на целостность: не вносились ли в них изменения после того, как поставили подпись. Реализуется ЭЦП через асимметричное шифрование и асимметричную криптографию.

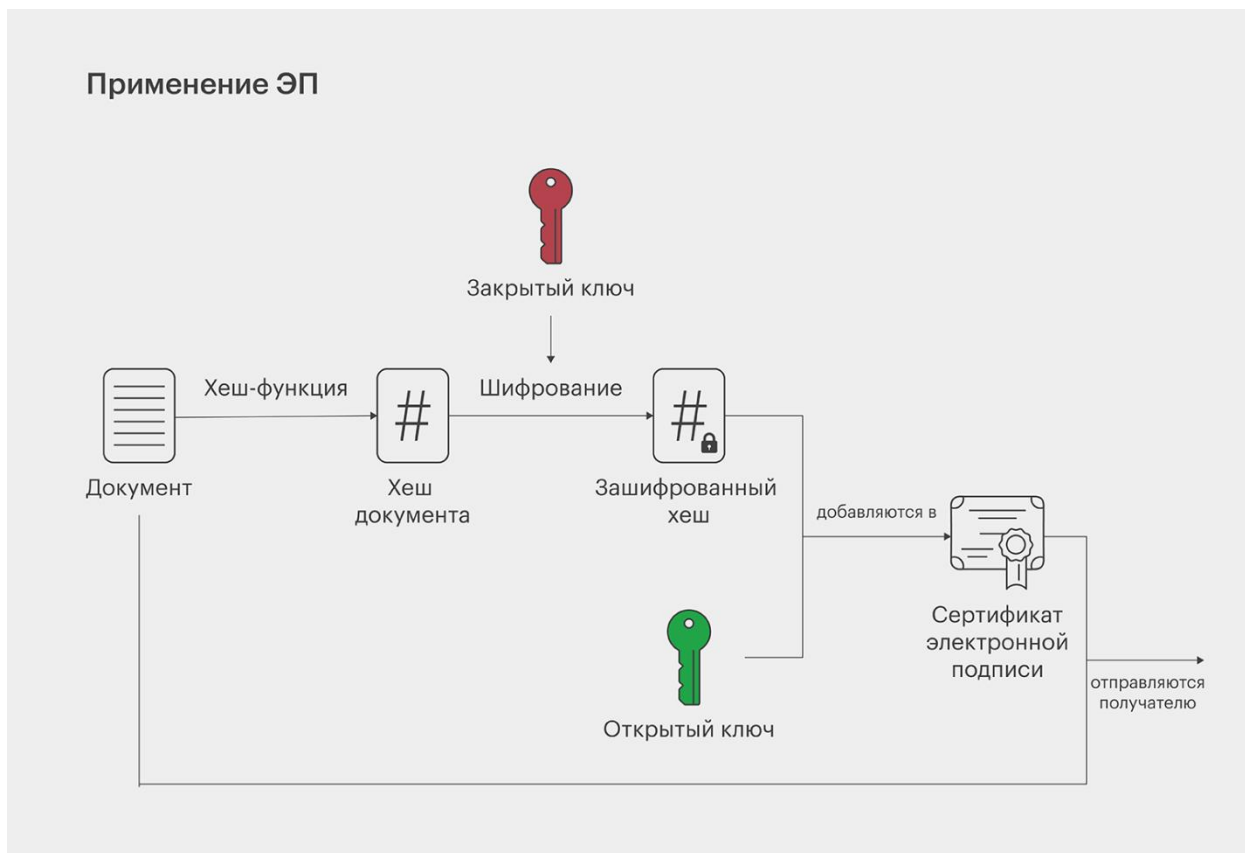


Рисунок 13. Реализация электронно-цифровой подписи.

Свойства: подтверждение авторства, гарантирование целостности, обеспечение невозможности отказа от авторства.

Работа и реализация электронной подписи:

1. Генерируются ключи и отправитель вычисляет хэш сообщения.
2. Шифрует хэш своим закрытым ключом = цифровая подпись
3. Получатель расшифровывает подпись открытым ключом отправителя и сравнивает/проверяет хэш.

Основные требования: Подпись уникальна для каждого сообщения и ключа. Подпись не может быть перенесена на другое сообщение. Подпись может быть проверена третьей стороной. Невозможно подделать подпись без закрытого ключа.

Применение: электронные документы, заявки, банковские транзакции, цифровые сертификаты (X.509), государственные системы (ЭП, Госуслуги, ГИС).

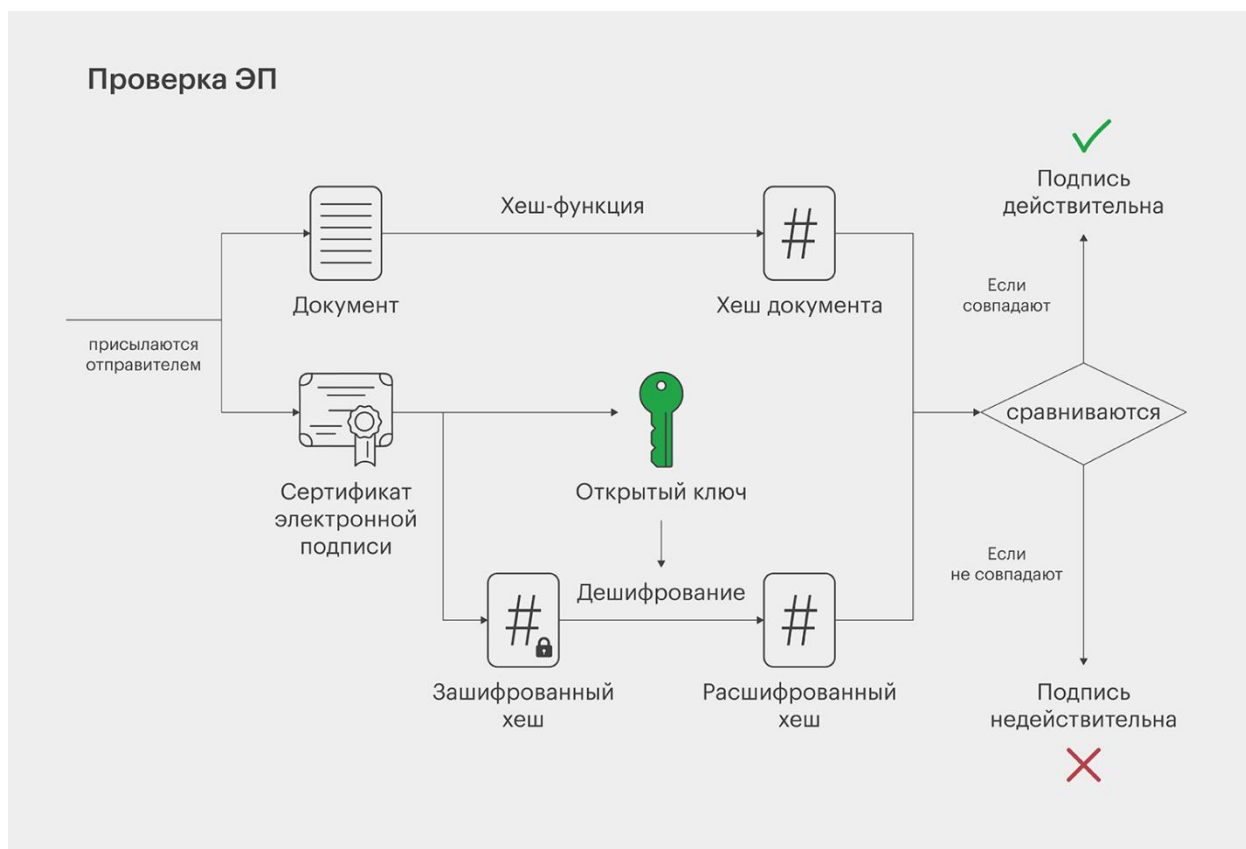


Рисунок 14. Проверка электронно-цифровой подписи.

Если после применения ЭП документ был хоть немного изменён, то при проверке хеши не совпадут. Если же они одинаковые, можно быть уверенным: данные добрались до получателя в целости и сохранности. Человек, который поставил свою ЭП, потом не сможет отрицать это. Дело в том, что доступ к закрытому ключу есть только у него, а значит, никто другой подписать документ не мог.

6. Понятие слоя, раунда, S-box

Слой (Layer) – в криптографических алгоритмах (блочных шифрах) это совокупность или набор однотипных операций (S-слой, линейный слой), выполняемых параллельно на всех битах, данных за один шаг. Блок шифры состоят из последовательности слоёв (слой подстановки, перемешивания, перестановки).

Цель слоёв: обеспечить диффузию и запутанность (по Шеннону).

Примеры слоёв: параллельная замена битов, перестановка битов, добавление ключа, линейное преобразование.

Раунд (Round) – это одна итерация шифрующего цикла (включает S-слой, линейный слой, добавление ключа) или один из последовательных шагов обработки данных в алгоритме шифрования.

S-box (Substitution box – блок замены) – нелинейное отображение (нелинейная таблица) замены, которая преобразует входные биты в выходные биты. S-box обеспечивает криптостойкость и создаёт путаницу (confusion). Обычно таблица замены 4x4 или 8x8.

Цель: обеспечить нелинейность, что важно для устойчивости к линейному и дифференциальному криптоанализу, а также S-box должен быть биекцией (взаимно однозначным).

7. Как в ГОСТ Р 32.12-2018 Магма (ГОСТ 28147) осуществляется преобразование с помощью S-boxes. Что подаётся на вход и получается на выходе.

ГОСТ Р 34.12-2018 – это российский стандарт, шифрования данных. Он определяет блочный шифр Магма, работающий с блоками по 64 бита и ключом 256 бит. Преобразование с использованием S-boxes является ключевым нелинейным этапом, обеспечивающим криптографическую стойкость алгоритма.

Структура шифра:

Блок в 64 бита, разбивается на два 32-битных полублока. Количество раундов – 32. Ключ в 256 бит разбивается на 8 подключей по 32 бита и они циклически используются в раунде.

На вход этого преобразования подаётся 32-битное значение, полученное в результате побитового XOR текущего раунда и 32-битного раундового ключа. Они складываются, затем значение разбивается на 8 последовательных 4-битных фрагментов, каждый из которых обрабатывается

отдельным S-box. Каждый фрагмент проходит через свой S-box. Всего в алгоритме используется 8 фиксированных таблиц замены, каждая из которых представляет собой нелинейную функцию, отображающую 4-битный вход в 4-битный выход. Таблицы утверждены конкретным набором S-boxes.

После обработки всех восьми фрагментов их выходные значения объединяются в единое 32-битное слово. Затем это слово подвергается циклическому сдвигу влево на 11 бит, что обеспечивает дополнительное перемешивание битов между раундами. Результирующее 32-битное значение является выходом функции F и в дальнейшем участвует в формировании нового левого полублока следующего раунда через операцию XOR с исходным левым полублоком.

Практическая часть выполнения

Переходим к выполнению практической части лабораторной работы с разбором работы алгоритма, особенностями, схемами отображения и реализацией алгоритма шифрования в соответствии с вариантом задания. По техническому заданию необходимо реализовать алгоритм шифрования Алгоритм ГОСТ Р 34.12-2018 Магма (ГОСТ 28147). Режим гаммирования с ОС.

ГОСТ 34.12 – это стандарт криптографической защиты информации. Общепринятый стандарт, принятый в России и некоторых других странах СНГ и устанавливающий требования к методам и алгоритмам шифрования информации.

ГОСТ 28147-89 «Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования» — это устаревший государственный стандарт СССР (а позже межгосударственный стандарт СНГ), описывающий алгоритм симметричного блочного шифрования и режимы его работы. Является примером DES-подобных криптосистем, созданных по классической итерационной схеме Фейстеля.

Магма (ГОСТ 28147-89) — это блочный шифр с длиной ключа 256 бит и 32 циклами (называемыми раундами) преобразования, оперирующий 64-битными блоками, вариант классического криптографического алгоритма включённого в современные стандарты (ГОСТ Р 34.12-2015/2018). Основная структура алгоритма шифра — это 32-раундная сеть Фейстеля с 32-битными половинами и применением 8-ми 4×4 S-блоков. (RFC8891 / RFC5830 и пояснения в RFC7836).

Выделяют четыре режима работы ГОСТ 28147-89:

- Простой замены.
- Гаммирование.
- Гаммирование с обратной связью.
- Режим выработки имитовставки.

История создания шифра и критерии разработчиков были впервые публично представлены в 2014 году руководителем группы разработчиков алгоритма Заботиным Иваном Александровичем на лекции, посвященной 25-летию принятия российского стандарта симметричного шифрования.

Работы над алгоритмом, положенным впоследствии в основу стандарта, начались в рамках темы «Магма» (защита информации криптографическими методами в ЭВМ ряда Единой системы) по поручению Научно-технического совета Восьмого главного управления КГБ СССР (ныне в структуре ФСБ). В марте 1978 года после длительного предварительного изучения опубликованного в 1976 году стандарта DES. В действительности работы по созданию алгоритма (или группы алгоритмов), схожего с алгоритмом DES, начались уже в 1976 году.

Изначально работы имели гриф «Совершенно секретно»⁸. Затем были понижены до грифа «Секретно». В 1983 году гриф алгоритма был понижен

⁸ «*Совершенно секретно*» — одна из трёх степеней секретности сведений или гриф секретности. Всего выделяют 3 грифа секретности: «особой важности», «совершенно секретно» и «секретно». Использование перечисленных грифов секретности для засекречивания сведений, не отнесенных к государственной тайне, не допускается.

до пометки «Для служебного пользования»⁹. Именно с последней пометкой алгоритм был подготовлен для публикации в 1989 году. 9 марта 1987 года группа разработчиков-криптографов получила авторское свидетельство с приоритетом на изобретение на устройство шифрования по алгоритму «Магма-2».

Согласно извещению ФСБ о порядке использования алгоритма блочного шифрования ГОСТ 28147-89, средства криптографической защиты информации, предназначенные для защиты информации, не содержащей сведений, составляющих государственную тайну, реализующие в том числе алгоритм ГОСТ 28147-89, не должны разрабатываться после 1 июня 2019 года, за исключением случаев, когда алгоритм ГОСТ 28147-89 в таких средствах предназначен для обеспечения совместимости с действующими средствами, реализующими этот алгоритм.

Режимы алгоритма

Режим простой замены – это режим работы блочного шифра, в котором *каждый блок открытого текста шифруется независимо от других*, используя прямое применение блочного алгоритма.

Простейший режим и аналог ECB в зарубежных стандартах, не скрывает одинаковые блоки. Не годится для больших данных, можно шифровать и расшифровывать блоки произвольно и параллельно.

$$C_i = E_K(P_i)$$

Для шифрования в этом режиме 64-битный блок открытого текста разбивается на 2 половины. Для генерации подключей исходный 256-битный ключ разбивается на восемь 32-битных чисел. Результатом выполнения всех 32 раундов алгоритма является 64-битный блок шифртекста. Расшифрование

⁹ «Для служебного пользования» (ДСП) — это пометка на документе, указывающая на ограничение доступа к информации. Такие документы содержат сведения ограниченного распространения, не подпадающие под государственную тайну, но доступ, к которым ограничен по служебной необходимости или в соответствии с законами.

осуществляется по тому же алгоритму, что и зашифрование, с тем изменением, что инвертируется порядок подключей.

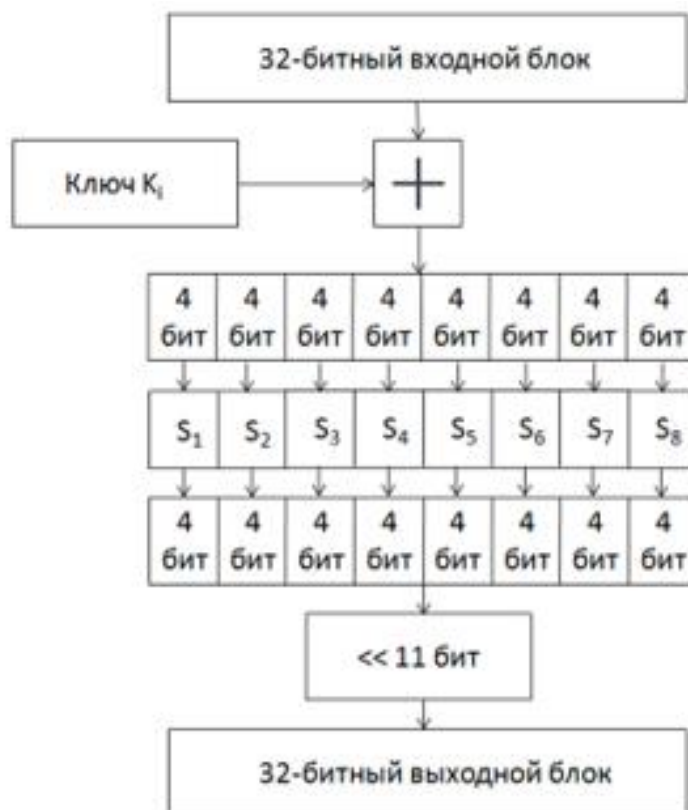


Рисунок 15. Функция, используемая в сети Фейстеля.

Режим простой замены имеет недостатки:

- Может применяться только для шифрования открытых текстов с длиной кратной 64 бит.
- При шифровании одинаковых блоков открытого текста получаются одинаковые блоки шифротекста, что может дать определённую информацию криптоаналитику.

Таким образом, применение данного режима желательно лишь для шифрования ключевых данных.

Режим гаммирования – это режим потокового шифрования, в котором блочный шифр используется только для генерации криптографической гаммы, а шифрование выполняется с помощью XOR.

При работе ГОСТ 28147-89 в режиме гаммирования описанным выше образом формируется криптографическая гамма, которая затем побитно складывается по модулю 2 с исходным открытым текстом для получения шифротекста. Шифрование в режиме гаммирования лишено недостатков, присущих режиму простой замены. Идентичные блоки исходного текста дают разный шифротекст, а для текстов с длиной, не кратной 64 бит, «лишние» биты гаммы отбрасываются. Кроме того, гамма может быть выработана заранее, что соответствует работе шифра в поточном режиме.

Гамма формируется так:

1. Гамма-регістром является переменная S (64 бита), начально равная синхрпосылке¹⁰(IV)
2. На каждом шаге выполнится блочное шифрование Магма. $S = f_K(S)$.
3. Полученная величина используется как гамма: $\Gamma_i = S$

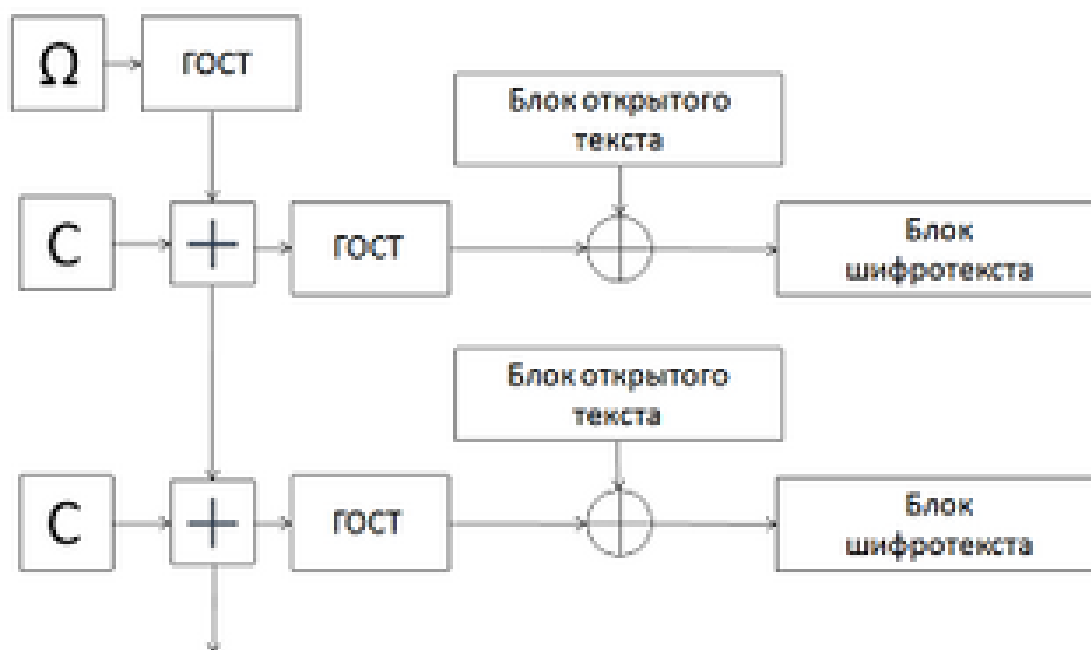


Рисунок 16. Схема работы в режиме гаммирования.

Выработка гаммы происходит на основе ключа и так называемой *синхрпосылки*, которая задает начальное состояние генератора.

¹⁰ *Синхрпосылка* (инициализационный вектор, IV) — элемент данных в криптографических алгоритмах, который инициализирует алгоритм шифрования.

Синхропосылка шифруется с использованием описанного алгоритма простой замены, полученные значения записываются во вспомогательные 32-разрядные регистры.

Блок шифра используется, как *детерминированный генератор псевдослучайной* последовательности длины 64 бит. Открытый текст шифруется/расшифровывается **XOR** с гаммой.

Для расшифровывания необходимо выработать такую же гамму, после чего побитно сложить её по модулю 2 с зашифрованным текстом. Очевидно, нужно использовать ту же синхропосылку, что и при шифровании. При этом, исходя из требований уникальности гаммы, нельзя использовать одну синхропосылку для шифрования нескольких массивов данных. Как правило, синхропосылка тем или иным образом передается вместе с шифротекстом.

Особенности:

- Аналог режима – OFB.
- Ошибка в передаче шифртекста влияет только на один блок.
- Данным режимом можно обрабатывать фрагменты любого размера, не требуется паддинг.
- Однако параллельность невозможна – гамма зависит от предыдущей.

Особенность работы ГОСТ 28147-89 в режиме гаммирования заключается в том, что при изменении одного бита шифротекста изменяется только один бит расшифрованного текста. С одной стороны, это может оказывать положительное влияние на помехозащищённость; с другой — злоумышленник может внести некоторые изменения в текст, даже не расшифровывая его.

Режим выработки имитовставки – это режим обработки данных блочным шифром, при котором формируется имитовставка (MAC) – короткое значение однозначно связанное с сообщением и ключом. MAC по ГОСТ 28147-89 используется для контроля целостности и аутентичности.

Этот режим не является в общепринятом смысле режимом шифрования. При работе в режиме выработки имитовставки создаётся некоторый дополнительный блок, зависящий от всего текста и ключевых данных. Данный блок используется для проверки того, что в шифротекст случайно или преднамеренно не были внесены искажения. Это особенно важно для шифрования в режиме гаммирования, где злоумышленник может изменить конкретные биты, даже не зная ключа; однако и при работе в других режимах вероятные искажения нельзя обнаружить, если в передаваемых данных нет избыточной информации.



Рисунок 17. Схема выработки имитовставки.

Имитовставка вырабатывается для $M \geq 2$ блоков открытого текста по 64 бит. Блок открытых данных записывается в регистры, после чего подвергается преобразованию, соответствующему первым 16 циклам шифрования в режиме простой замены. К полученному результату побитно по модулю 2 прибавляется следующий блок открытых данных. Последний блок при необходимости дополняется нулями. Сумма также шифруется, а после добавления и шифрования последнего блока из результата выбирается имитовставка длиной L бит.

Для проверки принимающая сторона проводит аналогичную описанной процедуру. В случае несовпадения результата с переданной имитовставкой все соответствующие М блоков считаются ложными.

Выработка имитовставки может проводиться параллельно шифрованию с использованием одного из описанных выше режимов работы.

Особенности:

- Имитовставка = аналог MAC (Message Authentication Code).
- Есть защита от подмены данных.
- Для вычисления требуется обработать данные последовательно (параллельно невозможно).
- Одно изменение бита в любом блоке меняет имитовставку непредсказуемо.

Алгоритм ГОСТ Р 34.12-2018 Магма (ГОСТ 28147). Режим гаммирования с обратной связью (ОС).

Режим гаммирования с обратной связью (ОС) – это потоковый режим применения блочного шифра, в котором на каждом шаге формируется гамма (по сути, псевдослучайный блок шифра обратной связи такой же длины) путём шифрования предыдущего блока шифртекста, а не как в классическом режиме просто счётчика или IV. Затем применяют XOR над гаммой с блоком открытого текста, давая новый блок шифртекста. Полученный шифроблок подаётся на вход для следующей итерации. Шифрование и расшифрование выполняются одной и той же процедурой XOR с гаммой.

Таким образом:

- Первый гамма-блок получается путём шифрования IV.
- Далее гамма зависит от ранее полученного шифртекста обеспечивается «завязка» на предыдущий результат.
- Любой изменённый или пропущенный блок влияет на последующие гаммы и шифртексты.

Алгоритм шифрования похож на режим гаммирования, однако гамма формируется на основе предыдущего блока зашифрованных данных, так что результат шифрования текущего блока зависит также и от предыдущих блоков. По этой причине данный режим работы также называют гаммированием с зацеплением блоков. **Потоковый режим**, который превращает блочный шифр «Магма» в потоковый, благодаря обратной связи.

Алгоритм шифрования следующий:

1. Синхропосылка заносится в регистры.
2. Содержимое регистров шифруется в соответствии с алгоритмом простой замены. Полученный результат является 64-битным блоком гаммы.
3. Блок гаммы побитно складывается по модулю 2 с блоком открытого текста. Полученный шифротекст заносится в регистры.
4. Операции 2-3 выполняются для оставшихся блоков требующего шифрования текста.

При изменении одного бита шифротекста, полученного с использованием алгоритма гаммирования с обратной связью, в соответствующем блоке расшифрованного текста меняется только один бит, так же затрагивается последующий блок открытого текста. При этом все остальные блоки остаются неизменными.

При использовании данного режима следует иметь в виду, что *синхропосылку* нельзя использовать повторно (например, при шифровании логически отдельных блоков информации: сетевых пакетов, секторов жёсткого диска). Это обусловлено тем, что первый блок шифротекста получен всего лишь сложением по модулю два с зашифрованной синхропосылкой.

Таким образом, знание всего лишь 8 первых байт исходного и шифрованного текста позволяют читать первые 8 байт любого другого шифротекста после повторного использования синхропосылки.

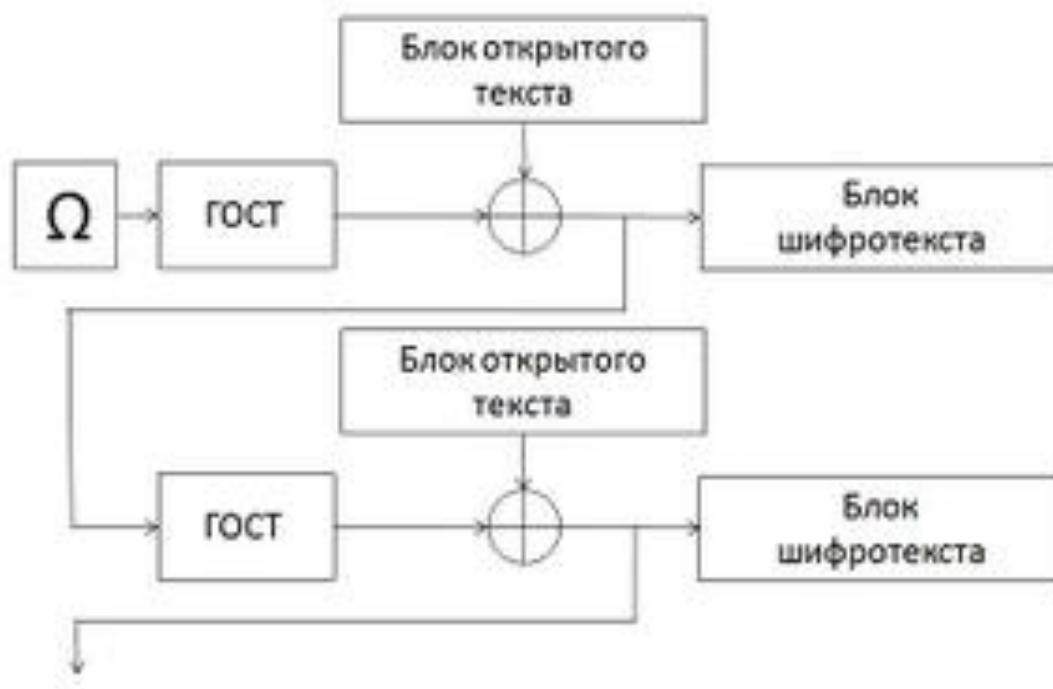


Рисунок 18. Схема работы в режиме гаммирования с обратной связью.

Особенности и требования:

- IV должен быть уникальным для каждого сообщения/сеанса, иначе возможны атаки.
- Нет необходимости в паддинге, если блоки открытого текста разделены по 64-битовой границе (режим потоковый).
- Любая ошибка или изменение блока шифртекста влияет как минимум на следующий гамма-блок и соответствующий открытый текст.
- Параллельное вычисление блоков затруднено блоки гаммы зависят от блока шифртекста.

Блочный шифр Магма, работает с блоками по 64 бита и ключом 256 бит. Преобразование с использованием S-boxes является ключевым нелинейным этапом, обеспечивающим криптографическую стойкость алгоритма.

Структура шифра

1. Берётся начальный вектор (IV), заносится в регистры.
2. Далее пропускается вектор через **шифр Магма**, для получения **гаммы**.

- Блок в 64 бита, разбивается на два 32-битных полублока.
- Количество раундов – 32 (сети Фейстеля).
- Ключ в 256 бит разбивается на 8 подключей по 32 бита и они циклически используются в раунде.

- На вход этого преобразования подаётся 32-битное значение, полученное в результате побитового XOR текущего раунда и 32-битного раундового ключа. Они складываются по модулю 2^{32} .

- Затем значение разбивается на 8 *последовательных 4-битных фрагментов*, каждый из которых обрабатывается отдельным **S-box**.

- Каждый фрагмент проходит через свой S-box. Всего в алгоритме используется 8 фиксированных таблиц замены, каждая из которых представляет собой нелинейную функцию, отображающую 4-битный вход в 4-битный выход. Таблицы утверждены конкретным набором S-boxes.

- Циклический сдвиг на 11 бит.

3. XOR гаммы открытого текста = шифротекст.

4. Обратная связь: следующий вход = предыдущий шифротекст.

5. Повторяем.

Алгоритм шифрования блока всегда один и тот же, режимы работы определяют, как применять шифр к потоку данных и обязательно содержит 8 S-блоков.

Свойства:

- Шифруется не блок открытого текста, а результат предыдущего шага (шифроблока).

- Магма работает только как генератор гаммы, незашифрованный текст не поступает в сам блочный шифр.

- XOR применяется над открытым текстом и гаммой – получаем шифротекст.

- Изменение одного бита шифротекста влияет на один бит расшифрованного текста на следующий блок (локальное влияние).

- Повторное использование IV атака тривиальна.

Использование *S-блоков при гаммировании с обратной связью* нужны, потому что гамма генерируется не сама по себе, а вырабатывается с помощью базового блочного шифра «Магма», режим – это способ применения шифра, а S-блоки – часть самого шифра.

Именно S-boxes обеспечивают нелинейность, без которой шифр сводился бы к системе линейных преобразований и был бы уязвим к простым алгебраическим атакам. Поэтому правильный выбор и фиксация S-boxes критически важны для устойчивости «Магмы» к дифференциальному, линейному и другим видам криптоанализа.

Криптоанализ

Считается, что ГОСТ устойчив к таким широко применяемым методам, как линейный и дифференциальный криптоанализ. Обратный порядок использования ключей в последних восьми раундах обеспечивает защиту от атак скольжения и отражения.

В мае 2011 года известный криптоаналитик Николя Куртуа доказал существование атаки на данный шифр, имеющей сложность в 2^8 (256) раз меньше сложности прямого перебора ключей при условии наличия 2^{64} пар «открытый текст/закрытый» текст. Данная атака не может быть осуществлена на практике ввиду слишком высокой вычислительной сложности. Более того, знание 2^{64} пар «открытый текст/закрытый» текст, очевидно, позволяет читать зашифрованные тексты, даже не вычисляя ключа.

В большинстве других работ также описываются атаки, применимые только при некоторых предположениях, таких как определённый вид ключей или таблиц замен, некоторая модификация исходного алгоритма, или же требующие всё ещё недостижимых объёмов памяти или вычислений. Вопрос о наличии применимых на практике атак без использования слабости отдельных ключей или таблиц замены остается открытым.

Сложность алгоритма (асимптотика)

- Сложность одного раунда Магмы

Сложение составляет $= O(1)$, S-боксы (8 подстановок) $= O(1)$

Сдвиг $= O(1)$ и XOR $= O(1)$.

Итого: 1 раунд $= O(1)$

- Один блок (32 раунда) – $32 \times O(1) = O(1)$
- Режим обратной связи для N блоков

Каждый блок: шифрование предыдущего блока $= O(1)$ и XOR $= O(1)$

Итого: алгоритм $= O(N)$, где N – количество 64-битных блоков.

Алгоритм линейный по количеству блоков.

Схемы шифрования Магма (ГОСТ 28147-89)

Схема структуры раунда выполнения алгоритма

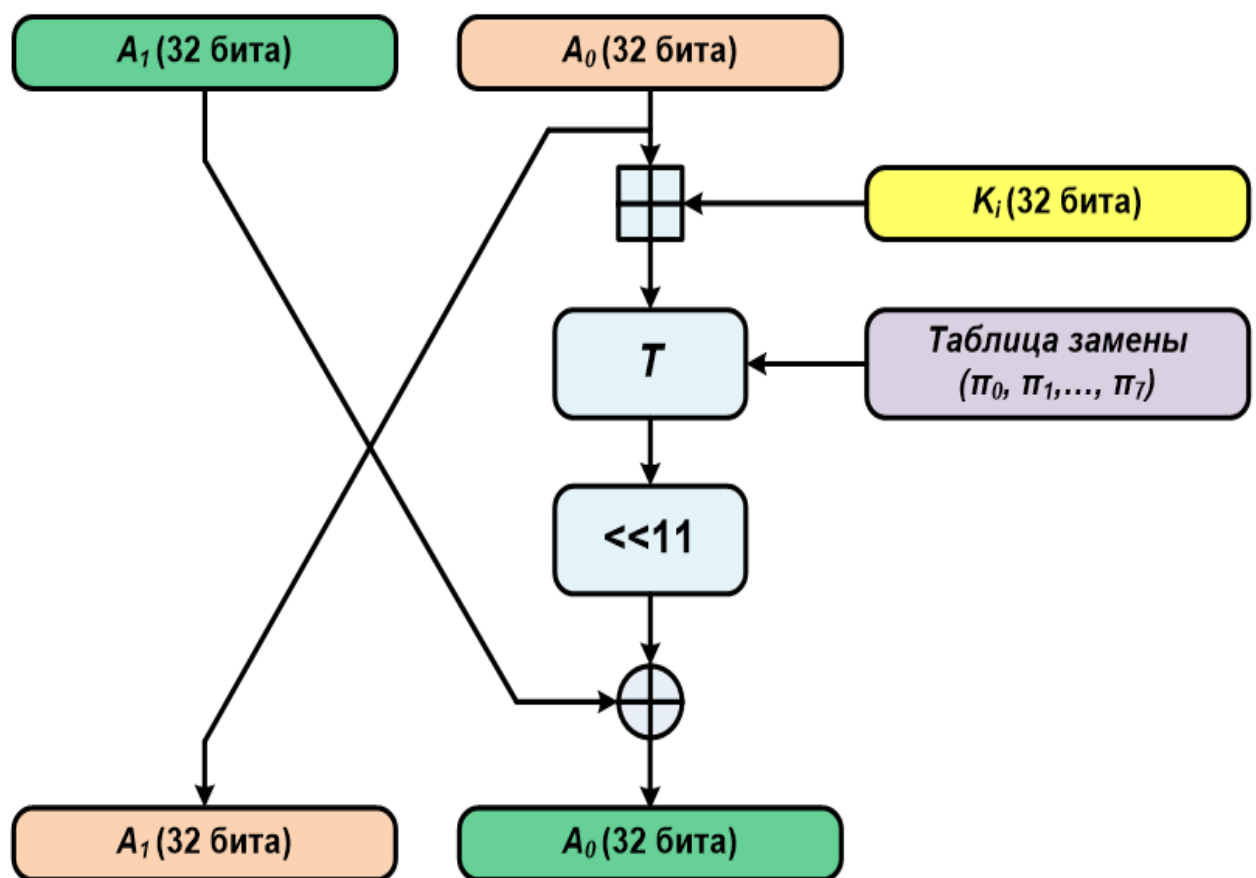
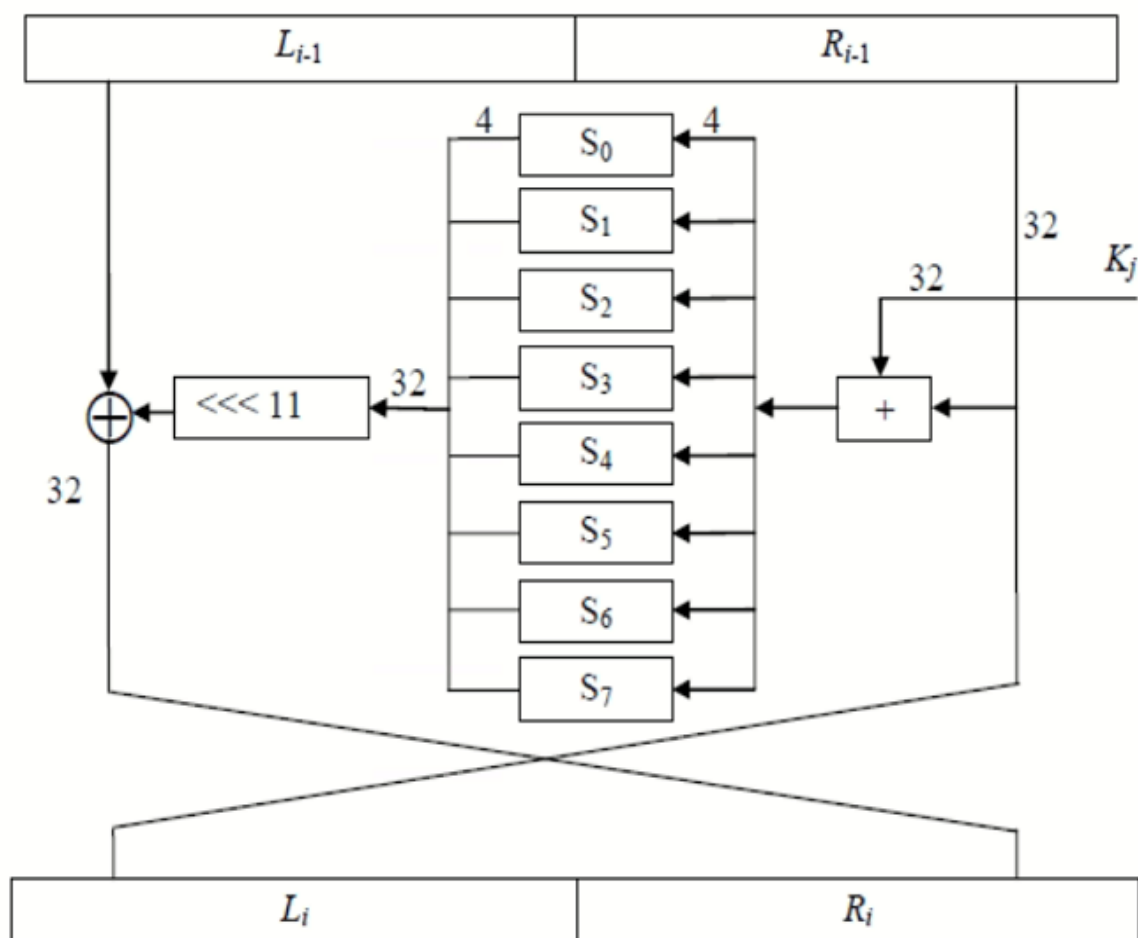


Рисунок 19. Схема структуры раунда алгоритма «Магма»
(Схема одной итерации).

Схема структуры раунда алгоритма с таблицей замен



Структура раунда алгоритма «Магма»

Рисунок 20. Схема структуры раунда алгоритма «Магма»
(Схема одной итерации с подробной таблицей замен).

Блочный шифр – 64 бита. В центре представлены таблицы замены или S-box, которые непосредственно участвуют как компоненты, определяющие нелинейность шифрующего преобразования. Подстановки, которые отображают n -битовый входной блок в выходной длиной m бит. Представлено набором из 8 S-блоков. Каждый из них имеет 4-битный вход и 4-битный выход, что устанавливает сложные нелинейные зависимости между символами открытых текстов, шифртекстов и ключей, реализовать принцип усложнения (confusion). Так же есть битовое сложение, сдвиги и регистры для хранения. Ниже представлена схема раундовых преобразований алгоритма.

Схема получения итерационных ключей.

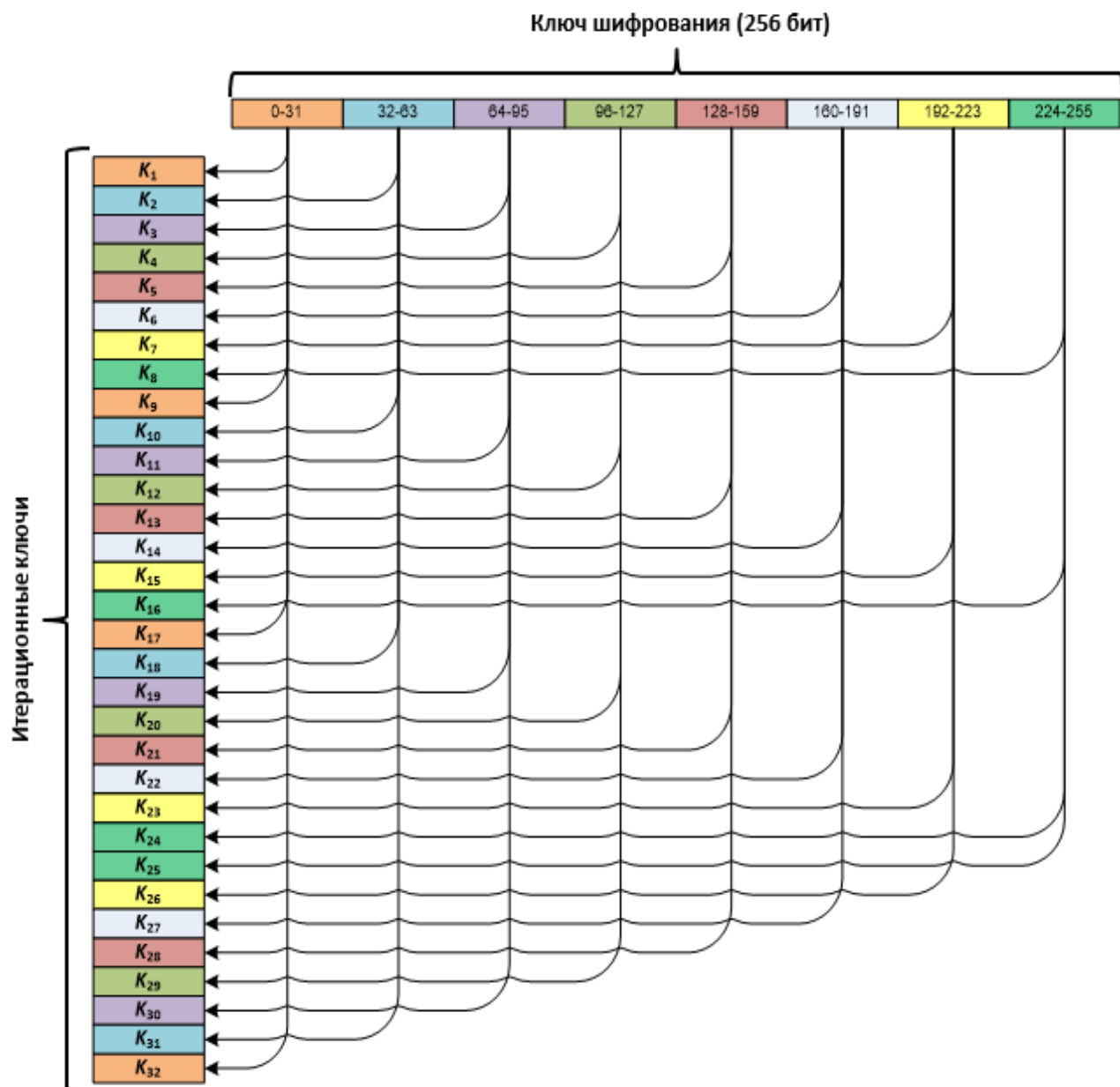


Рисунок 22. Схема получения итерационных ключей.

На представленной схеме показан процесс получения 32 итерационных (раундовых) ключей для алгоритма шифрования "Магма". Этот процесс является частью режима гаммирования, где раундовые ключи используются для формирования гаммы шифра. Входом для схемы является один 256-битный ключ шифрования. Этот 256-битный ключ разбивается на восемь равных частей по 32 бита каждая. Эти части обозначены диапазонами битов: 0-31, 32-63, 64-95, 96-127, 128-159, 160-191, 192-223 и 224-255. Схема показывает, что каждый из 32 раундовых ключей (K_1 - K_{32}) не вычисляется

сложным образом, а просто выбирается одной из этих восьми 32-битных частей исходного ключа. Так, каждая из восьми 32-битных частей исходного ключа используется ровно четыре раза в течение всех 32 раундов шифрования. Этот метод генерации ключей является характерной особенностью алгоритма "Магма" и обеспечивает его криптографическую стойкость при правильном использовании.

Схема алгоритма при шифровании по раундам

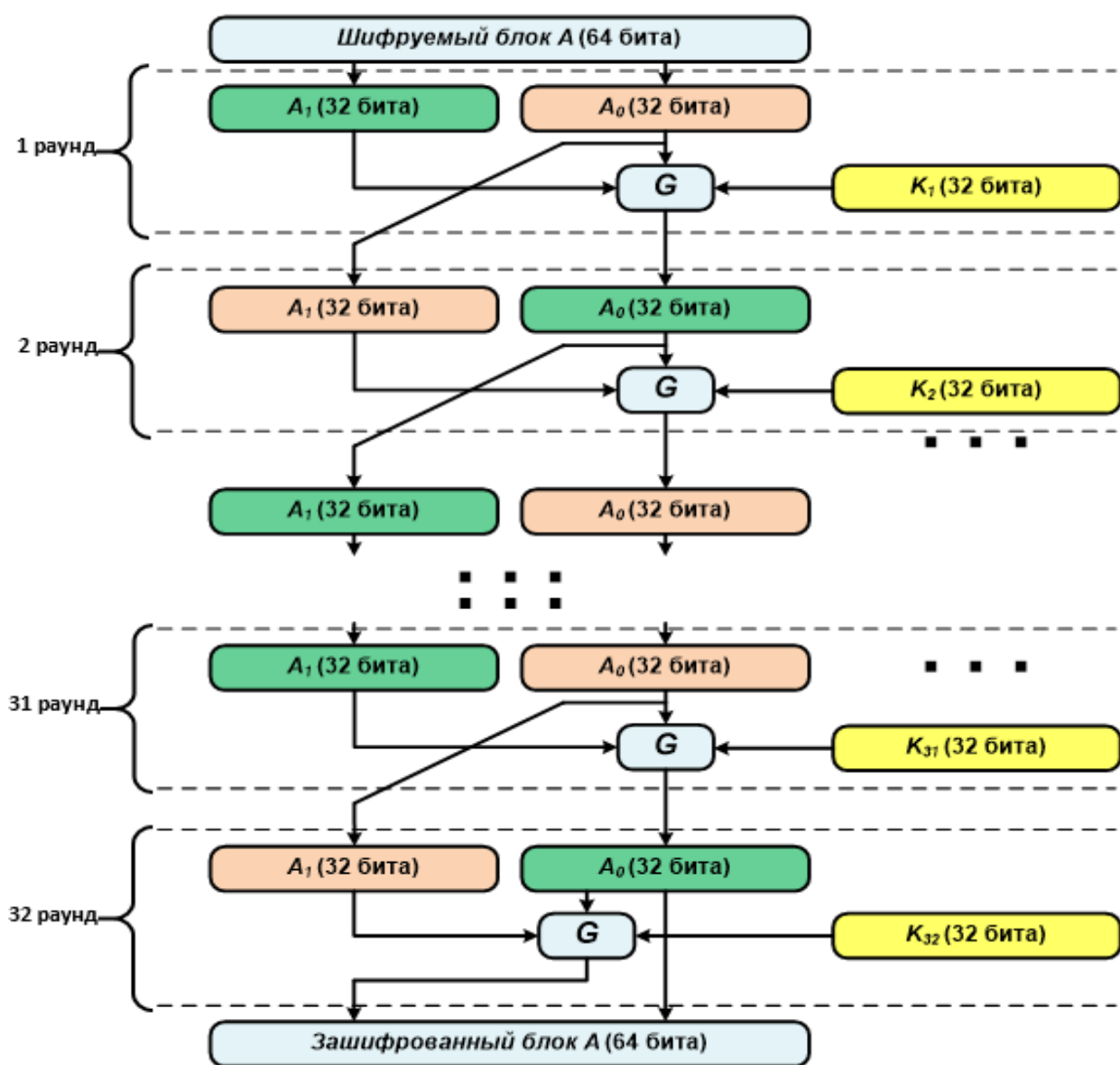


Рисунок 23. Схема алгоритма при шифровании по раундам.

Основной цикл (32 раунда) на каждом раунде i (от 1 до 32) обе половины A_1 и A_0 подаются на вход функции G . Функция G также использует 32-битный раундовый ключ K_i . На выходе функции G

получаются новые значения, которые становятся входом для следующего раунда. В отличие от режима простой замены, здесь обе половины блока изменяются на каждом раунде, а не только одна. После 32 раундов полученные 32-битные части объединяются в один 64-битный блок (зашифрованный блок A). Этот режим является режимом гаммирования, где используется все 32 раунда алгоритма (вместо 16, как в режиме простой замены). Он предназначен для формирования гаммы шифра, которая затем складывается по модулю 2 с открытым текстом. Однако на данной схеме показано непосредственное преобразование блока A в зашифрованный блок, что соответствует процессу генерации одного блока гаммы.

Схема работы алгоритма при расшифровке по раундам

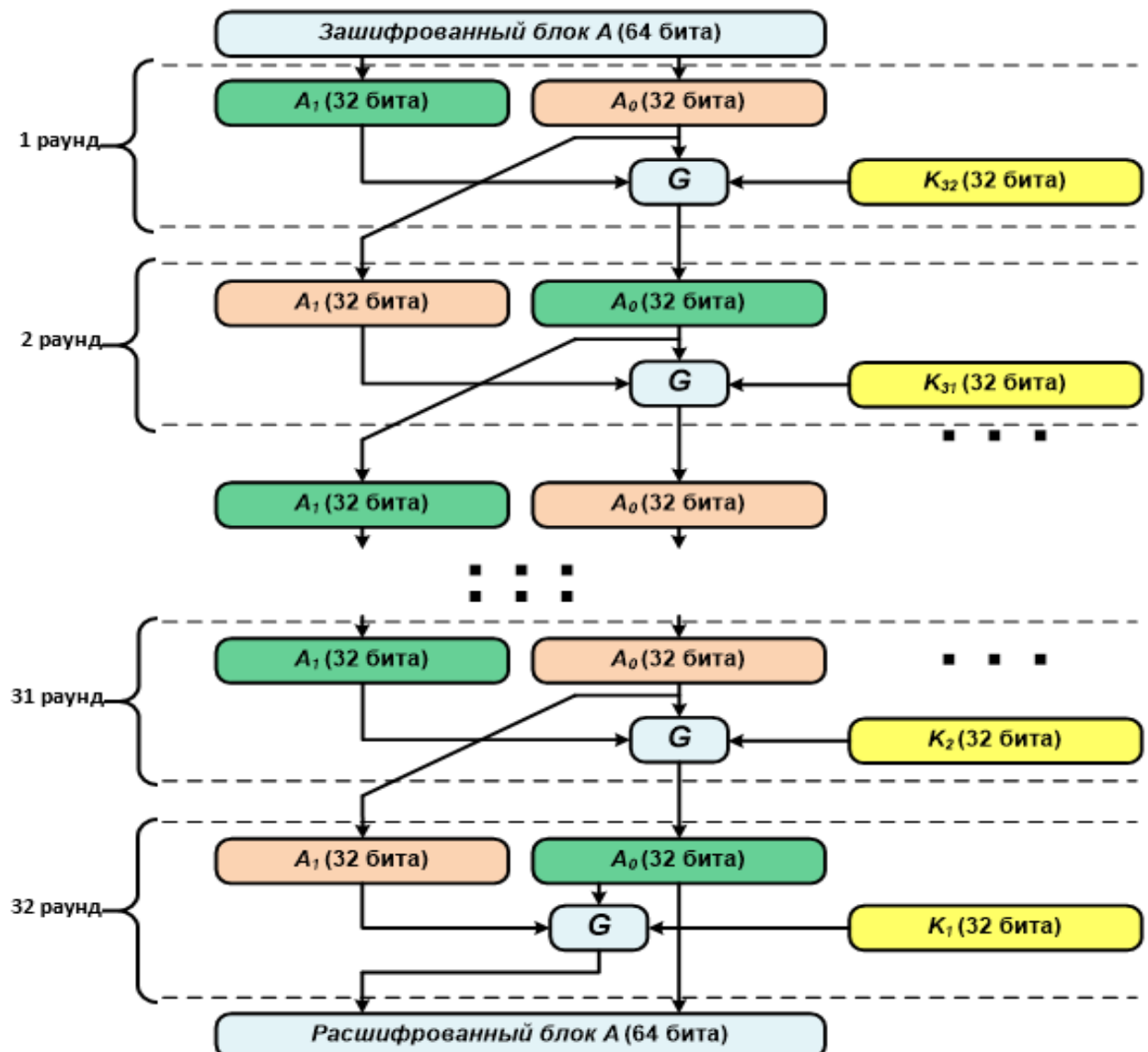


Рисунок 23. Схема работы алгоритма при расшифровке по раундам.

Блок-схема

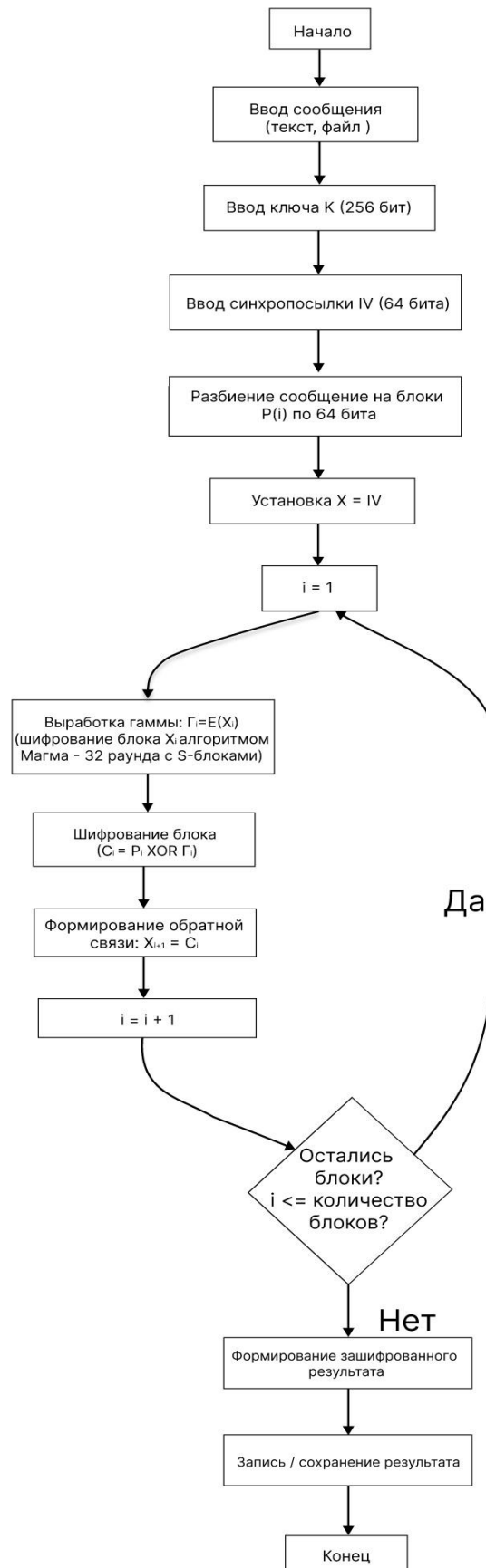


Рисунок 24. Блок-схема алгоритма шифрования Магма (ГОСТ 28147) в режиме гаммирования с обратной связью.

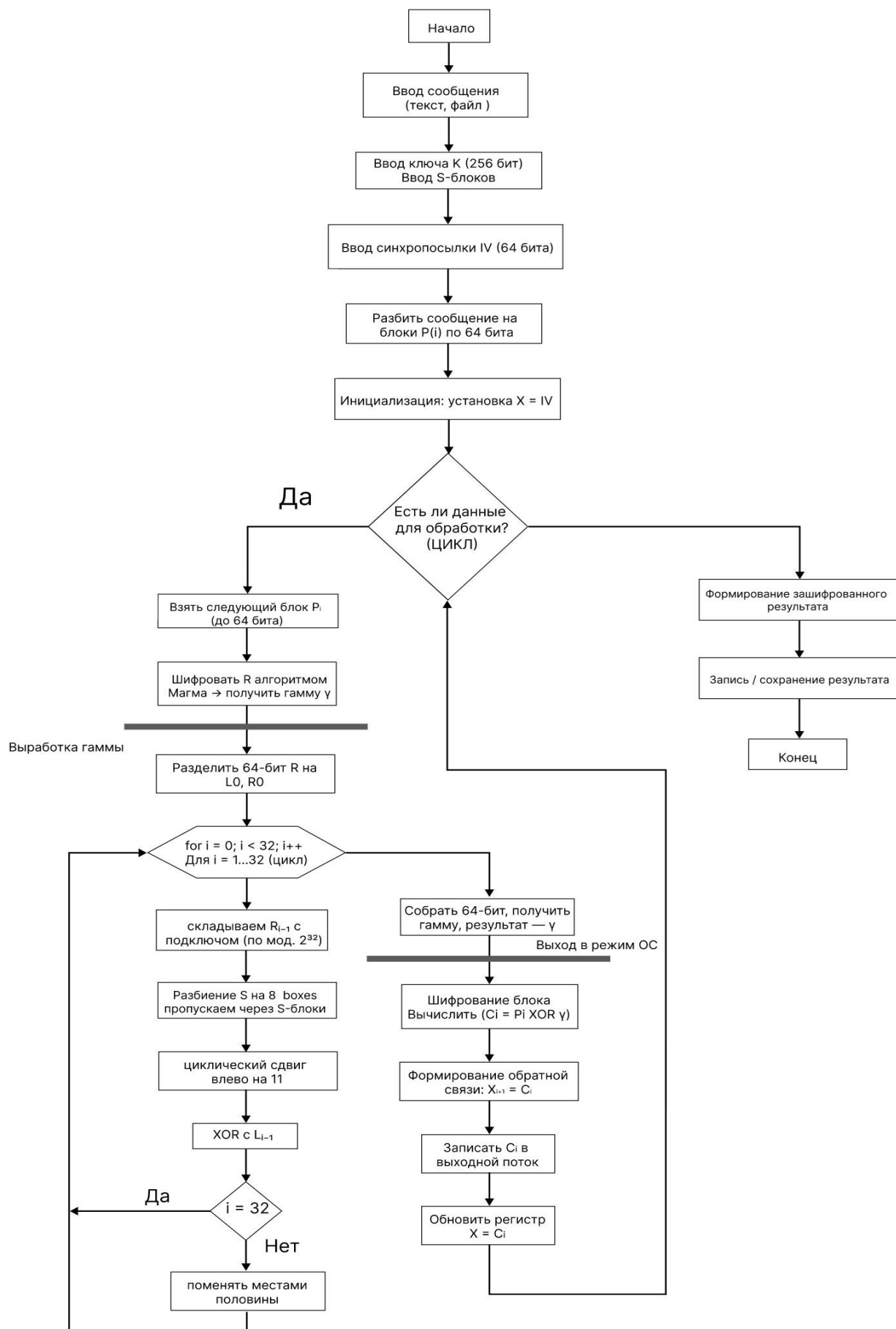


Рисунок 25. Блок-схема алгоритма шифрования Магма (ГОСТ 28147) в режиме гаммирования с обратной связью (Подробная схема).

Архитектура

Для реализации выбрана архитектура двухмодульной системы, состоящая из таких модулей:

- **Логический уровень (Logic Core)**, который содержит реализацию алгоритма MD5, включая все стадии: дополнение сообщения до кратности 512 бит, добавление длины исходного сообщения, инициализация регистров A, B, C, D, выполнение 4 раундов по 16 итераций и формирование финального хэша.

- **Интерфейсный уровень (UI Layer)**, который отвечает за взаимодействие с пользователем, ввод данных студента, выбор входного файла, отображение справочной информации об алгоритме MD5 и вывод вычисленного хэша. Реализуется в десктоп форме.

- **I/O уровень данных (слой работы с файлами)**, который реализует операции чтения и записи файлов, сохранение результатов хэширования и отчётных данных. Данный уровень обеспечивает возможность обработки файлов любого формата. Проверено на word/pdf/jpg форматах.

- **Utils уровень**, который реализует дополнительные функции, которые необходимы для обработки полей, данных, а также для работы с ключом, преобразования до нужной длины, которая установлена алгоритмом.

Стек технологий

Было реализовано программы на следующих языках программирования:

- Чистый C с реализацией классического Windows приложения в качестве GUI. Данная реализация на низкоуровневом языке программирования, позволяет увеличить производительность и поработать с памятью при разработке программы.

- Язык программирования Java использовался для реализации алгоритма шифрования Магма на другом стеке. В качестве UI отображения и интерфейса использовался Swing.
- Среда разработки Visual Studio и IntelliJ IDEA, которые удобны для отладки и визуализации структуры проекта.

Реализация на С

Реализация на С состоит из файлов реализации кода (.c) и заголовков кода (.h). Файл main является точкой запуска программной системы, откуда запускается программа.

main.c

```
#define UNICODE
#define _WIN32_WINNT 0x0600

#include <Windows.h>
#include <commctrl.h>
#include "ui.h"

#pragma comment(lib, "comctl32.lib") // ← линковка библиотеки

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PWSTR pCmdLine, int nCmdShow) {
    (void)hPrevInstance;
    (void)pCmdLine;

    // Инициализация Common Controls (для TaskDialog)
    INITCOMMONCONTROLSEX icc = { sizeof(icc), ICC_STANDARD_CLASSES };
    InitCommonControlsEx(&icc);

    return run_gui(hInstance, nCmdShow);
}
```

cipher.c

Режим гаммирования с обратной связью (CFB). Функции реализуют побайтовое шифрование потока данных, где гамма генерируется путём шифрования регистра (инициализируемого вектором IV), а после каждого байта регистр сдвигается и обновляется – при шифровании, аналогично работа с расшифровыванием только с блоком шифротекста. Это обеспечивает корректную работу в потоковом режиме без необходимости дополнения данных.

```
#include "cipher.h"
#include <string.h>

// === Инициализация === //
void magma_cfb_initialization(magma_cfb_t *ctx, const uint8_t key_raw[32], const uint8_t iv[8]) {
    MagmaSetKey(&ctx->key, key_raw);
    memcpy(ctx->reg, iv, MAGMA_BLOCK_SIZE);
}
```

```

}

/* Шифрование Gamma = E(R) обрабатываем по-байтово,
   регистр перемещается на 1 байт и добавляется новый шифротекст. */
void magma_cfb_encrypt_stream(magma_cfb_t* ctx, const uint8_t* in, uint8_t* out, size_t len) {
    uint8_t gamma[MAGMA_BLOCK_SIZE];

    for (size_t i = 0; i < len; ++i) {
        MagmaEncryptBlock(&ctx->key, ctx->reg, gamma);
        out[i] = in[i] ^ gamma[0];

        // сдвиг влево на один байт
        memmove(ctx->reg, ctx->reg + 1, MAGMA_BLOCK_SIZE - 1);
        ctx->reg[MAGMA_BLOCK_SIZE - 1] = out[i];
    }
}

/* Расшифрование Gamma = E(R) отличие в том, что при обновлении регистра добавляем входной C(i)*/
void magma_cfb_decrypt_stream(magma_cfb_t* ctx, const uint8_t* in, uint8_t* out, size_t len) {
    uint8_t gamma[MAGMA_BLOCK_SIZE];

    for (size_t i = 0; i < len; ++i) {
        MagmaEncryptBlock(&ctx->key, ctx->reg, gamma);
        out[i] = in[i] ^ gamma[0];
        memmove(ctx->reg, ctx->reg + 1, MAGMA_BLOCK_SIZE - 1);

        // используем байт зашифрованного текста
        ctx->reg[MAGMA_BLOCK_SIZE - 1] = in[i];
    }
}

```

magma.c

Ядро шифра Магма, функции, которые выполняют инициализацию 256-битного ключа, шифрование и расшифрование 64-битных блоков данных с использованием 32-раундовой сети Фейстеля, S-блоков из стандарта и правильного расписания ключей.

```

#include "magma.h"
#include <string.h>

// Реализация стандарта Магма (реализованы корректная упаковка/распаковка, S-блоки и 32 раунда сети Фейстеля
// Здесь используется представление: входной блок 64-бита разбивается на N1 и N2

static const uint8_t Sbox[8][16] = {
    {0xF,0xC,0x2,0xA,0x6,0x4,0x5,0x0,0x7,0x9,0xE,0xD,0x1,0xB,0x8,0x3},
    {0xB,0x6,0x3,0x4,0xC,0xF,0xE,0x2,0x7,0xD,0x8,0x0,0x5,0xA,0x9,0x1},
    {0x1,0xC,0xB,0x0,0xF,0xE,0x6,0x5,0xA,0xD,0x4,0x8,0x9,0x3,0x7,0x2},
    {0x1,0x5,0xE,0xC,0xA,0x7,0x0,0xD,0x6,0x2,0xB,0x4,0x9,0x3,0xF,0x8},
    {0x0,0xC,0x8,0x9,0xD,0x2,0xA,0xB,0x7,0x3,0x6,0x5,0x4,0xE,0xF,0x1},
    {0x8,0x0,0xF,0x3,0x2,0x5,0xE,0xB,0x1,0xA,0x4,0x7,0xC,0x9,0xD,0x6},
    {0x3,0x0,0x6,0xF,0x1,0xE,0x9,0x2,0xD,0x8,0xC,0x4,0xB,0xA,0x5,0x7},
    {0x1,0xA,0x6,0x8,0xF,0xB,0x0,0x4,0xC,0x3,0x5,0x9,0x7,0xD,0x2,0xE}
};

// Циклический поворот 32 бит
static inline uint32_t rol32(uint32_t x, unsigned n) {
    return (x << n) | (x >> (32 - n));
}

// S-образная замена 32-разрядного слова ( как 8 фрагментов)
static uint32_t substitute32(uint32_t x) {
    // блоки
    uint8_t nibbles[8];

```

```

    for (int i = 0; i < 8; ++i) {
        nibbles[i] = (x >> (28 - 4 * i)) & 0x0F;
    }

    for (int i = 0; i < 8; ++i) {
        nibbles[i] = Sbox[i][nibbles[i]];
    }

    uint32_t y = 0;

    for (int i = 0; i < 8; ++i) {
        y |= ((uint32_t)nibbles[i] << (28 - 4 * i));
    }

    return y;
}

// === F-функция Магмы === //
static uint32_t F(uint32_t x, uint32_t k) {
    // Сложение по модулю 2^32
    uint32_t t = x + k;
    uint32_t s = substitute32(t);
    return rol32(s, 11);
}

// Установка ключа
void MagmaSetKey(magma_key_t* key, const uint8_t user_key[32]) {
    // Ключ в порядке возрастания: ключ[0]
    for (int i = 0; i < 8; ++i) {
        uint32_t w = ((uint32_t)user_key[4 * i + 0] << 24) |
            ((uint32_t)user_key[4 * i + 1] << 16) |
            ((uint32_t)user_key[4 * i + 2] << 8) |
            ((uint32_t)user_key[4 * i + 3]);

        key->k[i] = w;
    }
}

// === Внутренний общий код 32-раундовой сети Фейстеля === //
static void MagmaFeistel32(const magma_key_t* key, uint32_t* N1, uint32_t* N2, int decrypt) {
    uint32_t n1 = *N1, n2 = *N2;

    if (!decrypt) {
        // Шифрование: K0-K7 повторяется 3 раза, затем K7-K0
        for (int r = 0; r < 24; ++r) {
            uint32_t t = n1;
            n1 = n2 ^ F(n1, key->k[r % 8]);
            n2 = t;
        }

        for (int r = 7; r >= 0; --r) {
            uint32_t t = n1;
            n1 = n2 ^ F(n1, key->k[r]);
            n2 = t;
        }
    }
    else {
        // Расшифровка выполняется по обратному процессу
        for (int r = 0; r < 8; ++r) {
            uint32_t t = n1;
            n1 = n2 ^ F(n1, key->k[r]);
            n2 = t;
        }

        for (int r = 31; r >= 8; --r) {
            uint32_t t = n1;
            n1 = n2 ^ F(n1, key->k[r % 8]);
            n2 = t;
        }
    }
}

```

```

}

// После раундов результат запишем в регистры
*N1 = n1;
*N2 = n2;
}

// Упаковывать/распаковывать 32-разрядные слова с большими порядковыми номерами в байты или из них
static inline uint32_t be32(const uint8_t b[4]) {
    return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16) | ((uint32_t)b[2] << 8) | (uint32_t)b[3];
}

static inline void store_be32(uint8_t out[4], uint32_t v) {
    out[0] = (uint8_t)(v >> 24);
    out[1] = (uint8_t)(v >> 16);
    out[2] = (uint8_t)(v >> 8);
    out[3] = (uint8_t)(v);
}

// === Шифрование и расшифрование через сеть Фейстеля === //
void MagmaEncryptBlock(const magma_key_t* key, const uint8_t in[8], uint8_t out[8]) {
    uint32_t n1 = be32(in + 0);
    uint32_t n2 = be32(in + 4);
    MagmaFeistel32(key, &n1, &n2, 0);

    // Вывод в том же порядке: 8 байт
    store_be32(out + 0, n1);
    store_be32(out + 4, n2);
}

void MagmaDecryptBlock(const magma_key_t* key, const uint8_t in[8], uint8_t out[8]) {
    uint32_t n1 = be32(in + 0);
    uint32_t n2 = be32(in + 4);
    MagmaFeistel32(key, &n1, &n2, 1);

    store_be32(out + 0, n1);
    store_be32(out + 4, n2);
}

```

fileIO.c

Позволяет безопасно работать с файлами для записи, чтения исходного текста и ключа, исходный код лежит в отдельном файле.

```

#include "fileIO.h"
#include <stdio.h>
#include <stdlib.h>

uint8_t* read_file(const char *path, size_t *outlen) {
    FILE* f = fopen(path, "rb");

    if (!f) return NULL;

    if (fseek(f, 0, SEEK_END) != 0) {
        fclose(f); return NULL;
    }

    long sz = ftell(f);

    if (sz < 0) {
        fclose(f); return NULL;
    }

    rewind(f);
    uint8_t* buf = (uint8_t*)malloc((size_t)sz + 1);

```



```

    if (!buf) {
        fclose(f); return NULL;
    }

    size_t r = fread(buf, 1, (size_t)sz, f);
    fclose(f);

    if (r != (size_t)sz) {
        free(buf); return NULL;
    }

    *outlen = (size_t)sz;
    return buf;
}

uint8_t* read_file_w(const wchar_t* path, size_t* outlen) {
    if (!path || !outlen) return NULL;
    FILE* f = _wfopen(path, L"rb");
    if (!f) return NULL;

    if (_fseeki64(f, 0, SEEK_END) != 0) {
        fclose(f);
        return NULL;
    }

    __int64 sz = _ftelli64(f);
    if (sz < 0 || sz > SIZE_MAX) {
        fclose(f);
        return NULL;
    }

    _fseeki64(f, 0, SEEK_SET);

    uint8_t* buf = (uint8_t*)malloc((size_t)sz + 1);
    if (!buf) {
        fclose(f);
        return NULL;
    }

    size_t r = fread(buf, 1, (size_t)sz, f);
    fclose(f);

    if (r != (size_t)sz) {
        free(buf);
        return NULL;
    }

    *outlen = (size_t)sz;
    return buf;
}

int write_file(const char* path, const uint8_t* buf, size_t len) {
    FILE* f = fopen(path, "wb");
    if (!f) return 0;
    size_t w = fwrite(buf, 1, len, f);
    fclose(f);
    return w == len;
}

// Тип callback остаётся прежним
typedef void (*process_block_cb)(void* ctx, const uint8_t* in, uint8_t* out, size_t n);

int process_file_stream_w(const wchar_t* inpath, const wchar_t* outpath, void* ctx_cb, process_block_cb cb) {
    FILE* fin = _wfopen(inpath, L"rb");
    if (!fin) return 0;

    FILE* fout = _wfopen(outpath, L"wb");
    if (!fout) {
        fclose(fin);
        return 0;
    }

```

```

    }

    const size_t BUF = 4096;
    uint8_t* inbuf = (uint8_t*)malloc(BUF);
    uint8_t* outbuf = (uint8_t*)malloc(BUF);
    if (!inbuf || !outbuf) {
        fclose(fin); fclose(fout);
        free(inbuf); free(outbuf);
        return 0;
    }

    size_t n;
    int ok = 1;
    while ((n = fread(inbuf, 1, BUF, fin)) > 0) {
        cb(ctx_cb, inbuf, outbuf, n);
        if (fwrite(outbuf, 1, n, fout) != n) {
            ok = 0;
            break;
        }
    }

    fclose(fin);
    fclose(fout);
    free(inbuf);
    free(outbuf);
    return ok;
}

```

kdf.c и sha256.c

Данный код реализует криптографическую функцию хэширования SHA-256 в соответствии со стандартом FIPS, а также простую функцию выработки ключа (KDF) на её основе. Он состоит из двух частей: первая — полноценная потоковая реализация SHA-256, поддерживающая инициализацию, пошаговое обновление хэша и финализацию, что позволяет хэшировать данные любого объёма, включая файлы. Вторая — это функция `kdf_sha256`, которая принимает произвольные входные данные (например, пароль или содержимое файла) и преобразует их в 256-битный (32-байтный) ключ с помощью SHA-256. Эта связка используется в проекте для генерации валидного криптографического ключа для алгоритма Магма из любого пользовательского ввода, обеспечивая при этом соответствие требованиям стандарта (фиксированная длина ключа, высокая энтропия).

```

#include "kdf.h"
#include "sha256.h"

void kdf_sha256(const uint8_t* input, size_t input_len, uint8_t out[32]) {
    sha256_hash(input, input_len, out);
}

#include "sha256.h"
#include <string.h>

static const uint32_t K[64] = {

```

```

0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
0xe49b69c1, 0xfeb4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
};

```

```

#define Ch(x, y, z) ((x & y) ^ (~x & z))
#define Maj(x, y, z) ((x & y) ^ (x & z) ^ (y & z))
#define RotR(x, n) ((x >> n) | (x << (32 - n)))
#define Sigma0(x) (RotR(x, 2) ^ RotR(x, 13) ^ RotR(x, 22))
#define Sigma1(x) (RotR(x, 6) ^ RotR(x, 11) ^ RotR(x, 25))
#define sigma0(x) (RotR(x, 7) ^ RotR(x, 18) ^ (x >> 3))
#define sigma1(x) (RotR(x, 17) ^ RotR(x, 19) ^ (x >> 10))

```

```

static const uint32_t H[8] = {
    0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
    0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
};

```

```

void sha256_init(SHA256_CTX* ctx) {
    memcpy(ctx->state, H, sizeof(H));
    ctx->count = 0;
    memset(ctx->buffer, 0, sizeof(ctx->buffer));
}

```

```

void sha256_transform(SHA256_CTX* ctx, const uint8_t* block) {
    uint32_t W[64];
    uint32_t S[8];
    int i;

```

```

    for (i = 0; i < 16; ++i) {
        W[i] = ((uint32_t)block[i * 4 + 0] << 24) |
            ((uint32_t)block[i * 4 + 1] << 16) |
            ((uint32_t)block[i * 4 + 2] << 8) |
            ((uint32_t)block[i * 4 + 3]);
    }

```

```

    for (; i < 64; ++i) {
        W[i] = sigma1(W[i - 2]) + W[i - 7] + sigma0(W[i - 15]) + W[i - 16];
    }

```

```

    memcpy(S, ctx->state, 32);

```

```

    for (i = 0; i < 64; ++i) {
        uint32_t T1 = S[7] + Sigma1(S[4]) + Ch(S[4], S[5], S[6]) + K[i] + W[i];
        uint32_t T2 = Sigma0(S[0]) + Maj(S[0], S[1], S[2]);
        memmove(&S[1], &S[0], 7 * sizeof(uint32_t));
        S[4] += T1;
        S[0] = T1 + T2;
    }

```

```

    for (i = 0; i < 8; ++i) {
        ctx->state[i] += S[i];
    }
}

```

```

void sha256_update(SHA256_CTX* ctx, const uint8_t* data, size_t len) {
    size_t i, j;

```

```

    j = (size_t)((ctx->count >> 3) & 0x3F);
    ctx->count += (uint64_t)len << 3;

```

```

    if ((j + len) > 63) {

```

```

        memcpy(&ctx->buffer[j], data, (i = 64 - j));
        sha256_transform(ctx, ctx->buffer);

```

```

        for (; i + 63 < len; i += 64) {
            sha256_transform(ctx, &data[i]);
        }
        j = 0;
    }
    else {
        i = 0;
    }
}

```

```

memcpy(&ctx->buffer[j], &data[i], len - i);
}

void sha256_final(SHA256_CTX* ctx, uint8_t digest[SHA256_DIGEST_SIZE]) {
    uint64_t bitlen = ctx->count;
    size_t i = (size_t)((bitlen >> 3) & 0x3F);
    ctx->buffer[i++] = 0x80;

    if (i > 56) {
        memset(&ctx->buffer[i], 0, 64 - i);
        sha256_transform(ctx, ctx->buffer);
        memset(ctx->buffer, 0, 56);
    }
    else {
        memset(&ctx->buffer[i], 0, 56 - i);
    }

    ctx->buffer[56] = (uint8_t)(bitlen >> 56);
    ctx->buffer[57] = (uint8_t)(bitlen >> 48);
    ctx->buffer[58] = (uint8_t)(bitlen >> 40);
    ctx->buffer[59] = (uint8_t)(bitlen >> 32);
    ctx->buffer[60] = (uint8_t)(bitlen >> 24);
    ctx->buffer[61] = (uint8_t)(bitlen >> 16);
    ctx->buffer[62] = (uint8_t)(bitlen >> 8);
    ctx->buffer[63] = (uint8_t)(bitlen);

    sha256_transform(ctx, ctx->buffer);

    for (i = 0; i < 8; ++i) {
        digest[i * 4 + 0] = (uint8_t)(ctx->state[i] >> 24);
        digest[i * 4 + 1] = (uint8_t)(ctx->state[i] >> 16);
        digest[i * 4 + 2] = (uint8_t)(ctx->state[i] >> 8);
        digest[i * 4 + 3] = (uint8_t)(ctx->state[i]);
    }
}

void sha256_hash(const uint8_t* data, size_t len, uint8_t out[SHA256_DIGEST_SIZE]) {
    SHA256_CTX ctx;
    sha256_init(&ctx);
    sha256_update(&ctx, data, len);
    sha256_final(&ctx, out);
}

```

ui.c

Визуальное отображение компонентов, которые включают кнопки для работы с файлами (загрузка), шифрование/расшифрование, поле для записи IV синхропосылки и ключа.

```

#define _CRT_SECURE_NO_WARNINGS
#define UNICODE
#define _WIN32_WINNT 0x0600

#include <windows.h>
#include <commdlg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <wctype.h>
#include <commctrl.h>

#include "ui.h"
#include "student_info.h"
#include "cipher.h"
#include "fileIO.h"
#include "kdf.h"

#pragma comment(linker, "\"/manifestdependency:type='win32' name='Microsoft.Windows.Common-Controls' version='6.0.0.0' processorArchitecture='*' publicKeyToken='6595b64144ccf1df' language='*'\")")
#pragma comment(lib, "comctl32.lib")

```

```

enum {
    IDC_EDIT_STUDENT = 1001,
    IDC_EDIT_VARIANT,
    IDC_EDIT_KEY,
    IDC_BTN_KEY_FILE,
    IDC_EDIT_IV,
    IDC_BTN_IN,
    IDC_BTN_OUT,
    IDC_STATIC_IN,
    IDC_STATIC_OUT,
    IDC_BTN_ENC,
    IDC_BTN_DEC,
    IDC_EDIT_LOG
};

static HINSTANCE ghInstance;
static HWND hEditStudent, hEditVariant, hEditKey, hBtnKeyFile, hEditIV;
static HWND hBtnIn, hBtnOut, hBtnEnc, hBtnDec, hEditLog, hStaticIn, hStaticOut;
static WCHAR inPath[MAX_PATH] = L"";
static WCHAR outPath[MAX_PATH] = L"";

// Вспомогательная функция: char* → wchar_t*
static wchar_t* ansi_to_wide(const char* src) {
    static wchar_t buf[512];

    if (!src) return L"";

    int len = MultiByteToWideChar(CP_UTF8, 0, src, -1, NULL, 0);
    if (len > 512) len = 512;

    MultiByteToWideChar(CP_UTF8, 0, src, -1, buf, len);
    return buf;
}

// ЛОГИРОВАНИЕ В ПРИЛОЖЕНИИ
static void append_log(const wchar_t* fmt, ...) {
    wchar_t buf[1024];
    va_list ap;
    va_start(ap, fmt);
    vswprintf(buf, sizeof(buf) / sizeof(buf[0]), fmt, ap);
    va_end(ap);
    int len = GetWindowTextLengthW(hEditLog);
    SendMessageW(hEditLog, EM_SETSEL, (WPARAM)len, (LPARAM)len);
    SendMessageW(hEditLog, EM_REPLACESEL, FALSE, (LPARAM)buf);
    SendMessageW(hEditLog, EM_REPLACESEL, FALSE, (LPARAM)L"\r\n");
}

static int hex_nibble(wchar_t c) {
    if (c >= L'0' && c <= L'9') return c - L'0';
    if (c >= L'A' && c <= L'F') return c - L'A' + 10;
    if (c >= L'a' && c <= L'f') return c - L'a' + 10;

    return -1;
}

static int hexw_to_bytes(const wchar_t* whex, uint8_t* out, size_t expected_len) {
    if (!whex || !out) return 0;
    size_t byte_idx = 0;
    int hi = -1;

    for (size_t i = 0; whex[i] != L'\0' && byte_idx < expected_len; i++) {

        wchar_t wc = whex[i];
        if (wc > 127) continue;

        int nib = hex_nibble(wc);

        if (nib == -1) continue;

        if (hi == -1) {
            hi = nib;
        }
        else {
            out[byte_idx++] = (uint8_t)((hi << 4) | nib);
            hi = -1;
        }
    }

    if (hi != -1 || byte_idx != expected_len) return 0;
}

```

```

    return 1;
}

static size_t clean_hex_string(const wchar_t* src, wchar_t* dst, size_t dst_size) {
    size_t j = 0;

    for (size_t i = 0; src[i] != L'\0' && j + 1 < dst_size; i++) {
        wchar_t c = src[i];

        if ((c >= L'0' && c <= L'9') ||
            (c >= L'A' && c <= L'F') ||
            (c >= L'a' && c <= L'f')) {
            dst[j++] = c;
        }
    }

    dst[j] = L'\0';
    return j;
}

static void choose_open_file(HWND owner, WCHAR* outbuf) {
    OPENFILENAMEW ofn = { 0 };
    WCHAR szFile[MAX_PATH] = L"";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = owner;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;
    ofn.lpstrFilter = L"All Files\0*.*\0";

    if (GetOpenFileNameW(&ofn)) {
        wcscpy_s(outbuf, MAX_PATH, szFile);
    }
}

static void choose_save_file(HWND owner, WCHAR* outbuf) {
    OPENFILENAMEW ofn = { 0 };
    WCHAR szFile[MAX_PATH] = L"";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = owner;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_OVERWRITEPROMPT;
    ofn.lpstrFilter = L"All Files\0*.*\0";

    if (GetSaveFileNameW(&ofn)) {
        wcscpy_s(outbuf, MAX_PATH, szFile);
    }
}

typedef struct {
    magma_cfb_t* ctx;
} cb_ctx_t;

static void proc_cb_encrypt(void* ctxv, const uint8_t* in, uint8_t* out, size_t n) {
    cb_ctx_t* c = (cb_ctx_t*)ctxv;
    magma_cfb_encrypt_stream(c->ctx, in, out, n);
}

static void proc_cb_decrypt(void* ctxv, const uint8_t* in, uint8_t* out, size_t n) {
    cb_ctx_t* c = (cb_ctx_t*)ctxv;
    magma_cfb_decrypt_stream(c->ctx, in, out, n);
}

// ===== OKHO "O CTYДЕНTE" =====
static void show_student_info(HWND parent) {
    WCHAR msg[1024];
    swprintf_s(msg, _countof(msg),
        L"Студент: %ls\n"
        L"Группа: %ls\n"
        L"Вариант: %ls\n\n"
        L"Задание:\n%ls",
        STUDENT_FIO, STUDENT_GROUP, STUDENT_VARIANT, STUDENT_TASK);

    TASKDIALOGCONFIG tdf = { 0 };
    tdf.cbSize = sizeof(tdf);
    tdf.hwndParent = parent;
    tdf.dwFlags = TDF_ALLOW_DIALOG_CANCELLATION | TDF_POSITION_RELATIVE_TO_WINDOW;
    tdf.pszWindowTitle = L"Курсовая работа — Магма (CFB)";
    tdf.pszMainIcon = TD_INFORMATION_ICON;

```

```

    tdf.pszMainInstruction = L"Сведения о студенте";
    tdf.pszContent = msg;
    tdf.pszExpandedInformation = L"Реализация криптографического алгоритма «Магма» (ГОСТ Р 34.12-2015) в режиме гаммирования с
    обратной связью.";
    tdf.pszFooter = L"ГОСТ Р 34.12-2015 | Учебное приложение";
    TaskDialogIndirect(&tdf, NULL, NULL, NULL);
}

// ===== ОЧОБХОЕ ОКНО =====
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            HFONT hFont = CreateFontW(16, 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE,
            DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
            DEFAULT_PITCH | FF_DONTCARE, L"Segoe UI");
            HFONT hBtnFont = CreateFontW(15, 0, 0, 0, FW_MEDIUM, FALSE, FALSE, FALSE,
            DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
            DEFAULT_PITCH | FF_DONTCARE, L"Segoe UI");
            HFONT hLogFont = CreateFontW(14, 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE,
            DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
            FIXED_PITCH | FF_DONTCARE, L"Consolas");

            CreateWindowW(L"STATIC", L"Студент:", WS_VISIBLE | WS_CHILD, 10, 10, 80, 24, hwnd, NULL, ghInstance, NULL);
            hEditStudent = CreateWindowW(L"EDIT", STUDENT_FIO, WS_VISIBLE | WS_CHILD | WS_BORDER | ES_READONLY, 90, 10, 300,
            24, hwnd, (HMENU)IDC_EDIT_STUDENT, ghInstance, NULL);
            SendMessageW(hEditStudent, WM_SETFONT, (WPARAM)hFont, TRUE);

            CreateWindowW(L"STATIC", L"Вариант:", WS_VISIBLE | WS_CHILD, 10, 42, 80, 24, hwnd, NULL, ghInstance, NULL);
            hEditVariant = CreateWindowW(L"EDIT", ansi_to_wide(STUDENT_VARIANT), WS_VISIBLE | WS_CHILD | WS_BORDER |
            ES_READONLY, 90, 42, 160, 24, hwnd, (HMENU)IDC_EDIT_VARIANT, ghInstance, NULL);
            SendMessageW(hEditVariant, WM_SETFONT, (WPARAM)hFont, TRUE);

            CreateWindowW(L"STATIC", L"Ключ (hex или пароль):", WS_VISIBLE | WS_CHILD, 10, 74, 200, 24, hwnd, NULL, ghInstance,
            NULL);
            hEditKey = CreateWindowW(L"EDIT", L"", WS_VISIBLE | WS_CHILD | WS_BORDER, 220, 74, 520, 24, hwnd,
            (HMENU)IDC_EDIT_KEY, ghInstance, NULL);
            SendMessageW(hEditKey, WM_SETFONT, (WPARAM)hFont, TRUE);

            hBtnKeyFile = CreateWindowW(L"BUTTON", L"Загрузить ключ...", WS_VISIBLE | WS_CHILD, 770, 74, 150, 26, hwnd,
            (HMENU)IDC_BTN_KEY_FILE, ghInstance, NULL);
            SendMessageW(hBtnKeyFile, WM_SETFONT, (WPARAM)hBtnFont, TRUE);

            CreateWindowW(L"STATIC", L"IV (hex, 16 символов):", WS_VISIBLE | WS_CHILD, 10, 106, 160, 24, hwnd, NULL, ghInstance,
            NULL);
            hEditIV = CreateWindowW(L"EDIT", L"0001020304050607", WS_VISIBLE | WS_CHILD | WS_BORDER, 180, 106, 180, 24, hwnd,
            (HMENU)IDC_EDIT_IV, ghInstance, NULL);
            SendMessageW(hEditIV, WM_SETFONT, (WPARAM)hFont, TRUE);
            SendMessageW(hEditIV, EM_SETLIMITTEXT, 16, 0);

            hBtnIn = CreateWindowW(L"BUTTON", L"Входной файл...", WS_VISIBLE | WS_CHILD, 10, 142, 160, 28, hwnd,
            (HMENU)IDC_BTN_IN, ghInstance, NULL);
            SendMessageW(hBtnIn, WM_SETFONT, (WPARAM)hBtnFont, TRUE);
            hBtnOut = CreateWindowW(L"BUTTON", L"Выходной файл...", WS_VISIBLE | WS_CHILD, 180, 142, 160, 28, hwnd,
            (HMENU)IDC_BTN_OUT, ghInstance, NULL);
            SendMessageW(hBtnOut, WM_SETFONT, (WPARAM)hBtnFont, TRUE);

            hBtnEnc = CreateWindowW(L"BUTTON", L"Зашифровать", WS_VISIBLE | WS_CHILD, 360, 142, 140, 32, hwnd,
            (HMENU)IDC_BTN_ENC, ghInstance, NULL);
            SendMessageW(hBtnEnc, WM_SETFONT, (WPARAM)hBtnFont, TRUE);
            hBtnDec = CreateWindowW(L"BUTTON", L"Расшифровать", WS_VISIBLE | WS_CHILD, 510, 142, 140, 32, hwnd,
            (HMENU)IDC_BTN_DEC, ghInstance, NULL);
            SendMessageW(hBtnDec, WM_SETFONT, (WPARAM)hBtnFont, TRUE);

            CreateWindowW(L"STATIC", L"Лог операций:", WS_VISIBLE | WS_CHILD, 10, 182, 120, 24, hwnd, NULL, ghInstance, NULL);
            hEditLog = CreateWindowW(L"EDIT", L"", WS_VISIBLE | WS_CHILD | WS_BORDER | ES_LEFT | ES_MULTILINE |
            ES_AUTOVSCROLL | ES_READONLY | WS_VSCROLL,
            10, 208, 910, 260, hwnd, (HMENU)IDC_EDIT_LOG, ghInstance, NULL);
            SendMessageW(hEditLog, WM_SETFONT, (WPARAM)hLogFont, TRUE);

            append_log(L"Приложение готово к работе.");
            break;
        }
        case WM_COMMAND: {
            int id = LOWORD(wParam);

            if (id == IDC_BTN_KEY_FILE) {

                WCHAR path[MAX_PATH] = L"";
                choose_open_file(hwnd, path);
            }
        }
    }
}

```

```

if (path[0]) {

    size_t keylen;
    uint8_t* kbuf = read_file_w(path, &keylen);

    if (!kbuf) {
        append_log(L"Ошибка: не удалось прочитать файл ключа: %s", path);
    }
    else {
        uint8_t final_key[32];
        kdf_sha256(kbuf, keylen, final_key);
        append_log(L"Ключ из файла нормируется через SHA-256 для соответствия hex-ключ (64 символа): %s", path);

        WCHAR whex[65];

        for (int i = 0; i < 32; i++) {
            wsprintfW(&whex[i * 2], L"%02X", final_key[i]);
        }

        whex[64] = L'\0';
        SetWindowTextW(hEditKey, whex);
        free(kbuf);
    }
}
else if (id == IDC_BTN_IN) {
    choose_open_file(hwnd, inPath);
    if (inPath[0]) append_log(L"Входной файл: %s", inPath);
}
else if (id == IDC_BTN_OUT) {
    choose_save_file(hwnd, outPath);
    if (outPath[0]) append_log(L"Выходной файл: %s", outPath);
}
else if (id == IDC_BTN_ENC || id == IDC_BTN_DEC) {
    if (!inPath[0] || !outPath[0]) {
        append_log(L"Ошибка: выберите входной и выходной файлы.");
        break;
    }
}

WCHAR keyw[512];
GetWindowTextW(hEditKey, keyw, _countof(keyw));
uint8_t key_raw[32];

WCHAR clean_key[128] = { 0 };
size_t hex_count = clean_hex_string(keyw, clean_key, _countof(clean_key));

if (hex_count == 64) {

    if (hexw_to_bytes(clean_key, key_raw, 32)) {
        append_log(L"Используется введенный hex-ключ (64 символа).");
    }
    else {
        append_log(L"Ошибка: не удалось распарсить hex-ключ.");
        break;
    }
}
else {
    append_log(L"Введен не hex-ключ (%zu символов) → хэшируем как пароль.", hex_count);
    int len_utf8 = WideCharToMultiByte(CP_UTF8, 0, keyw, -1, NULL, 0, NULL, NULL);

    if (len_utf8 <= 1) {
        append_log(L"Ошибка: пустой ввод.");
        break;
    }

    char* key_utf8 = (char*)malloc(len_utf8);
    WideCharToMultiByte(CP_UTF8, 0, keyw, -1, key_utf8, len_utf8, NULL, NULL);
    kdf_sha256((uint8_t*)key_utf8, len_utf8 - 1, key_raw);
    free(key_utf8);
}

WCHAR ivw[64];
GetWindowTextW(hEditIV, ivw, _countof(ivw));
WCHAR clean_iv[32] = { 0 };
size_t iv_hex = clean_hex_string(ivw, clean_iv, _countof(clean_iv));

if (iv_hex != 16) {
    append_log(L"Ошибка: IV должен быть ровно 16 hex-символов. Фактически: %zu", iv_hex);
    break;
}

```



```

    }

    uint8_t iv[8];

    if (!hexw_to_bytes(clean_iv, iv, 8)) {
        append_log(L"Ошибка: не удалось распарсить IV.");
        break;
    }

    magma_cfb_t ctx;
    magma_cfb_initialization(&ctx, key_raw, iv);
    cb_ctx_t cbctx = { .ctx = &ctx };

    int ok = process_file_stream_w(inPath, outPath, &cbctx,
        (id == IDC_BTN_ENC) ? proc_cb_encrypt : proc_cb_decrypt);

    if (ok) {
        append_log(L"Успешно: %s", outPath);
    }
    else {
        append_log(L"Ошибка при обработке файлов!");
    }
    }
    break;
}

case WM_DESTROY:
    PostQuitMessage(0);
    break;
default:
    return DefWindowProcW(hwnd, msg, wParam, lParam);
}
return 0;
}

int run_gui(HINSTANCE hInstance, int nCmdShow) {
    ghInstance = hInstance;
    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = L"MagmaCFBWin";
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = CreateSolidBrush(RGB(248, 248, 250));
    RegisterClassW(&wc);

    HWND hwnd = CreateWindowW(
        wc.lpszClassName,
        L"Магма (CFB) — Демонстрация шифрования",
        WS_OVERLAPPEDWINDOW & ~WS_THICKFRAME,
        CW_USEDEFAULT, CW_USEDEFAULT, 950, 520,
        NULL, NULL, hInstance, NULL
    );

    if (!hwnd) return -1;

    // Показываем информацию о студенте
    show_student_info(hwnd);

    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG msg;
    while (GetMessageW(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    return (int)msg.wParam;
}

```

Заголовочные файлы

Данные файлы предназначены для работы с исходным кодом во всех файлах, предоставляя вызовы функции, без заголовочных файлов — это не предоставляется возможным.

```

#pragma once
#ifndef CIPHER_H
#define CIPHER_H

#include <stdint.h>
#include <stddef.h>
#include "magma.h"

// == Структура CFB-контекста (сохранение в регистр) == //
typedef struct {
    magma_key_t key;
    uint8_t reg[MAGMA_BLOCK_SIZE]; // 8 байт регистр (IV)
} magma_cfb_t;

// == Инициализация key_raw 32 байта, IV 8 байт == //
void magma_cfb_initialization(magma_cfb_t *ctx, const uint8_t key_raw[32], const uint8_t iv[8]);

// == Шифрование/дешифрование потоков произвольной длины == //
// decrypt и encrypt используют разные обновления регистра
void magma_cfb_encrypt_stream(magma_cfb_t *ctx, const uint8_t *in, uint8_t *out, size_t len);

void magma_cfb_decrypt_stream(magma_cfb_t *ctx, const uint8_t *in, uint8_t *out, size_t len);
#endif

#pragma once
#ifndef FILE_IO_H
#define FILE_IO_H

#include <stddef.h>
#include <stdint.h>

/* Простая функция чтения файла в динамический буфер.
   Возвращает указатель длину через outlen. Возвращает NULL при ошибке. */
uint8_t* read_file(const char *path, size_t *outlen);

uint8_t* read_file_w(const wchar_t* path, size_t* outlen);

// Запись буфера в файл. Возвращает 1 при успехе, 0 при ошибке
int write_file(const char *path, const uint8_t *buf, size_t len);

/* Поблочная обработка (считывание входа и запись результат с вызовом
   для обработки блоков */
typedef void (*process_block_cb)(void *ctx, const uint8_t *in, uint8_t *out, size_t n);

int process_file_stream_w(const wchar_t *inpath, const wchar_t *outpath, void *ctx_cb, process_block_cb cb);
#endif

#pragma once
#include <stdint.h>
#include <stddef.h>

// Безопасное получение 32-байтного ключа через SHA-256
void kdf_sha256(const uint8_t *input, size_t input_len, uint8_t out[32]);

#pragma once
#ifndef MAGMA_H
#define MAGMA_H

#include <stdint.h>
#include <stddef.h>

#define MAGMA_BLOCK_SIZE 8U

typedef struct {
    // 8 * 32 = 256 бит
    uint32_t k[8];
} magma_key_t;

// == Инициализация ключа (32 байта) == //
void MagmaSetKey(magma_key_t *key, const uint8_t user_key[32]);

// == Шифрование одного блока (8 байт) == //
void MagmaEncryptBlock(const magma_key_t *key, const uint8_t in[8], uint8_t out[8]);

// == Расшифровка одного блока (8 байт) == //
void MagmaDecryptBlock(const magma_key_t *key, const uint8_t in[8], uint8_t out[8]);
#endif

#pragma once

```

```

#include <stdint.h>
#include <stddef.h>

#define SHA256_DIGEST_SIZE 32

typedef struct {
    uint32_t state[8];
    uint64_t count;
    uint8_t buffer[64];
} SHA256_CTX;

void sha256_init(SHA256_CTX* ctx);
void sha256_update(SHA256_CTX* ctx, const uint8_t* data, size_t len);
void sha256_final(SHA256_CTX* ctx, uint8_t digest[SHA256_DIGEST_SIZE]);

// Удобная обёртка: хэширует всё за один вызов
void sha256_hash(const uint8_t* data, size_t len, uint8_t out[SHA256_DIGEST_SIZE]);

#pragma once
#ifndef STUDENT_INFO_H
#define STUDENT_INFO_H

#define STUDENT_FIO L"ФИО: Белянин Никита Николаевич"
#define STUDENT_GROUP L"Группа: ПИбд-41"
#define STUDENT_VARIANT "6"
#define STUDENT_TASK L"Лабораторная работа 3: Реализация шифра Магма (ГОСТ 28147) - режим CFB (гаммирование с обратной связью)"
#endif

#pragma once
#ifndef UI_H
#define UI_H
#include <Windows.h>
#include <stdint.h>

// Запуск GUI
int run_gui(HINSTANCE hInstance, int nCmdShow);

// Функция логирования для GUI
void append_log(const wchar_t* fmt, ...);
#endif

```

Реализация на Java

Main.java

Этот файл содержит точку входа в приложение (main метод). Он отвечает за запуск графического интерфейса Swing. Внутри потока он пытается установить системный внешний вид для элементов интерфейса: отображает окно справки о студенте и главное окно приложения.

```

package org.encrypting;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.io.File;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;

public class Main {
    private JFrame mainFrame;
    private JTextField keyField;
    private JTextField ivField;
    private File inputFile, outputFile;

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Main().showStudentInfo());
    }

    private void showStudentInfo() {

```

```

JOptionPane.showMessageDialog(
    null,
    "<html><b>Студент:</b> " + StudentInfo.FIO + "<br>" +
    "<b>Группа:</b> " + StudentInfo.GROUP + "<br>" +
    "<b>Вариант:</b> " + StudentInfo.VARIANT + "<br><br>" +
    "<b>Задание:</b><br>" + StudentInfo.TASK + "</html>",
    "Сведения о студенте",
    JOptionPane.INFORMATION_MESSAGE
);
createAndShowGUI();
}

private void createAndShowGUI() {
    mainFrame = new JFrame("Марма (CFB) — Шифрование");
    mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    mainFrame.setLayout(new BorderLayout());

    // Верхняя панель
    JPanel topPanel = new JPanel(new GridBagLayout());
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.anchor = GridBagConstraints.WEST;

    gbc.gridx = 0; gbc.gridy = 0;
    topPanel.add(new JLabel("Ключ (hex или текст):", gbc);
    gbc.gridx = 1;
    keyField = new JTextField(40);
    topPanel.add(keyField, gbc);
    gbc.gridx = 2;
    JButton loadKeyBtn = new JButton("Загрузить ключ...");
    loadKeyBtn.addActionListener(this::loadKeyFromFile);
    topPanel.add(loadKeyBtn, gbc);

    gbc.gridx = 0; gbc.gridy = 1;
    topPanel.add(new JLabel("IV (16 hex символов):", gbc);
    gbc.gridx = 1;
    ivField = new JTextField("1234567890ABCDEF");
    ivField.setColumns(20);
    topPanel.add(ivField, gbc);

    mainFrame.add(topPanel, BorderLayout.NORTH);

    // Центр: кнопки
    JPanel centerPanel = new JPanel(new FlowLayout());
    JButton selectInput = new JButton("Выбрать входной файл");
    selectInput.addActionListener(e -> selectFile(true));
    JButton selectOutput = new JButton("Выбрать выходной файл");
    selectOutput.addActionListener(e -> selectFile(false));
    JButton encryptBtn = new JButton("Зашифровать");
    encryptBtn.addActionListener(e -> processFile(true));
    JButton decryptBtn = new JButton("Расшифровать");
    decryptBtn.addActionListener(e -> processFile(false));

    centerPanel.add(selectInput);
    centerPanel.add(selectOutput);
    centerPanel.add(encryptBtn);
    centerPanel.add(decryptBtn);
    mainFrame.add(centerPanel, BorderLayout.CENTER);

    // Лог
    JTextArea logArea = new JTextArea(10, 50);
    logArea.setEditable(false);
    logArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
    JScrollPane scroll = new JScrollPane(logArea);
    mainFrame.add(scroll, BorderLayout.SOUTH);

    // Перенаправление System.out в лог
    System.setOut(new PrintStream(new TextAreaOutputStream(logArea)));

    mainFrame.setSize(800, 500);
    mainFrame.setLocationRelativeTo(null);
    mainFrame.setVisible(true);
}

private void loadKeyFromFile(ActionEvent e) {
    JFileChooser chooser = new JFileChooser();
    if (chooser.showOpenDialog(mainFrame) == JFileChooser.APPROVE_OPTION) {
        try {
            String keyText = FileUtils.readTextFile(chooser.getSelectedFile());

```

```

        keyField.setText(keyText);
        System.out.println("Ключ из файла хэширован через SHA-256");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка чтения ключа: " + ex.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
    }
}

private void selectFile(boolean input) {
    JFileChooser chooser = new JFileChooser();
    if (chooser.showOpenDialog(mainFrame) == JFileChooser.APPROVE_OPTION) {
        if (input) {
            inputFile = chooser.getSelectedFile();
            System.out.println("Входной файл: " + inputFile.getAbsolutePath());
        } else {
            outputFile = chooser.getSelectedFile();
            System.out.println("Выходной файл: " + outputFile.getAbsolutePath());
        }
    }
}

private void processFile(boolean encrypt) {
    if (inputFile == null || outputFile == null) {
        JOptionPane.showMessageDialog(mainFrame, "Выберите входной и выходной файлы!", "Ошибка",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    try {
        // Обработка ключа
        byte[] keyBytes = parseKey(keyField.getText());
        // Обработка IV
        byte[] ivBytes = parseIV(ivField.getText());

        MagmaCFB cipher = new MagmaCFB(keyBytes, ivBytes);
        byte[] data = FileUtils.readFile(inputFile);
        byte[] result = encrypt ? cipher.encrypt(data) : cipher.decrypt(data);
        FileUtils.writeFile(outputFile, result);

        String op = encrypt ? "Зашифровано" : "Расшифровано";
        System.out.println(op + ": " + outputFile.getAbsolutePath());
        JOptionPane.showMessageDialog(mainFrame, op + " успешно!", "Успех", JOptionPane.INFORMATION_MESSAGE);

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка: " + ex.getMessage(), "Ошибка", JOptionPane.ERROR_MESSAGE);
        ex.printStackTrace();
    }
}

private byte[] parseKey(String keyInput) {
    // ВСЕГДА трактуем как пароль
    System.out.println("Ключ обработан как пароль → SHA-256");
    return HashUtils.sha256(keyInput.getBytes(StandardCharsets.UTF_8));
}

private byte[] parseIV(String ivInput) {
    ivInput = ivInput.replaceAll("[^0-9A-Fa-f]", "");
    if (ivInput.length() != 16) {
        throw new IllegalArgumentException("IV должен содержать 16 hex-символов");
    }
    return hexToBytes(ivInput);
}

private static byte[] hexToBytes(String hex) {
    byte[] bytes = new byte[hex.length() / 2];
    for (int i = 0; i < bytes.length; i++) {
        bytes[i] = (byte) Integer.parseInt(hex.substring(2 * i, 2 * i + 2), 16);
    }
    return bytes;
}

private static String bytesToHex(byte[] bytes) {
    StringBuilder sb = new StringBuilder();
    for (byte b : bytes) {
        sb.append(String.format("%02X", b));
    }
    return sb.toString();
}

```

```
// Вспомогательный класс для перенаправления вывода в JTextArea
private static class TextAreaOutputStream extends java.io.OutputStream {
    private final JTextArea textArea;

    public TextAreaOutputStream(JTextArea textArea) {
        this.textArea = textArea;
    }

    @Override
    public void write(int b) {
        textArea.append(String.valueOf((char) b));
        textArea.setCaretPosition(textArea.getDocument().getLength());
    }

    @Override
    public void write(byte[] b, int off, int len) {
        textArea.append(new String(b, off, len));
        textArea.setCaretPosition(textArea.getDocument().getLength());
    }
}
}
```

MagmaCFB.java

Данный Java-класс является прямым аналогом классической C-реализации блочного шифра «Магма» в режиме гаммирования с ОС (CFB). Используется S-блоки, которые расширяют 256-битный ключ в 32 раундах и применяется 32-раундовую сеть Фейстеля с корректной F, а в режиме CFB побайтово генерирует гамму, обновляя регистр в соответствии с требованиями стандарта.

```
package org.encrypting;

import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.util.Arrays;

// Реализация блочного шифра "Магма" (ГОСТ Р 34.12-2015) в режиме гаммирования с обратной связью (CFB).
/* Параметры алгоритма: Длина блока 64 бита (8 байт), длина ключа 256 бит (32 байта). Количество раундов: 32
Режим работы: CFB (Cipher Feedback) — потоковый режим, не требует дополнения (padding) */
public class MagmaCFB {

    /* S-блоки (таблицы замен) — фиксированная часть стандарта.
    Используются 8 таблиц по 16 значений (4-битная замена). Значения официально заданы в ГОСТ */
    private static final int[][] S = {
        {12, 8, 2, 10, 6, 4, 14, 5, 11, 13, 9, 7, 15, 3, 1, 0},
        {8, 10, 12, 6, 2, 11, 0, 7, 4, 13, 15, 1, 9, 5, 3, 14},
        {12, 14, 1, 13, 10, 8, 9, 15, 2, 0, 6, 4, 11, 3, 7, 5},
        {13, 7, 5, 12, 10, 9, 0, 6, 1, 11, 14, 8, 3, 15, 2, 4},
        {10, 14, 13, 8, 2, 11, 7, 15, 12, 6, 3, 9, 0, 5, 4, 1},
        {7, 9, 11, 15, 12, 2, 14, 13, 10, 3, 8, 4, 1, 5, 6, 0},
        {14, 6, 3, 5, 10, 15, 8, 12, 11, 1, 4, 7, 9, 13, 0, 2},
        {5, 0, 15, 10, 3, 12, 9, 6, 8, 13, 2, 11, 14, 4, 1, 7}
    };

    // Массив раундовых ключей
    private final int[] roundKeys = new int[32];

    // Вектор инициализации (IV) — 64 бита (8 байт) обеспечивает уникальность гаммы при одинаковом ключе.
    private byte[] iv;

    public MagmaCFB(byte[] userKey, byte[] iv) {
        if (userKey.length != 32) throw new IllegalArgumentException("Ключ должен быть 32 байта");
        if (iv.length != 8) throw new IllegalArgumentException("IV должен быть 8 байт");
        this.iv = iv.clone();
        expandKey(userKey);
    }
}
```

```

// Расширение 256-битного ключа до 32 раундовых 32-битных ключей
private void expandKey(byte[] key) {
    // Интерпретируем байты ключа как 8 32-битных слов
    ByteBuffer bb = ByteBuffer.wrap(key).order(ByteOrder.BIG_ENDIAN);
    int[] k = new int[8];
    for (int i = 0; i < 8; i++) k[i] = bb.getInt();

    // Формируем 32 раундовых ключа K0..K7, K0..K7, K0..K7, K7..K0
    for (int i = 0; i < 24; i++) roundKeys[i] = k[i % 8];
    for (int i = 0; i < 8; i++) roundKeys[24 + i] = k[7 - i];
}

// F-функция (раундовая функция сети Фейстеля)
private int F(int x) {
    int y = 0;

    // Обрабатываем 8 4-битных фрагментов слова (слева направо)
    for (int i = 0; i < 8; i++) {
        int nibble = (x >> (28 - 4 * i)) & 0xF;

        // Применяем замену через i-й S-блок
        int s = S[i][nibble];
        y |= (s << (28 - 4 * i));
    }

    // Циклический сдвиг влево на 11 бит
    return Integer.rotateLeft(y, 11);
}

// Шифрование одного 64-битного блока (основная функция сети Фейстеля)
private void encryptBlock(byte[] in, int inOff, byte[] out, int outOff) {
    ByteBuffer bb = ByteBuffer.wrap(in, inOff, 8).order(ByteOrder.BIG_ENDIAN);
    int n1 = bb.getInt();
    int n2 = bb.getInt();

    // 32 раунда сети Фейстеля
    for (int i = 0; i < 32; i++) {
        int t = n1;
        n1 = n2 ^ F(n1 ^ roundKeys[i]);
        n2 = t;
    }

    bb = ByteBuffer.wrap(out, outOff, 8).order(ByteOrder.BIG_ENDIAN);
    bb.putInt(n1);
    bb.putInt(n2);
}

// Шифрование данных в режиме CFB (гаммирование с обратной связью).
/* Гамма генерируется шифрованием регистра (начинается с IV)
   Каждый байт открытого текста XOR-ится с первым байтом гаммы
   Регистр сдвигается, и в конец добавляется байт шифротекста */
public byte[] encrypt(byte[] data) {
    if (data == null || data.length == 0) return new byte[0];

    byte[] result = new byte[data.length];

    // Инициализация регистра IV
    byte[] reg = iv.clone();

    for (int i = 0; i < data.length; i++) {
        // Генерация 8-байтной гаммы
        byte[] gamma = new byte[8];
        encryptBlock(reg, 0, gamma, 0);

        // XOR открытого текста с первым байтом гаммы
        result[i] = (byte) (data[i] ^ gamma[0]);

        // Обновление регистра
        System.arraycopy(reg, 1, reg, 0, 7);
        reg[7] = result[i];
    }
    return result;
}

// Расшифрование данных в режиме CFB.
public byte[] decrypt(byte[] data) {
    if (data == null || data.length == 0) return new byte[0];
}

```

```

    byte[] result = new byte[data.length];

    // Инициализируем регистр IV
    byte[] reg = iv.clone();

    for (int i = 0; i < data.length; i++) {

        // Генерация гаммы (аналогично шифрованию)
        byte[] gamma = new byte[8];
        encryptBlock(reg, 0, gamma, 0);

        // XOR шифротекста с первым байтом гаммы
        result[i] = (byte) (data[i] ^ gamma[0]);

        // Обновление регистра
        System.arraycopy(reg, 1, reg, 0, 7);
        reg[7] = data[i];
    }
    return result;
}
}

```

HashUtils.java

Данный класс содержит чистую реализацию криптографической хэш-функции SHA-256 на языке Java. Он реализует все этапы алгоритма — от предварительной обработки входных данных (добавление бита 1, выравнивание до 448 бит и добавление 64-битной длины) до итеративного сжатия 512-битных блоков с использованием предопределённых констант К и начальных значений хэш-переменных Н. *Предназначен для выработки 32-байтного ключа из пароля или файла при работе с алгоритмом Магма.*

```

package org.encrypting;

public class HashUtils {

    // Начальные хэш-значения (первые 32 бита дробных частей квадратных корней первых 8 простых чисел)
    private static final int[] H = {
        0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
        0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
    };

    // Константы (первые 32 бита дробных частей кубических корней первых 64 простых чисел)
    private static final int[] K = {
        0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
        0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
        0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
        0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
        0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
        0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
        0x19a44c11, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
        0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
    };

    public static byte[] sha256(byte[] input) {
        // Предварительная обработка
        int length = input.length;
        // Длина в битах
        long bitLength = (long) length << 3;

        // Добавляем бит 1
        int paddingLength = (56 - (length + 1) % 64 + 64) % 64;
        byte[] padded = new byte[length + 1 + paddingLength + 8];
        System.arraycopy(input, 0, padded, 0, length);
        padded[length] = (byte) 0x80;
    }
}

```



```
// Добавляем длину в битах (64-bit)
for (int i = 0; i < 8; i++) {
    padded[padded.length - 8 + i] = (byte) (bitLength >>> (56 - i * 8));
}
```

```
// Инициализация хеш-значений
int[] h = H.clone();
```

```
// Обработка блоков по 512 бит (64 байта)
for (int i = 0; i < padded.length; i += 64) {
```

```
    int[] w = new int[64];
    // Копируем 16 слов
    for (int j = 0; j < 16; j++) {

        w[j] = ((padded[i + j * 4] & 0xFF) << 24) |
            ((padded[i + j * 4 + 1] & 0xFF) << 16) |
            ((padded[i + j * 4 + 2] & 0xFF) << 8) |
            (padded[i + j * 4 + 3] & 0xFF);
    }
```

```
    // Расширяем до 64 слов
    for (int j = 16; j < 64; j++) {

        int s0 = Integer.rotateRight(w[j-15], 7) ^
            Integer.rotateRight(w[j-15], 18) ^
            (w[j-15] >>> 3);
        int s1 = Integer.rotateRight(w[j-2], 17) ^
            Integer.rotateRight(w[j-2], 19) ^
            (w[j-2] >>> 10);
        w[j] = w[j-16] + s0 + w[j-7] + s1;
    }
```

```
    // Инициализация рабочих переменных
    int a = h[0], b = h[1], c = h[2], d = h[3],
        e = h[4], f = h[5], g = h[6], h7 = h[7];
```

```
    // Основной цикл
    for (int j = 0; j < 64; j++) {
```

```
        int s1 = Integer.rotateRight(e, 6) ^
            Integer.rotateRight(e, 11) ^
            Integer.rotateRight(e, 25);
        int ch = (e & f) ^ ((~e) & g);
        int temp1 = h7 + s1 + ch + K[j] + w[j];
        int s0 = Integer.rotateRight(a, 2) ^
            Integer.rotateRight(a, 13) ^
            Integer.rotateRight(a, 22);
        int maj = (a & b) ^ (a & c) ^ (b & c);
        int temp2 = s0 + maj;
```

```
        h7 = g;
        g = f;
        f = e;
        e = d + temp1;
        d = c;
        c = b;
        b = a;
        a = temp1 + temp2;
    }
```

```
// Добавляем результат блока
h[0] += a;
h[1] += b;
h[2] += c;
h[3] += d;
h[4] += e;
h[5] += f;
h[6] += g;
h[7] += h7;
}
```

```
// Формируем хеш
byte[] hash = new byte[32];
for (int i = 0; i < 8; i++) {

    hash[i * 4] = (byte) (h[i] >>> 24);
    hash[i * 4 + 1] = (byte) (h[i] >>> 16);
    hash[i * 4 + 2] = (byte) (h[i] >>> 8);
    hash[i * 4 + 3] = (byte) (h[i] >>> 0);
}
```

```

hash[i * 4 + 3] = (byte) h[i];
}

return hash;
}
}

```

FileUtils.java

Реализация работы с файлами для чтения/записи файлов.

```

package org.encrypting;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;

public class FileUtils {
    public static byte[] readFile(File file) throws IOException {
        return Files.readAllBytes(file.toPath());
    }

    // Чтение файла как текста в UTF-8 (только для ключей!)
    public static String readTextFile(File file) throws IOException {
        return Files.readString(file.toPath(), StandardCharsets.UTF_8);
    }

    public static void writeFile(File file, byte[] data) throws IOException {
        Files.write(file.toPath(), data);
    }
}

```

StudentInfo.java

Информация о студенте, зафиксированная в полях и достаётся в программной системе.

```

package org.encrypting;

public class StudentInfo {
    public static final String FIO = "Белянин Никита Николаевич";
    public static final String GROUP = "ПИбд-41";
    public static final String VARIANT = "6";
    public static final String TASK = "Реализация шифра Магма (ГОСТ Р 34.12-2015) в режиме гаммирования с обратной связью (CFB)";
}

```

Демонстрация работы программной системы

Приступим к демонстрации программ. После отладки и запуска программы, нам предоставляется справочная информация об авторе программы, вариант и описание алгоритма, далее по кнопке «Ок» можно перейти на выполнение программы. На экране с работой по шифрованию, есть 5 кнопок, 2 константных поля и 2 редактируемых поля. Для загрузки файла ключа, правая кнопка «загрузить ключ», либо вписать его в поле редактируемое с ключом. Написать любое слово текстовое, которое приведётся к нужному формату. Кнопки «Входной файл» и «Выходной файл» предназначены для получения данных и записи результата. Кнопка

«Зашифровать» шифрует текст, либо файл и не даёт его открыть, а расшифровать, при загруженном зашифрованном файле и правильным соответствующем ключе даст расшифрованный текст и файл. Все результаты хэширования будут представлены ниже в разделе результатов. Приступим к демонстрации экранов.

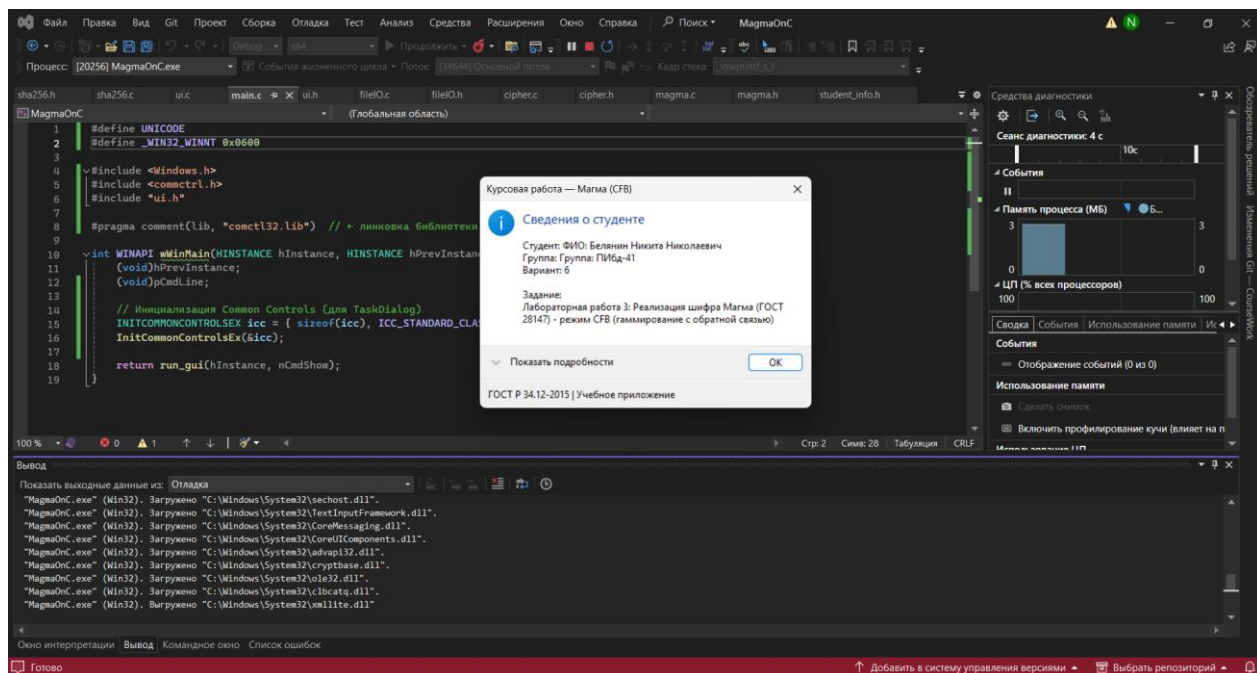


Рисунок 26. Запуск программы на С.

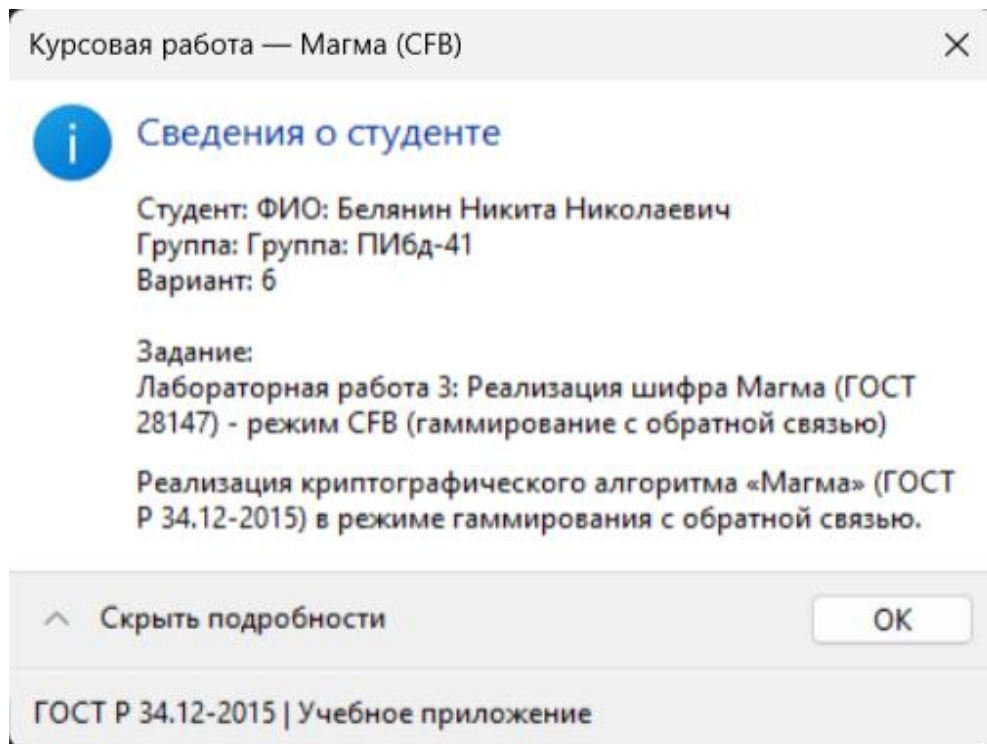


Рисунок 27. Справочная информация о студенте на С.

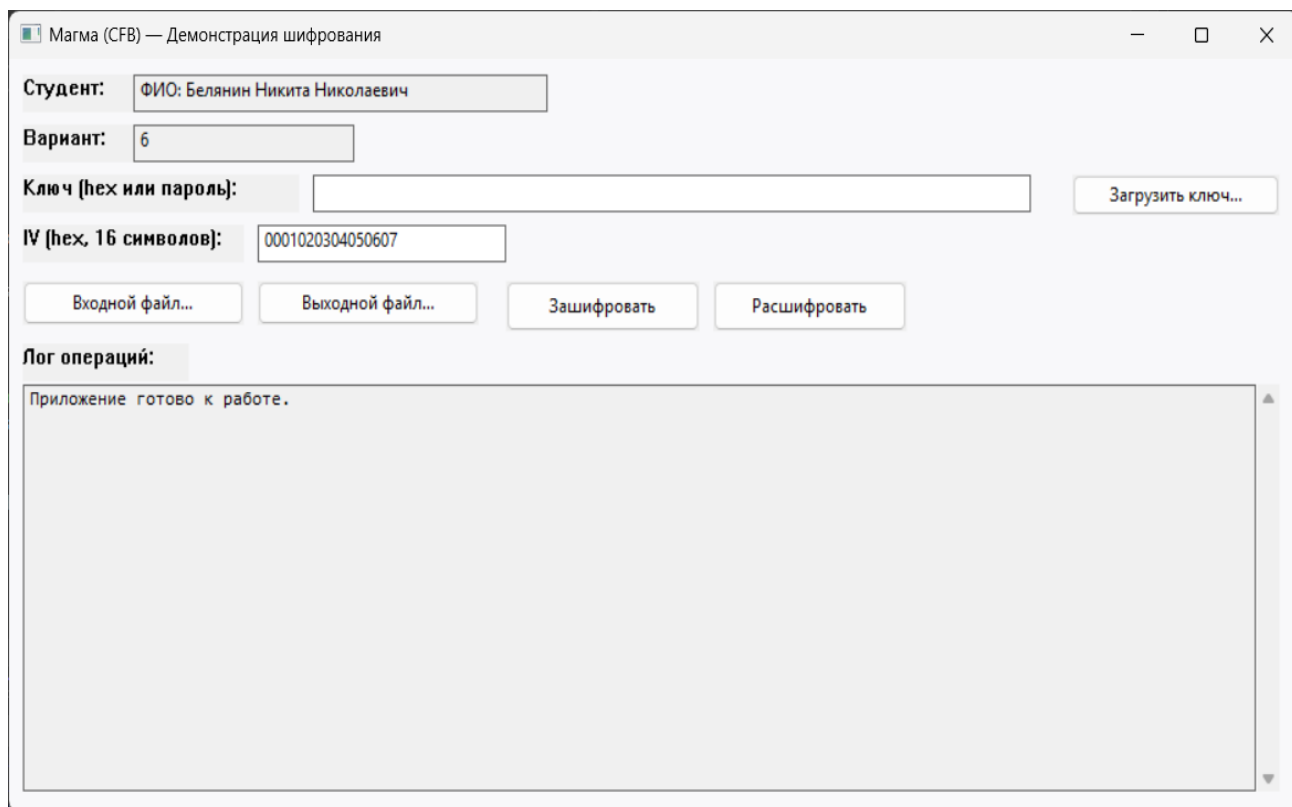


Рисунок 28. Главное окно работы с шифрованием на C.

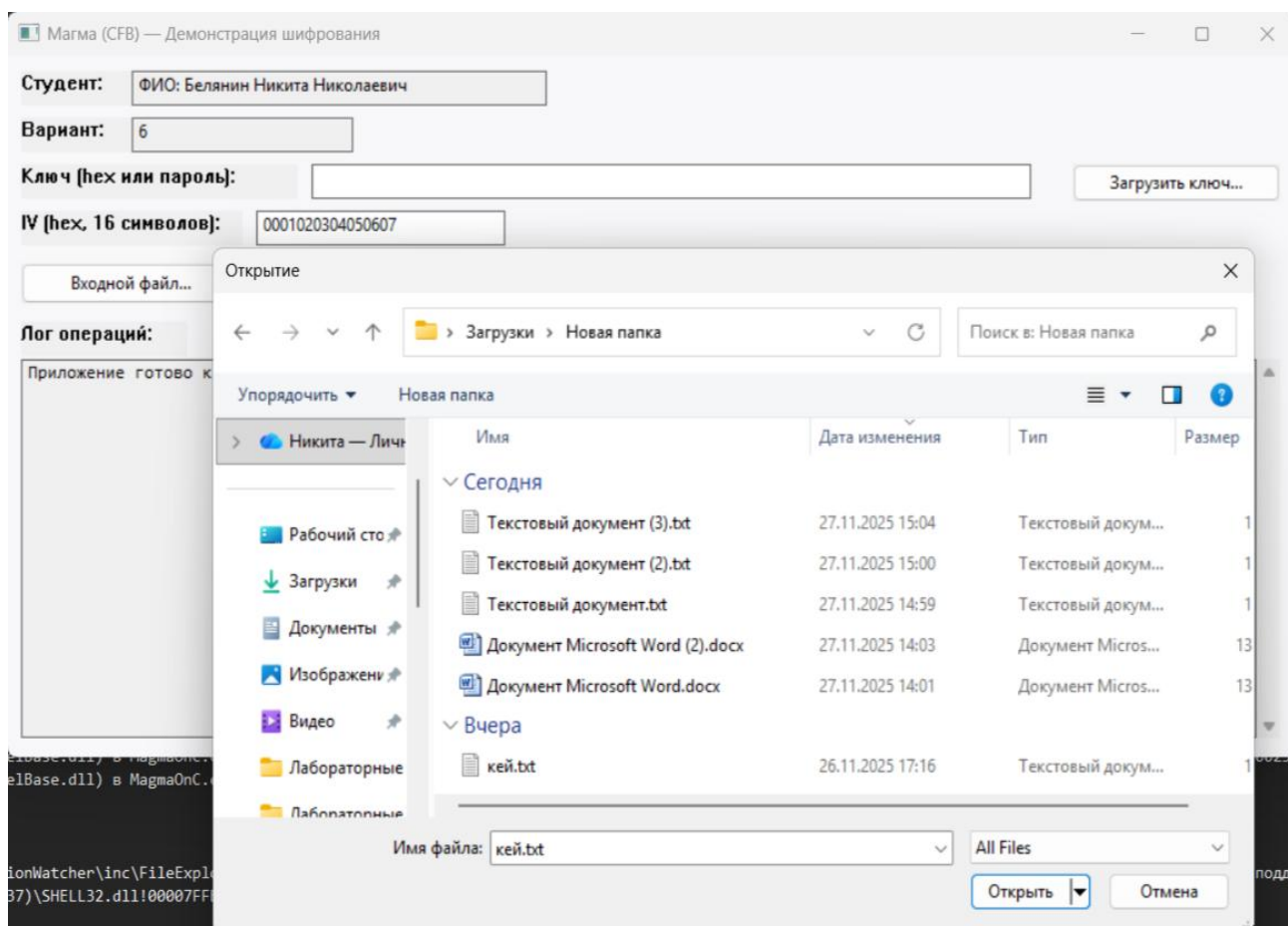


Рисунок 29. Окно загрузки ключа, входного, выходного файла.

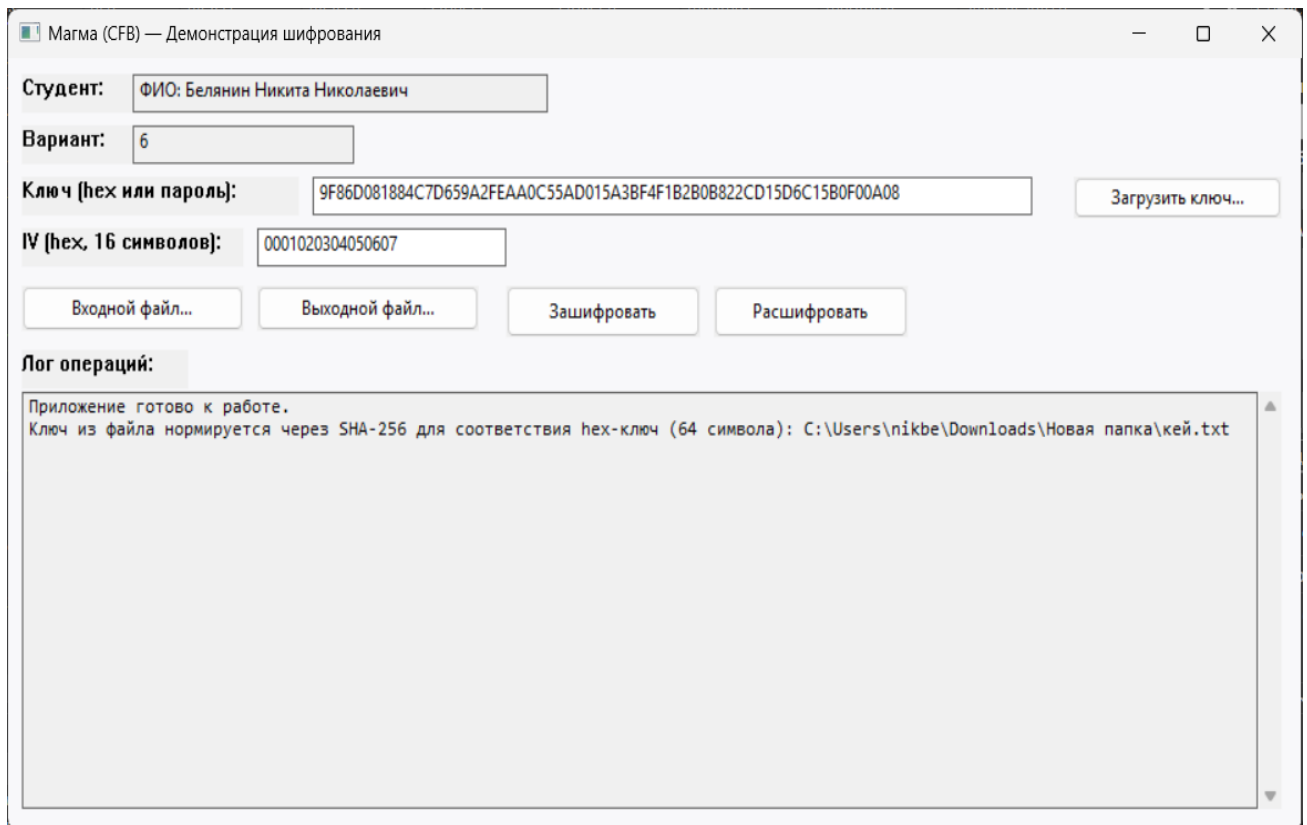


Рисунок 30. Результат в логах.

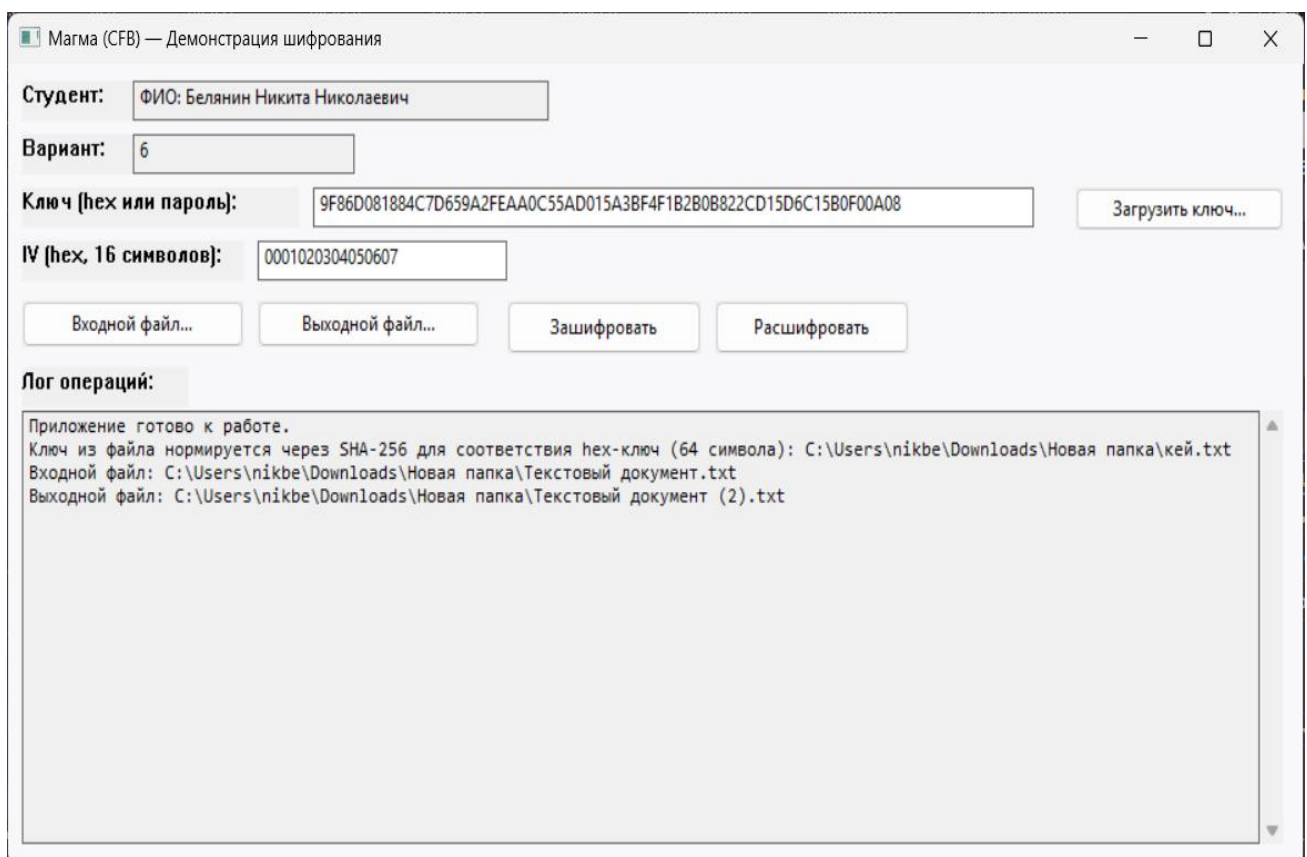


Рисунок 31. Результат в логах (добавления входного и выходного файлов).

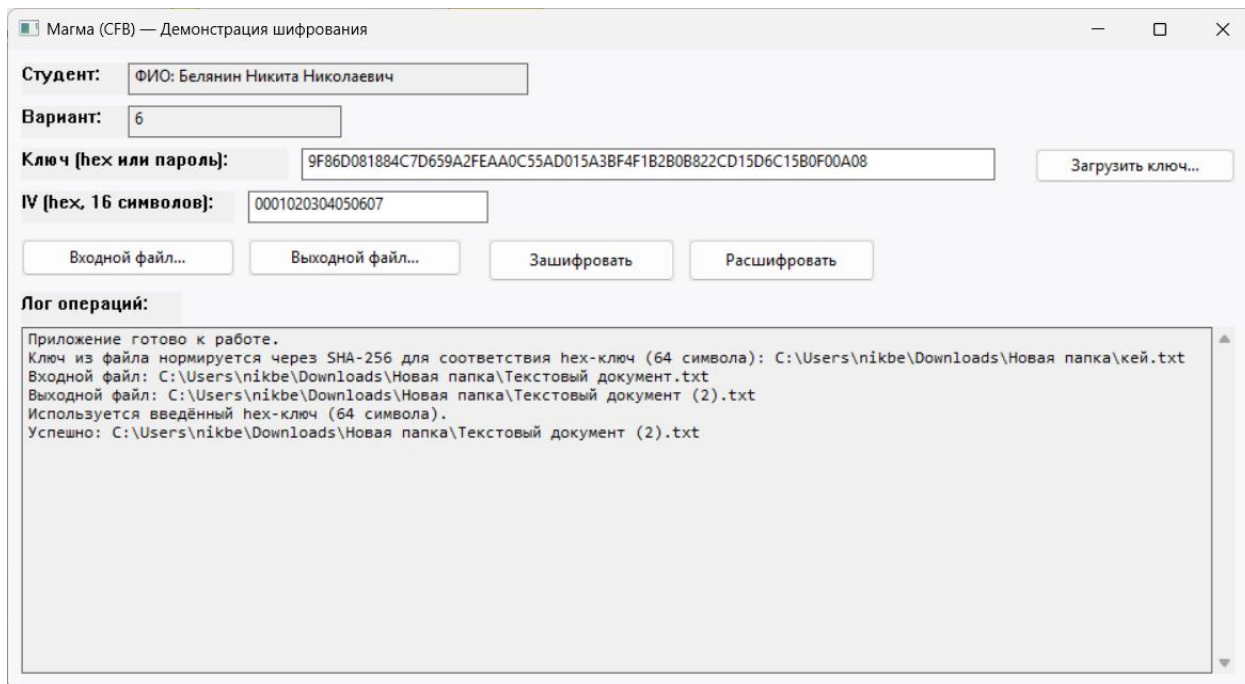


Рисунок 32. Результат в логах (шифрования файла).

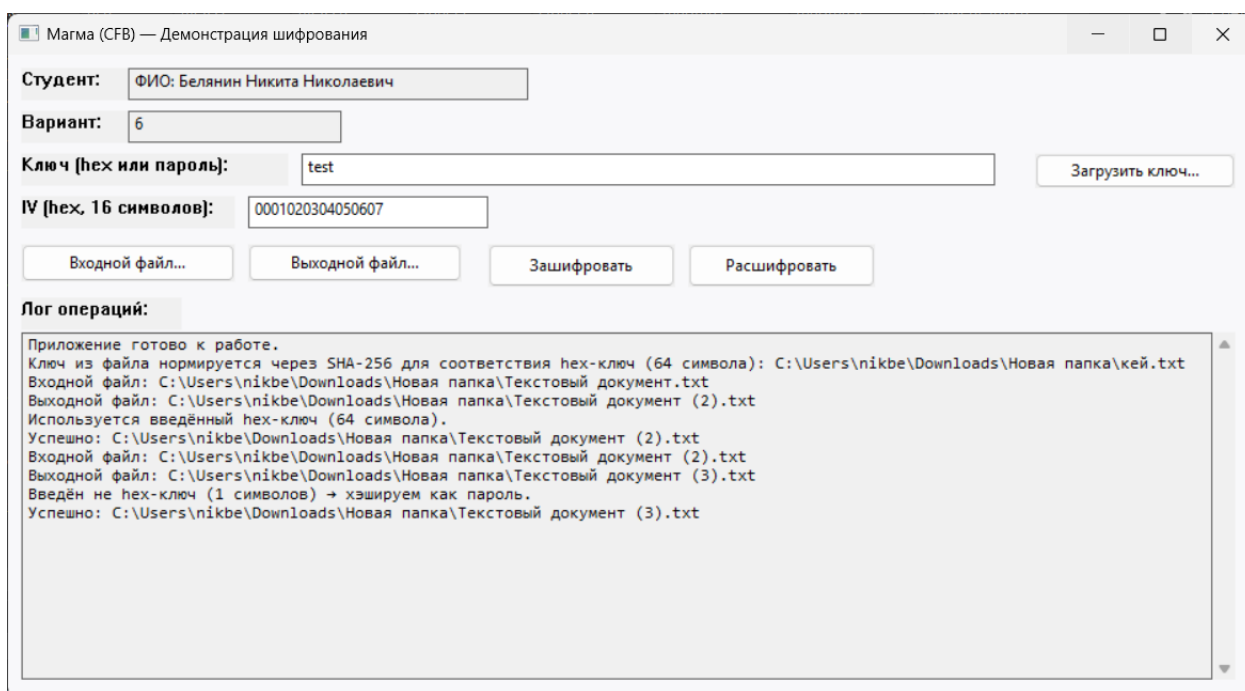


Рисунок 33. Результат в логах (расшифрования файла, ключ введён в строке).

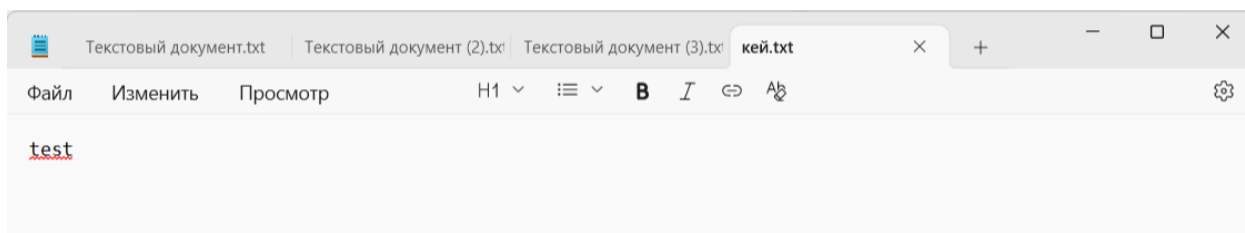


Рисунок 34. Файл с ключом.

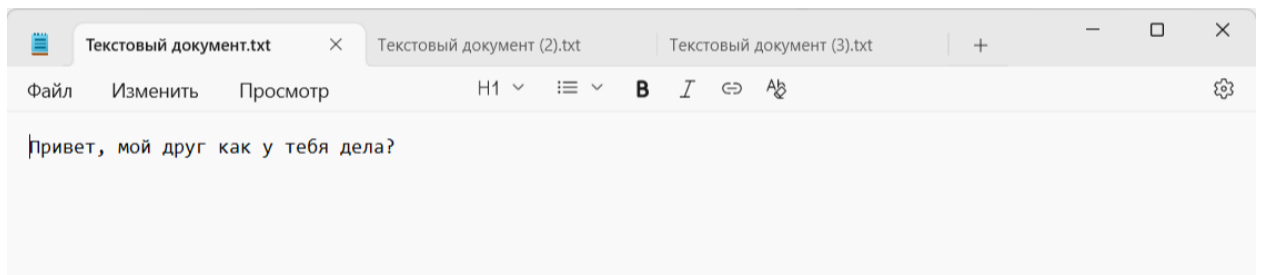


Рисунок 35. Файл с входным текстом для шифрования.

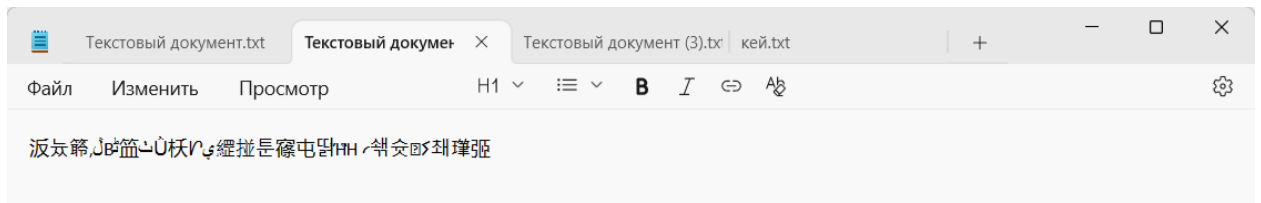


Рисунок 36. Файл с результатом шифрования (текст такой из-за кодировок).

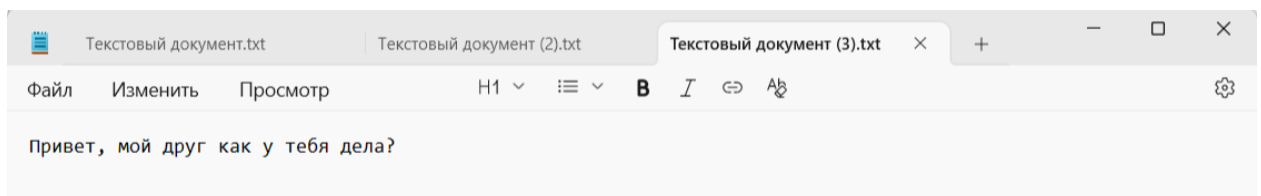


Рисунок 36. Файл с результатом расшифрования.

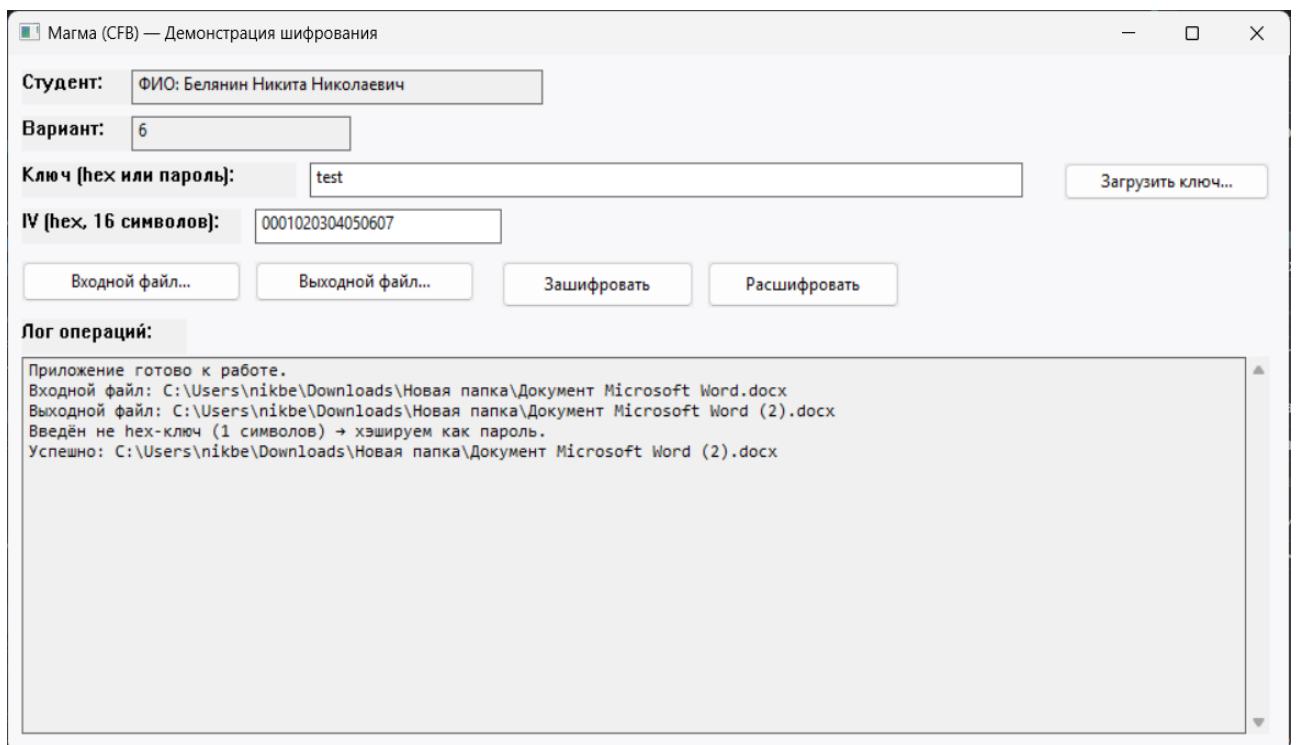


Рисунок 37. Шифрования файла Word.

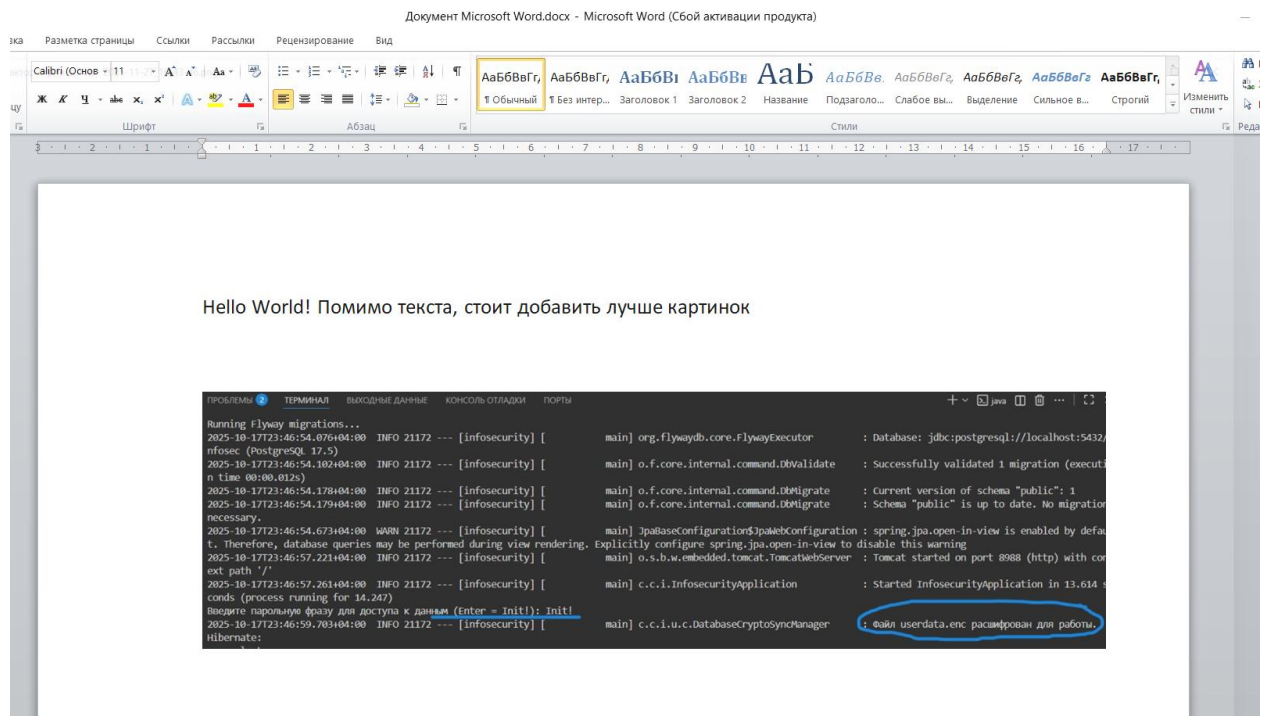


Рисунок 38. Исходный файл Word.

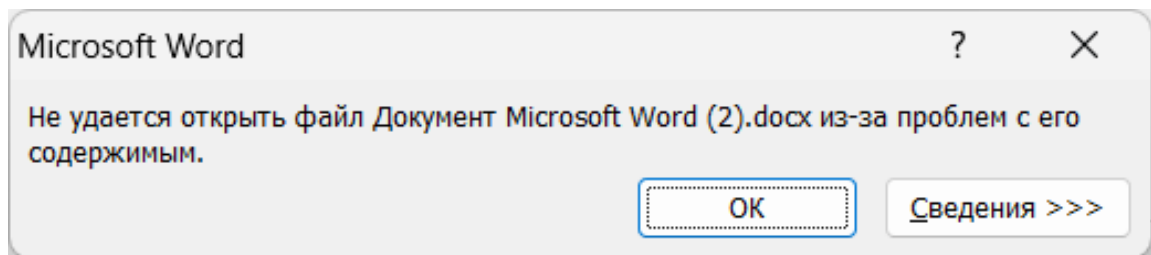


Рисунок 39. Результат после шифрования файла.

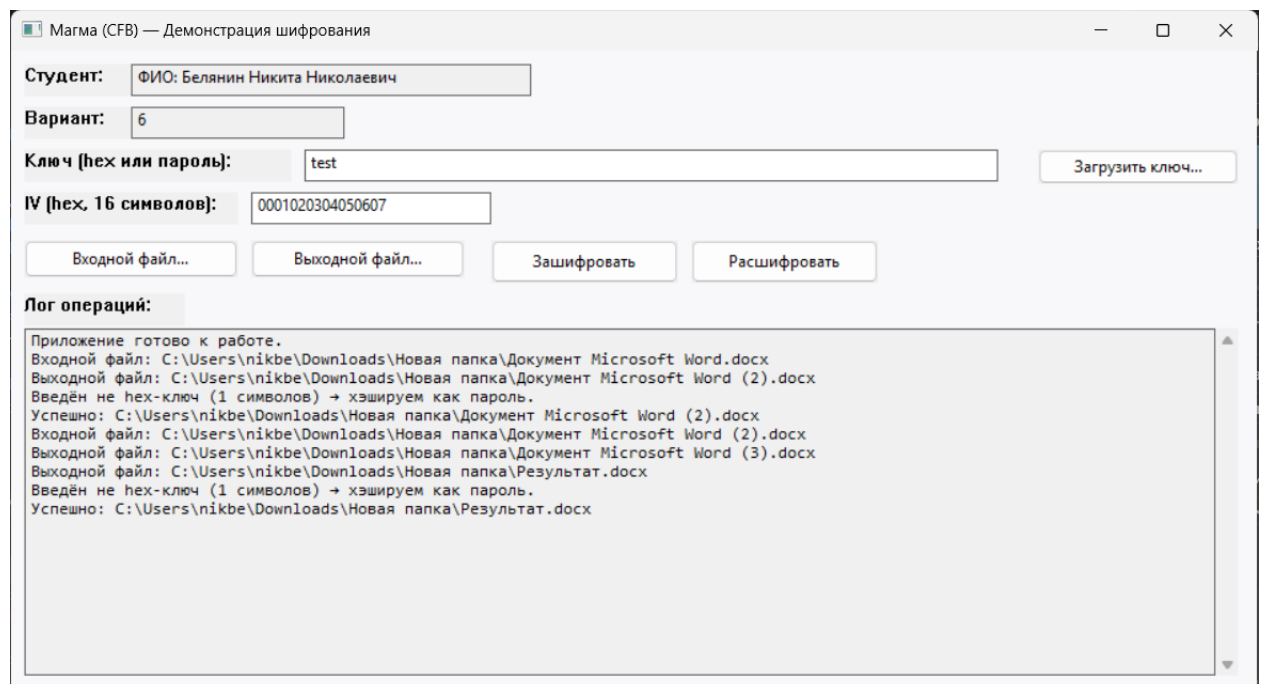


Рисунок 40. Результат после расшифрования файла.

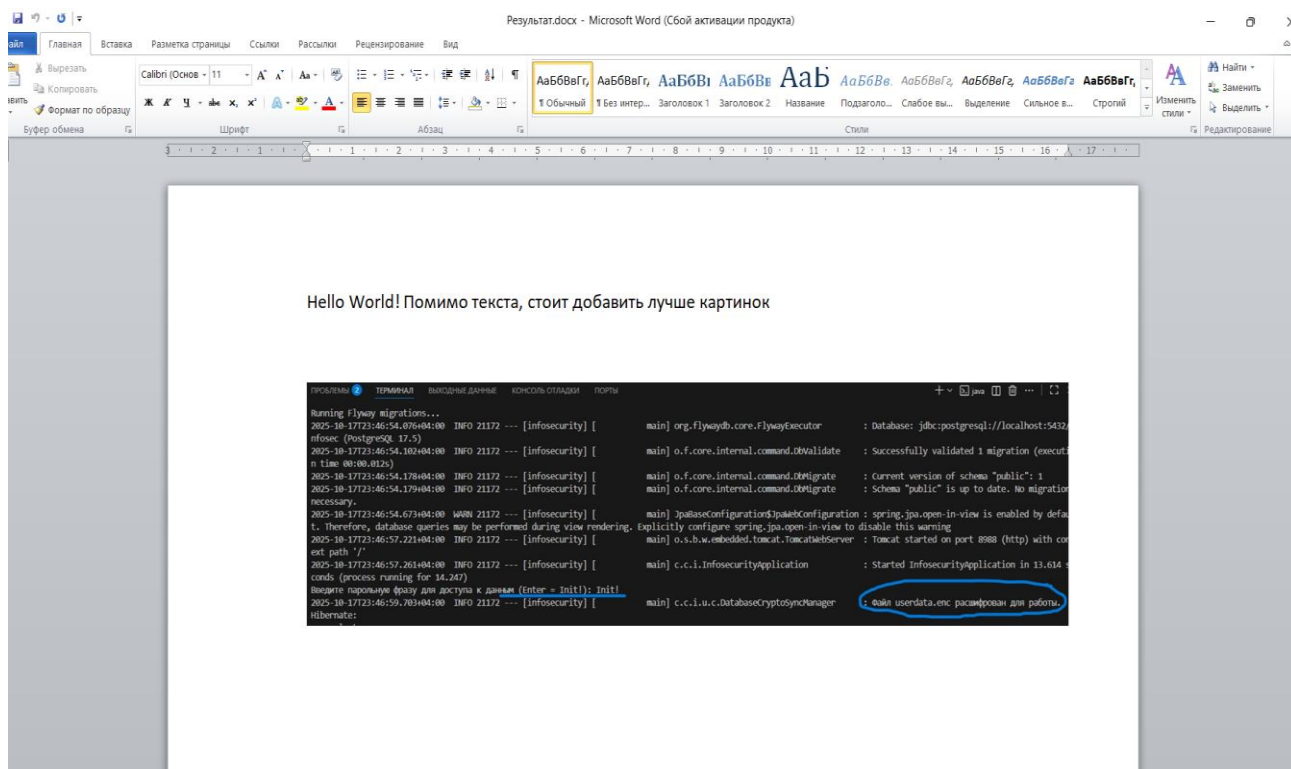


Рисунок 41. Результат после расшифрования файла.

Приступим к демонстрации программы на языке программирования Java. Функционал аналогичный и работает, так же как и для реализации на языке C. Аналогичным образом выводится информация в логах программы, загрузка и выгрузка файлов, а так же добавление ключа. Есть поля редактируемые. IV можно редактировать или использовать по умолчанию. При шифровании данных на кириллице были некоторые нарушения в кодировке, однако при латинице такого не происходит и всё работает штатно. В итоге в ходе шифрования было принято решение производить приведение к нужной кодировке.

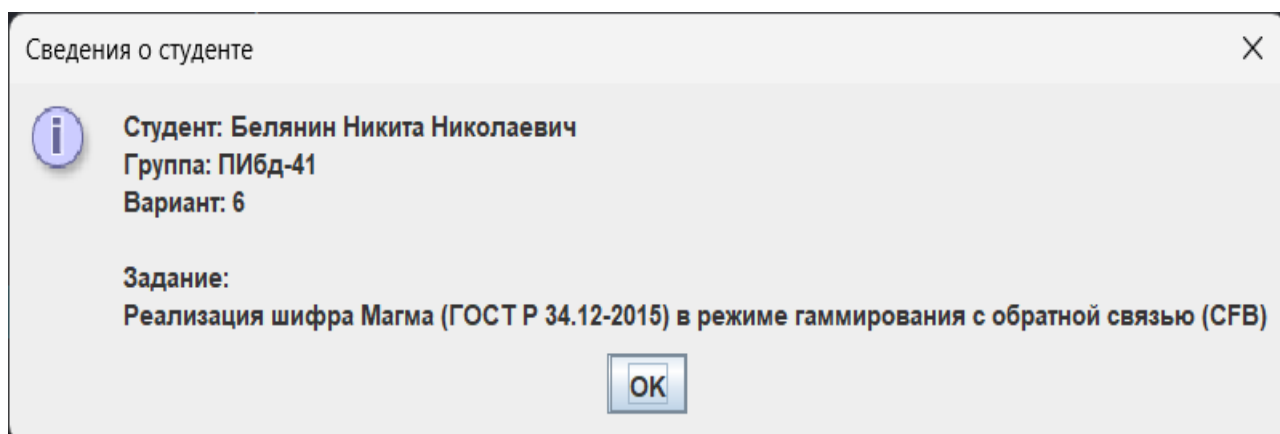


Рисунок 42. Справочная информация о студенте (Java).

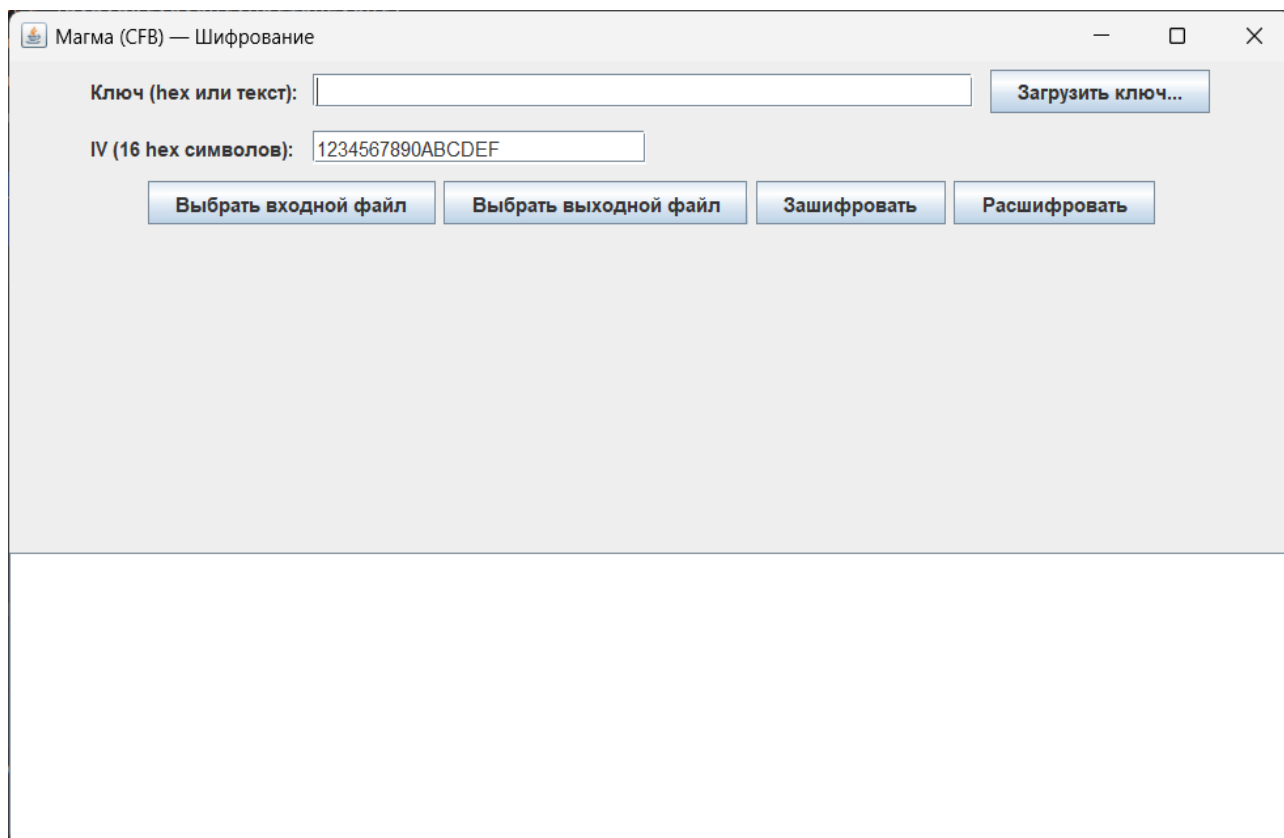


Рисунок 43. Главный экран работы (Java).

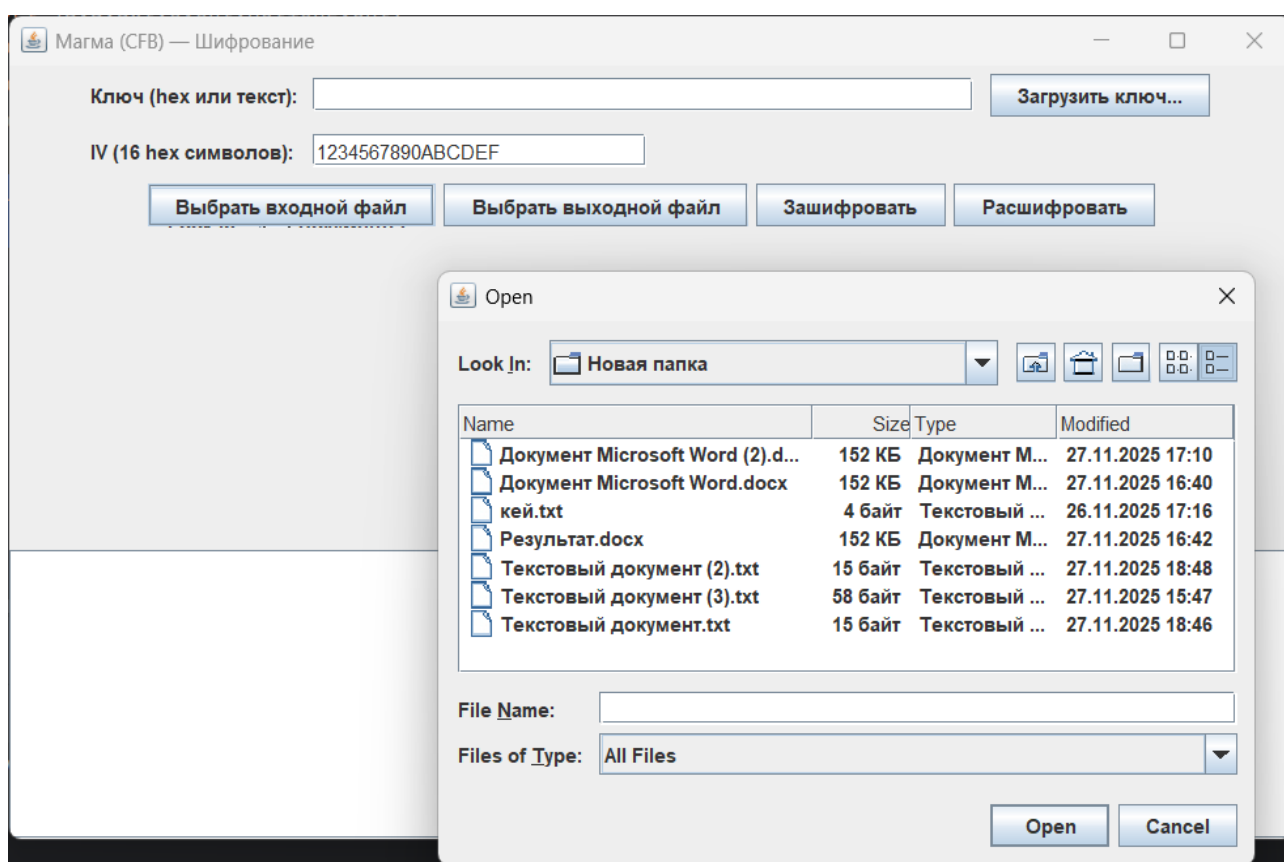


Рисунок 44. Добавление файлов для входных и выходных данных и ключа для шифрования (Java).

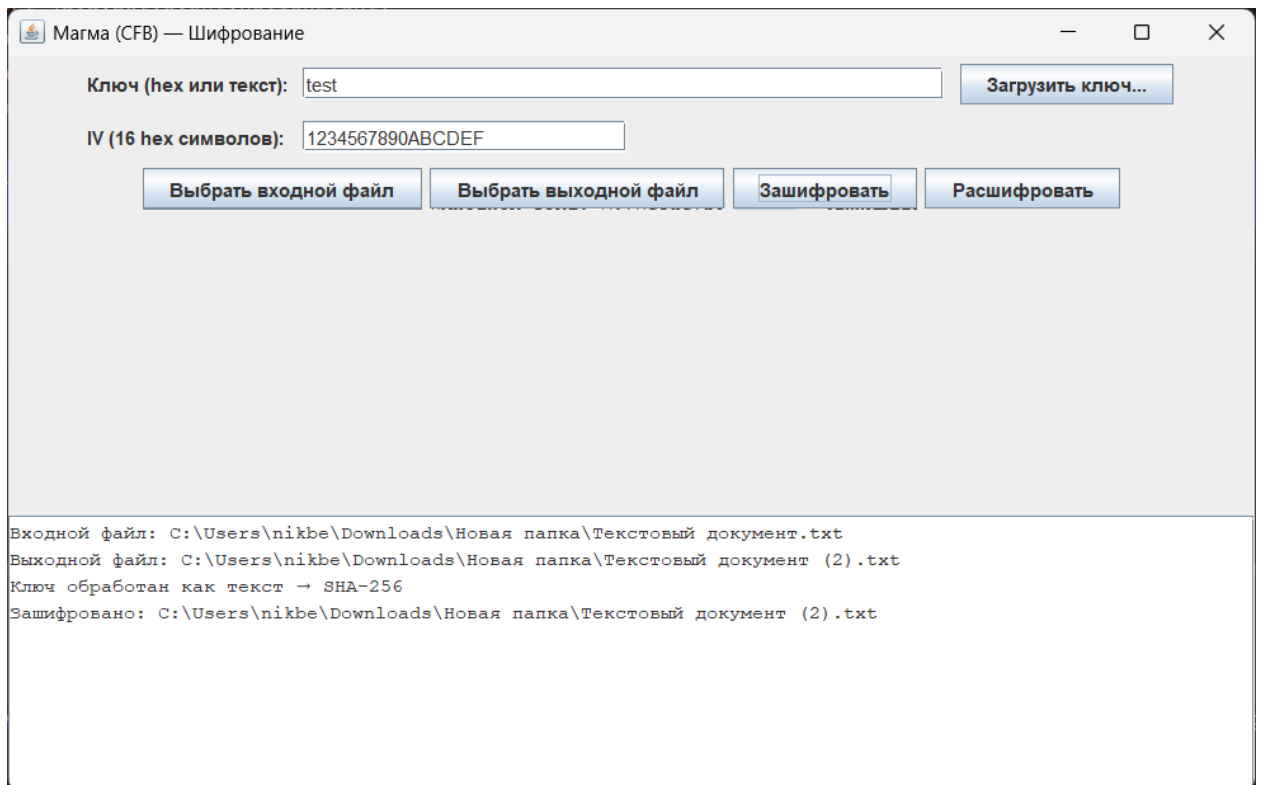


Рисунок 45. Логи работы программы (Java).

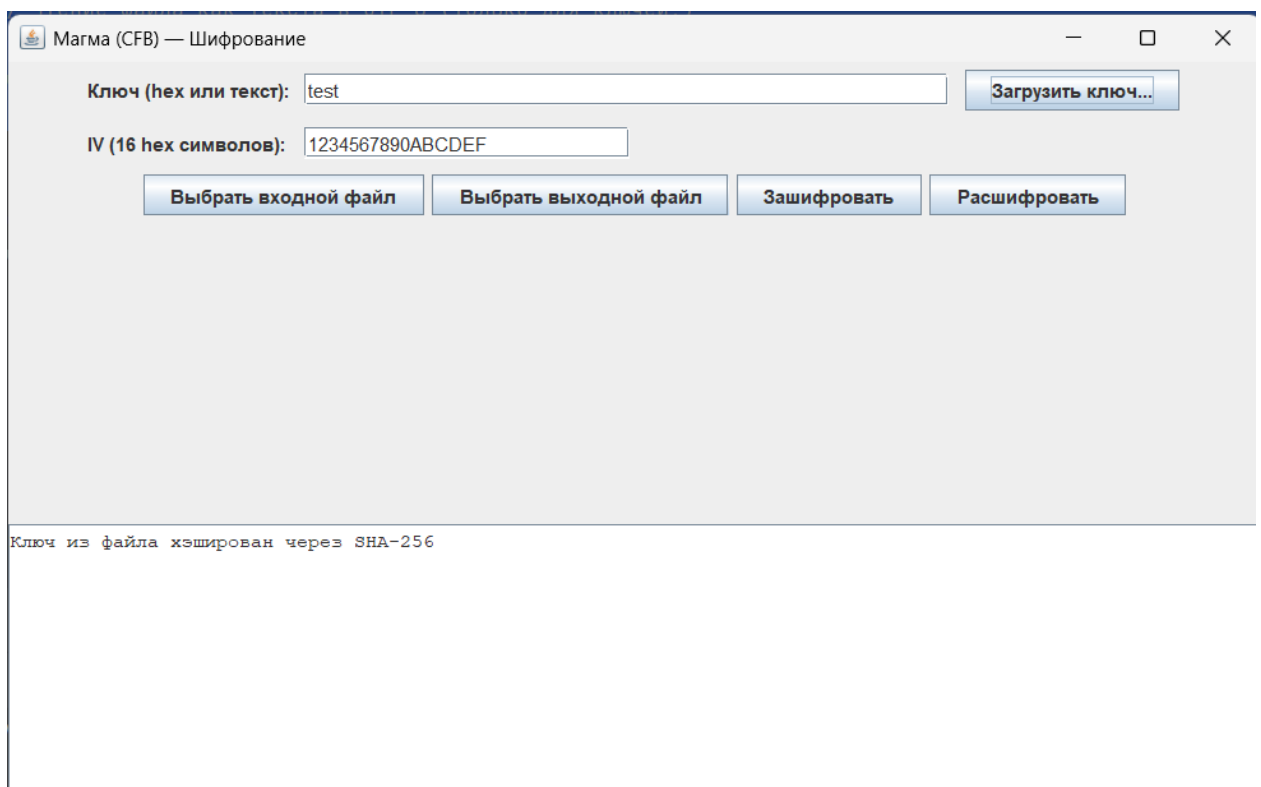


Рисунок 46. Добавление ключа из файла (Java).

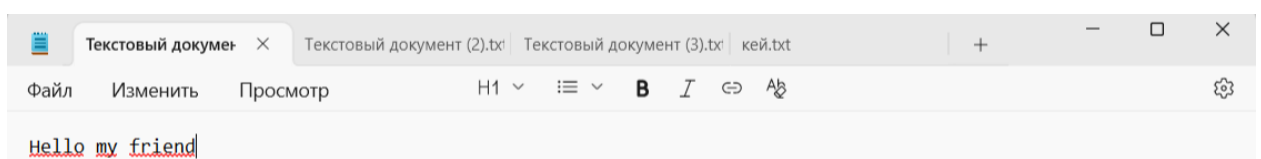


Рисунок 47. Информация входного файла программы (Java).

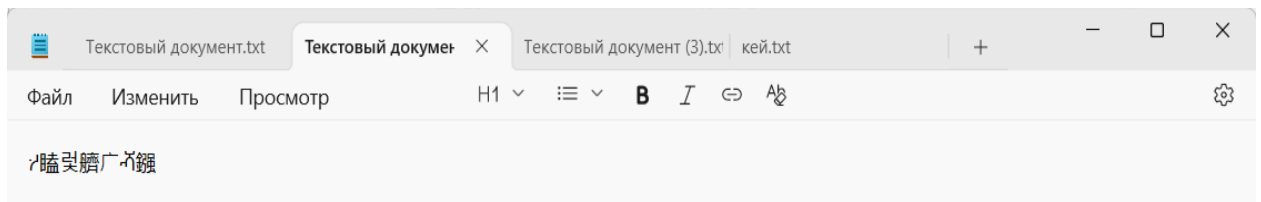


Рисунок 48. Результат шифрования входного файла программы (Java).

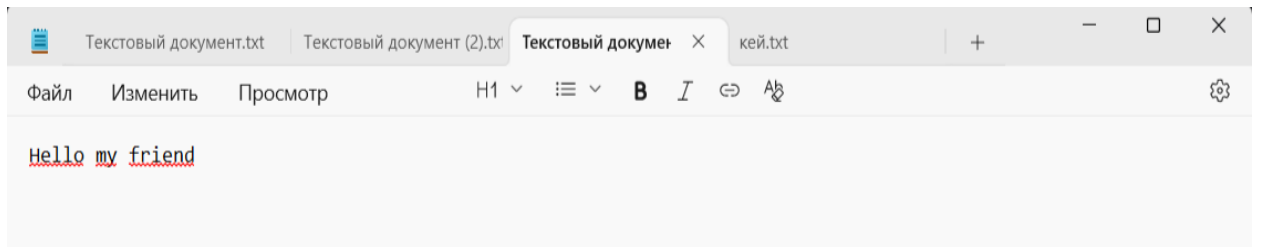


Рисунок 49. Результат расшифрования файла программы (Java).

Трассировка

Исходные данные и входные параметры

Блок (64 бита) – ASCII: *TestRnd!* = hex: 54 65 73 74 52 6E 64 21 (8 байт).

Разбиваем на две 32-битные половины (по 4 байта каждая):

N_1 (левая половина) = 0x54657374

N_2 (правая половина) = 0x526E6421

Ключ 256 бит сделаем из повторяющихся цифр:

$K_1 = 0x00112233$

$K_2 = 0x44556677$

$K_3 = 0x8899AABB$

$K_4 = 0xCCDDEEFF$

$K_5 = 0x00112233$

$K_6 = 0x44556677$

$K_7 = 0x8899AABB$

$K_8 = 0xCCDDEEFF$

Для одного раунда используется K_1 , применяем один раунд сети Фейстеля для Магма.

- **S-блок** (или блок подстановок, англ. s-box от substitution-box) — функция в коде программы или аппаратная система, принимающая на входе n бит, преобразующая их по определённому алгоритму и возвращающая на выходе m бит. n и m не обязательно равны.

Тогда для нашего S-box зададим простую функцию:

$$S(x) = (7 * x + 3) \bmod 16$$

Таблица $S(0..15) = [3, 10, 1, 8, 15, 6, 13, 4, 11, 2, 9, 0, 7, 14, 5, 12]$. Её я буду применять к 8-ми 4-битным нибблам¹¹ 32-битного слова. Сдвиг (ротация) после S-подстановки: левый циклический сдвиг на 11 бит (по стандарту Магма).

Алгоритм раунда

Для текущего раунда с ключом K :

1. $T = (N_1 + K) \bmod 2^{32}$ (сложение по модулю 2^{32})
2. Разбить 32-бит T на 8 нибблов (по 4 бита) — от старшего ниббла к младшему.
3. Каждый ниббл заменить через S-бокс
4. Собрать 32-бит из заменённых нибблов $S(T)$.
5. Ротировать $S(T)$ влево на 11 бит $R = \text{Rol}_{11}(S(T))$
6. Вычислить новое правое слово через XOR
7. Для раунда сети Фейстеля обычно происходит обмен половин: следующая пара будет.

Шаги с числами

$N_1 = 0x54657374 =$ (в двоичном)

0101 0100 0110 0101 0111 0011 0111 0100

$N_2 = 0x526E6421$

Ключ для этого раунда: $K = K_1 = 0x00112233$

¹¹ «Ниббл» (англ. nibble, nybble) — единица измерения информации, равная четырём двоичным разрядам (битам).

1. *Сложение по модулю 2^{32}*

$$T = (N_1 + K) \bmod 2^{32} = 0x54657374 + 0x00112233 = 0x547695A7$$

2. *Разбиение T на 8 нибблов*

Возьмём $T = 0x54\ 76\ 96\ A7$ как 4 байта (в ниббле 4 бита), от наиболее значимого к наименее значимому: [5, 4, 7, 6, 9, 5, 10, 7]

то есть $5 = 0x5$, $4 = 0x4$, $7 = 0x7$, $6 = 0x6$, $9 = 0x9$, $10 = 0xA$, $7 = 0x7$

3. *Применение S -бокса к каждому нибблу*

$S(x) = (7 \cdot x + 3) \bmod 16$, поэтому для наших нибблов получаем:

$$S(5) = (35 + 3) \bmod 16 = 38 \bmod 16 = 6$$

$$S(4) = (28 + 3) = 31 \bmod 16 = 15$$

$$S(7) = (49 + 3) = 52 \bmod 16 = 4$$

$$S(6) = (42 + 3) = 45 \bmod 16 = 13$$

$$S(9) = (63 + 3) = 66 \bmod 16 = 2$$

$$S(5) = 6$$

$$S(10) = (70 + 3) = 73 \bmod 16 = 9$$

$$S(7) = 4$$

Итого: [6, 15, 4, 13, 2, 6, 9, 4]

4. *Сборка 32-битного слова после S -замены*

Собираем нибблы (переводим в HEX): $0x6F4D2694$

5. *Левый циклический сдвиг на 11 бит*

$$R = \text{ROL}_{11}(0x6F4D2694) = 0x6934A37A$$

$$6\ F\ 4\ D\ 2\ 6\ 9\ 4 \rightarrow 0110\ 1111\ 0100\ 1101\ 0010\ 0110\ 1001\ 0100$$

$$01101111\ 01001101\ 00100110\ 10010100 =$$

сдвиг влево на 11 = 01101111010011010010011010010100000000000000
(добавилось 11 нулей)

Обрезаем, оставляя только 32 бита:

01101111010|||011010010011010010100||| 000000000000 (отсекаем и на место них переносим старшие биты)

R = 01100110010011010010100000000 | **01101111010**

0110 1001 0011 0100 1010 0000 0011 01111 1010

6	9	3	4	A	0	3	7	A
---	---	---	---	---	---	---	---	---

6. XOR с правой половиной N_2

R = 0x6934A37A XOR 0x526E6421 = 0x3B5AC75B

7. Далее после раунда идёт обмен половинами. После завершения раунда мы обычно делаем обмен половин.

Следующая левая половина $N_2 = 0x526E6421$, следующая правая половина N = 0x3B5AC75B

Таким образом, выглядит трассировка, S-бокс был взят как пример, не из ГОСТ использовался порядок операций и все преобразования. Единственное, что изменится при подстановке реального S-бокса — будут другие значения на шаге 3 и дальше.

Уязвимости режима гаммирования ОС Магмы

1. *Повтор гаммы при повторном использовании (Ключ, IV).* Если зашифровать 2 разных сообщения с одинаковым ключом и синхропосылкой (IV), то гамма будет одинаково. Последствие: злоумышленник получает XOR открытых текстов, что часто позволяет восстановить содержимое (особенно при известной структуре JSON, XML). Требование ГОСТ: синхропосылка должна быть уникальной для каждой сессии шифрования данным ключом. Нельзя допускать, чтобы IV генерировалась предсказуемо (счётчик без сохранения состояния между перезапусками) или фиксированный IV.

2. *Отсутствие MAC.* Режим гаммирования не обеспечивает целостность и подлинность. Злоумышленник может изменить биты шифротекста, вызвать предсказуемые изменения в открытом тексте, удалить блоки, если известна структура (подменить role) или выполнить bit-flipping attack¹². Рекомендация: использовать режим сцепления шифрования с имитовставкой (CMAC).

3. *Слабая энтропия¹³ ключа или IV.* Если ключ или IV генерируются из недостаточно случайных источников, то возможен brute-force или dictionary-attack на ключ, возможен предсказуемый IV (повтор гаммы)

Верификация и борьба с уязвимостями

1. **Криптографическая верификация реализации.** Проверка на соответствие осуществляется путём стандартизированных тест-векторов, которые являются официальными тестовыми примерами из стандарт. Проверка на всех этапах (выработка гаммы, шифрование, длина блоков) и проверка длин ключа, блока и IV.

2. **Безопасная генерация IV.** IV должна быть криптографически случайной, нельзя использовать часть ключа, фиксированные значения, предсказуемые последовательности без сохранения состояния. Рекомендация: логировать пары и сверять, чтобы они не повторялись.

3. **Защита от повторного использования ключа.** Использовать ключ только для ограниченного объёма данных.

4. **Добавление MAC.** Реализовать имитовставку, применять режим шифрования с имитовставкой, отдельно вычислять хэш по другому алгоритму. Верифицировать MAC до расшифровки (чтобы избежать атак на парсинг).

5. **Аудит кода и side-channel¹⁴ защита.** Убедиться, что реализация постоянного времени (constant-time), нет ветвления, доступов

¹² **Bit flipping attack** — это изменение значения бита (двоичной цифры) от 0 до 1 или наоборот.

¹³ **Энтропия** — мера непредсказуемости в битах

по секретным данным, избегать утечек через кэш-тайминги, ошибки при неправильной длине входа.

Достоинства стандарта

1. Бесперспективность атаки полным перебором (XSL-атаки¹⁵ в учёт не берутся, так как их эффективность на данный момент полностью не доказана).

2. Эффективность реализации и высокое быстродействие на современных компьютерах. (В действительности программные реализации ГОСТ 28147-89 как любой шифр сети Фейстеля медленнее современных шифров типа AES и других).

3. Наличие защиты от навязывания ложных данных (выработка имитовставки) и одинаковый цикл шифрования во всех четырёх алгоритмах стандарта.

Критика стандарта

Основные проблемы стандарта связаны с неполнотой стандарта в части генерации ключей и таблиц замен. Считается, что у стандарта существуют «слабые» ключи и таблицы замены, но в стандарте не описываются критерии выбора и отсева «слабых».

В октябре 2010 года на заседании 1-го объединённого технического комитета Международной организации по стандартизации (ISO/IEC JTC 1/SC 27) ГОСТ был выдвинут на включение в международный стандарт блочного шифрования ISO/IEC. В связи с этим в январе 2011 года были сформированы фиксированные наборы узлов замены и проанализированы их криптографические свойства. Однако ГОСТ не был принят в качестве стандарта, и соответствующие таблицы замен не были опубликованы.

¹⁴ **side-channel** (атака по сторонним или побочным каналам) — класс атак, направленных на уязвимости в практической реализации криптосистемы.

¹⁵ **XSL-атака** (англ. **eXtended Sparse Linearization**, алгебраическая атака) — это метод криптографического анализа, основанный на алгебраических свойствах шифра. Метод предполагает решение особой системы уравнений.

Таким образом, существующий стандарт не специфицирует алгоритм генерации таблицы замен (S-блоков). С одной стороны, это может являться дополнительной секретной информацией (помимо ключа), а с другой, поднимает ряд проблем:

- Нельзя определить криптостойкость алгоритма, не зная заранее таблицы замен.
- Реализации алгоритма от различных производителей могут использовать разные таблицы замен и могут быть несовместимы между собой.
- Возможность преднамеренного предоставления слабых таблиц замен лицензирующими органами РФ.
- Потенциальная возможность (отсутствие запрета в стандарте) использования таблиц замены, в которых узлы не являются перестановками, что может привести к чрезвычайному снижению стойкости шифра.

Заключение

В ходе выполнения лабораторной работы была проведена всесторонняя и тщательная работа по изучению криптографических алгоритмов симметричного шифрования. Особо подробно был изучен отечественный блочный шифр "Магма" ГОСТ Р 34.12-2018 (ГОСТ 28147). Данный государственный стандарт Российской Федерации играет ключевую роль в обеспечении информационной безопасности и широко применяется в системах защиты конфиденциальной и персональных данных, а также в критически важных инфраструктурах. Его использование регламентируется нормативными актами и рекомендовано для внедрения в государственных и коммерческих организациях, где требуется соответствие требованиям российского законодательства в области криптографической защиты информации.

Перед реализацией алгоритма была изучена теоретическая часть работы с шифрованием для понимания всей картины и плана работ.

Произведён разбор схем и зарисованы блок-схемы для лучшего понимания алгоритма.

Была детально разобрана теоретическая часть, включающая структуру алгоритма, принципы построения на основе сети Фейстеля, а также ключевые этапы раундового преобразования — в особенности роль S-boxes как основного нелинейного компонента, обеспечивающего устойчивость к дифференциальному и линейному криптоанализу. Также был рассмотрен процесс генерации раундовых ключей, побитовые операции сложения по модулю, применение таблиц замены и циклические сдвиги результата функции.

На практической части работы алгоритм шифрования «Магма» был полностью реализован программно, что позволило закрепить понимание всех этапов обработки данных, проверить корректность работы. В результате проделанной работы достигнуто глубокое понимание принципов построения современных блочных шифров, а также приобретены навыки реализации криптографических алгоритмов с соблюдением строгих технических спецификаций.

Таким образом, в результате выполнения лабораторной работы был достигнут глубокий уровень понимания технологий шифрования и архитектуры отечественного криптографического алгоритма Магма, приобретены навыки работы с нормативными документами по применению алгоритма шифрования ГОСТ 28147 и их практического применения, а также закреплены компетенции в области разработки и анализа криптографических систем. Выполнение данной работы способствует формированию профессиональной подготовки в области информационной безопасности и подчеркивает значимость использования стандартизованных и сертифицированных криптографических решений.

Приложение. Листинг кода

Реализация на C

main.c

```
#define UNICODE
#define _WIN32_WINNT 0x0600

#include <Windows.h>
#include <commctrl.h>
#include "ui.h"

#pragma comment(lib, "comctl32.lib") // ← линковка
библиотеки

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE
hPrevInstance, PWSTR pCmdLine, int nCmdShow) {
    (void)hPrevInstance;
    (void)pCmdLine;

    // Инициализация Common Controls (для TaskDialog)
    INITCOMMONCONTROLSEX icc = { sizeof(icc),
ICC_STANDARD_CLASSES };
    InitCommonControlsEx(&icc);

    return run_gui(hInstance, nCmdShow);
}
```

cipher.c

```
#include "cipher.h"
#include <string.h>
```

```
// == Инициализация == //
```

```
void magma_cfb_initialization(magma_cfb_t *ctx, const uint8_t
key_raw[32], const uint8_t iv[8]) {
    MagmaSetKey(&ctx->key, key_raw);
    memcpy(ctx->reg, iv, MAGMA_BLOCK_SIZE);
}
```

```
/* Шифрование Gamma = E(R) обрабатываем по-байтово,
регистр перемещается на 1 байт и добавляется новый
шифротекст. */
void magma_cfb_encrypt_stream(magma_cfb_t* ctx, const
uint8_t* in, uint8_t* out, size_t len) {
    uint8_t gamma[MAGMA_BLOCK_SIZE];

    for (size_t i = 0; i < len; ++i) {
        MagmaEncryptBlock(&ctx->key, ctx->reg,
gamma);

        out[i] = in[i] ^ gamma[0];

        // сдвиг влево на один байт
        memmove(ctx->reg, ctx->reg + 1,
MAGMA_BLOCK_SIZE - 1);
        ctx->reg[MAGMA_BLOCK_SIZE - 1] =
out[i];
    }
}
```

```
/* Расшифрование Gamma = E(R) отличие в том, что при
обновлении регистра добавляем входной C(i)*/
void magma_cfb_decrypt_stream(magma_cfb_t* ctx, const
uint8_t* in, uint8_t* out, size_t len) {
    uint8_t gamma[MAGMA_BLOCK_SIZE];

    for (size_t i = 0; i < len; ++i) {
        MagmaEncryptBlock(&ctx->key, ctx->reg,
gamma);

        out[i] = in[i] ^ gamma[0];
        memmove(ctx->reg, ctx->reg + 1,
MAGMA_BLOCK_SIZE - 1);
    }
}
```

```
// используем байт зашифрованного
текста
ctx->reg[MAGMA_BLOCK_SIZE - 1] =
in[i];
}
```

fileIO.c

```
#include "fileIO.h"
#include <stdio.h>
#include <stdlib.h>
```

```
uint8_t* read_file(const char *path, size_t *outlen) {
    FILE* f = fopen(path, "rb");

    if (!f) return NULL;

    if (fseek(f, 0, SEEK_END) != 0) {
        fclose(f); return NULL;
    }

    long sz = ftell(f);

    if (sz < 0) {
        fclose(f); return NULL;
    }
}
```

```
rewind(f);
uint8_t* buf = (uint8_t*)malloc((size_t)sz + 1);

if (!buf) {
    fclose(f); return NULL;
}
```

```
size_t r = fread(buf, 1, (size_t)sz, f);
fclose(f);

if (r != (size_t)sz) {
    free(buf); return NULL;
}
```

```
*outlen = (size_t)sz;
return buf;
}
```

```
uint8_t* read_file_w(const wchar_t* path, size_t* outlen) {
    if (!path || !outlen) return NULL;
    FILE* f = _wfopen(path, L"rb");
    if (!f) return NULL;
```

```
if (_fseeki64(f, 0, SEEK_END) != 0) {
    fclose(f);
    return NULL;
}
```

```
__int64 sz = _ftelli64(f);
if (sz < 0 || sz > SIZE_MAX) {
    fclose(f);
    return NULL;
}
```

```
_fseeki64(f, 0, SEEK_SET);
```

```
uint8_t* buf = (uint8_t*)malloc((size_t)sz + 1);
if (!buf) {
    fclose(f);
    return NULL;
}
```

```
size_t r = fread(buf, 1, (size_t)sz, f);
fclose(f);
```

```
if (r != (size_t)sz) {
    free(buf);
    return NULL;
}
```

```

        *outlen = (size_t)sz;
        return buf;
    }

int write_file(const char* path, const uint8_t* buf, size_t len) {
    FILE* f = fopen(path, "wb");
    if (!f) return 0;
    size_t w = fwrite(buf, 1, len, f);
    fclose(f);
    return w == len;
}

// Тип callback остаётся прежним
typedef void (*process_block_cb)(void* ctx, const uint8_t* in,
uint8_t* out, size_t n);

int process_file_stream_w(const wchar_t* inpath, const wchar_t*
outpath, void* ctx_cb, process_block_cb cb) {
    FILE* fin = _wopen(inpath, L"rb");
    if (!fin) return 0;

    FILE* fout = _wopen(outpath, L"wb");
    if (!fout) {
        fclose(fin);
        return 0;
    }

    const size_t BUF = 4096;
    uint8_t* inbuf = (uint8_t*)malloc(BUF);
    uint8_t* outbuf = (uint8_t*)malloc(BUF);
    if (!inbuf || !outbuf) {
        fclose(fin); fclose(fout);
        free(inbuf); free(outbuf);
        return 0;
    }

    size_t n;
    int ok = 1;
    while ((n = fread(inbuf, 1, BUF, fin)) > 0) {
        cb(ctx_cb, inbuf, outbuf, n);
        if (fwrite(outbuf, 1, n, fout) != n) {
            ok = 0;
            break;
        }
    }

    fclose(fin);
    fclose(fout);
    free(inbuf);
    free(outbuf);
    return ok;
}

```

kdf.c

```

#include "kdf.h"
#include "sha256.h"

```

```

void kdf_sha256(const uint8_t* input, size_t input_len, uint8_t
out[32]) {
    sha256_hash(input, input_len, out);
}

```

magma.c

```

#include "magma.h"
#include <string.h>

```

```

// Реализация стандарта Магма (реализованы корректная
упаковка/распаковка, S-блоки и 32 раунда сети Фейстеля
// Здесь используется представление: входной блок 64-бита
разбивается на N1 и N2

```

```

static const uint8_t Sbox[8][16] = {

{0xF,0xC,0x2,0xA,0x6,0x4,0x5,0x0,0x7,0x9,0xE,0xD,0x1,0xB,0x
8,0x3},

```

```

{0xB,0x6,0x3,0x4,0xC,0xF,0xE,0x2,0x7,0xD,0x8,0x0,0x5,0xA,0x
9,0x1},

```

```

{0x1,0xC,0xB,0x0,0xF,0xE,0x6,0x5,0xA,0xD,0x4,0x8,0x9,0x3,0x
7,0x2},

```

```

{0x1,0x5,0xE,0xC,0xA,0x7,0x0,0xD,0x6,0x2,0xB,0x4,0x9,0x3,0x
F,0x8},

```

```

{0x0,0xC,0x8,0x9,0xD,0x2,0xA,0xB,0x7,0x3,0x6,0x5,0x4,0xE,0x
F,0x1},

```

```

{0x8,0x0,0xF,0x3,0x2,0x5,0xE,0xB,0x1,0xA,0x4,0x7,0xC,0x9,0x
D,0x6},

```

```

{0x3,0x0,0xF,0x1,0xE,0x9,0x2,0xD,0x8,0xC,0x4,0xB,0xA,0x
5,0x7},

```

```

{0x1,0xA,0x6,0x8,0xF,0xB,0x0,0x4,0xC,0x3,0x5,0x9,0x7,0xD,0x
2,0xE}
};

```

```

// Циклический поворот 32 бит
static inline uint32_t rol32(uint32_t x, unsigned n) {
    return (x << n) | (x >> (32 - n));
}

```

```

// S-образная замена 32-разрядного слова ( как 8 фрагментов)
static uint32_t substitute32(uint32_t x) {
    // блоки
    uint8_t nibbles[8];

```

```

    for (int i = 0; i < 8; ++i) {
        nibbles[i] = (x >> (28 - 4 * i)) & 0x0F;
    }

```

```

    for (int i = 0; i < 8; ++i) {
        nibbles[i] = Sbox[i][nibbles[i]];
    }

```

```

    uint32_t y = 0;

```

```

    for (int i = 0; i < 8; ++i) {
        y |= ((uint32_t)nibbles[i] << (28 - 4 * i));
    }

```

```

    return y;
}

```

```

// === F-функция Магмы === //
static uint32_t F(uint32_t x, uint32_t k) {
    // Сложение по модулю 2^32
    uint32_t t = x + k;
    uint32_t s = substitute32(t);
    return rol32(s, 11);
}

```

```

// Установка ключа
void MagmaSetKey(magma_key_t* key, const uint8_t
user_key[32]) {
    // Ключ в порядке возрастания: ключ[0]
    for (int i = 0; i < 8; ++i) {
        uint32_t w = ((uint32_t)user_key[4 * i + 0] << 24) |
            ((uint32_t)user_key[4 * i + 1] << 16) |
            ((uint32_t)user_key[4 * i + 2] << 8) |
            ((uint32_t)user_key[4 * i + 3]);

        key->k[i] = w;
    }
}

```

```

// === Внутренний общий код 32-раундовой сети Фейстеля
=== //

```

```

static void MagmaFeistel32(const magma_key_t* key, uint32_t*
N1, uint32_t* N2, int decrypt) {
    uint32_t n1 = *N1, n2 = *N2;

```

```

    if (!decrypt) {

```

```

// Шифрование: K0-K7 повторяется 3 раза, затем K7-K0
for (int r = 0; r < 24; ++r) {
    uint32_t t = n1;
    n1 = n2 ^ F(n1, key->k[r % 8]);
    n2 = t;
}

for (int r = 7; r >= 0; --r) {
    uint32_t t = n1;
    n1 = n2 ^ F(n1, key->k[r]);
    n2 = t;
}
}
else {
    // Расшифровка выполняется по обратному процессу
    for (int r = 0; r < 8; ++r) {
        uint32_t t = n1;
        n1 = n2 ^ F(n1, key->k[r]);
        n2 = t;
    }

    for (int r = 31; r >= 8; --r) {
        uint32_t t = n1;
        n1 = n2 ^ F(n1, key->k[r % 8]);
        n2 = t;
    }
}

// После раундов результат запишем в регистры
*N1 = n1;
*N2 = n2;
}

```

```

// Упаковывать/распаковывать 32-разрядные слова с
// большими порядковыми номерами в байты или из них
static inline uint32_t be32(const uint8_t b[4]) {
    return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16) |
    ((uint32_t)b[2] << 8) | (uint32_t)b[3];
}

```

```

static inline void store_be32(uint8_t out[4], uint32_t v) {
    out[0] = (uint8_t)(v >> 24);
    out[1] = (uint8_t)(v >> 16);
    out[2] = (uint8_t)(v >> 8);
    out[3] = (uint8_t)(v);
}

```

```

// === Шифрование и расшифрование через сеть Фейстеля ===
//
void MagmaEncryptBlock(const magma_key_t* key, const uint8_t
in[8], uint8_t out[8]) {
    uint32_t n1 = be32(in + 0);
    uint32_t n2 = be32(in + 4);
    MagmaFeistel32(key, &n1, &n2, 0);

    // Вывод в том же порядке: 8 байт
    store_be32(out + 0, n1);
    store_be32(out + 4, n2);
}

```

```

void MagmaDecryptBlock(const magma_key_t* key, const uint8_t
in[8], uint8_t out[8]) {
    uint32_t n1 = be32(in + 0);
    uint32_t n2 = be32(in + 4);
    MagmaFeistel32(key, &n1, &n2, 1);

    store_be32(out + 0, n1);
    store_be32(out + 4, n2);
}

```

sha256.c

```

#include "sha256.h"
#include <string.h>

```

```

static const uint32_t K[64] = {
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
    0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,

```

```

    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
    0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
    0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
    0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
    0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
    0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0cbb5,
    0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa,
    0xa4506ceb, 0xbef9a3f7, 0xc67178f2
};

```

```

#define Ch(x, y, z) ((x & y) ^ (~x & z))
#define Maj(x, y, z) ((x & y) ^ (x & z) ^ (y & z))
#define RotR(x, n) ((x >> n) | (x << (32 - n)))
#define Sigma0(x) (RotR(x, 2) ^ RotR(x, 13) ^ RotR(x, 22))
#define Sigma1(x) (RotR(x, 6) ^ RotR(x, 11) ^ RotR(x, 25))
#define sigma0(x) (RotR(x, 7) ^ RotR(x, 18) ^ (x >> 3))
#define sigma1(x) (RotR(x, 17) ^ RotR(x, 19) ^ (x >> 10))

```

```

static const uint32_t H[8] = {
    0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
    0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
};

```

```

void sha256_init(SHA256_CTX* ctx) {
    memcpy(ctx->state, H, sizeof(H));
    ctx->count = 0;
    memset(ctx->buffer, 0, sizeof(ctx->buffer));
}

```

```

void sha256_transform(SHA256_CTX* ctx, const uint8_t* block)
{
    uint32_t W[64];
    uint32_t S[8];
    int i;

```

```

    for (i = 0; i < 16; ++i) {
        W[i] = ((uint32_t)block[i * 4 + 0] << 24) |
        ((uint32_t)block[i * 4 + 1] << 16) |
        ((uint32_t)block[i * 4 + 2] << 8) |
        ((uint32_t)block[i * 4 + 3]);
    }

```

```

    for (; i < 64; ++i) {
        W[i] = sigma1(W[i - 2]) + W[i - 7] + sigma0(W[i - 15]) + W[i
- 16];
    }

```

```

    memcpy(S, ctx->state, 32);

```

```

    for (i = 0; i < 64; ++i) {
        uint32_t T1 = S[7] + Sigma1(S[4]) + Ch(S[4], S[5], S[6]) +
K[i] + W[i];
        uint32_t T2 = Sigma0(S[0]) + Maj(S[0], S[1], S[2]);
        memmove(&S[1], &S[0], 7 * sizeof(uint32_t));
        S[4] += T1;
        S[0] = T1 + T2;
    }

```

```

    for (i = 0; i < 8; ++i) {
        ctx->state[i] += S[i];
    }
}

```

```

void sha256_update(SHA256_CTX* ctx, const uint8_t* data,
size_t len) {
    size_t i, j;

```

```

    j = (size_t)((ctx->count >> 3) & 0x3F);
    ctx->count += (uint64_t)len << 3;

```

```

    if ((j + len) > 63) {
        memcpy(&ctx->buffer[j], data, (i = 64 - j));
        sha256_transform(ctx, ctx->buffer);
    }

```

```

        for (; i + 63 < len; i += 64) {
            sha256_transform(ctx, &data[i]);
        }
        j = 0;
    }
    else {
        i = 0;
    }

    memcpy(&ctx->buffer[j], &data[i], len - i);
}

void sha256_final(SHA256_CTX* ctx, uint8_t
digest[SHA256_DIGEST_SIZE]) {
    uint64_t bitlen = ctx->count;
    size_t i = (size_t)((bitlen >> 3) & 0x3F);
    ctx->buffer[i++] = 0x80;

    if (i > 56) {
        memset(&ctx->buffer[i], 0, 64 - i);
        sha256_transform(ctx, ctx->buffer);
        memset(ctx->buffer, 0, 56);
    }
    else {
        memset(&ctx->buffer[i], 0, 56 - i);
    }

    ctx->buffer[56] = (uint8_t)(bitlen >> 56);
    ctx->buffer[57] = (uint8_t)(bitlen >> 48);
    ctx->buffer[58] = (uint8_t)(bitlen >> 40);
    ctx->buffer[59] = (uint8_t)(bitlen >> 32);
    ctx->buffer[60] = (uint8_t)(bitlen >> 24);
    ctx->buffer[61] = (uint8_t)(bitlen >> 16);
    ctx->buffer[62] = (uint8_t)(bitlen >> 8);
    ctx->buffer[63] = (uint8_t)(bitlen);

    sha256_transform(ctx, ctx->buffer);
    for (i = 0; i < 8; ++i) {
        digest[i * 4 + 0] = (uint8_t)(ctx->state[i] >> 24);
        digest[i * 4 + 1] = (uint8_t)(ctx->state[i] >> 16);
        digest[i * 4 + 2] = (uint8_t)(ctx->state[i] >> 8);
        digest[i * 4 + 3] = (uint8_t)(ctx->state[i]);
    }
}

void sha256_hash(const uint8_t* data, size_t len, uint8_t
out[SHA256_DIGEST_SIZE]) {
    SHA256_CTX ctx;
    sha256_init(&ctx);
    sha256_update(&ctx, data, len);
    sha256_final(&ctx, out);
}

```

ui.c

```

#define _CRT_SECURE_NO_WARNINGS
#define UNICODE
#define _WIN32_WINNT 0x0600

```

```

#include <windows.h>
#include <commdlg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <wctype.h>
#include <commctrl.h>

```

```

#include "ui.h"
#include "student_info.h"
#include "cipher.h"
#include "fileIO.h"
#include "kdf.h"

```

```

#pragma comment(linker, "/manifestdependency:type='win32'
name='Microsoft.Windows.Common-Controls' version='6.0.0.0'
processorArchitecture='*' publicKeyToken='6595b64144ccf1df'
language='*\\'")
#pragma comment(lib, "comctl32.lib")

```

```

enum {
    IDC_EDIT_STUDENT = 1001,
    IDC_EDIT_VARIANT,
    IDC_EDIT_KEY,
    IDC_BTN_KEY_FILE,
    IDC_EDIT_IV,
    IDC_BTN_IN,
    IDC_BTN_OUT,
    IDC_STATIC_IN,
    IDC_STATIC_OUT,
    IDC_BTN_ENC,
    IDC_BTN_DEC,
    IDC_EDIT_LOG
};

```

```

static HINSTANCE ghInstance;
static HWND hEditStudent, hEditVariant, hEditKey, hBtnKeyFile,
hEditIV;
static HWND hBtnIn, hBtnOut, hBtnEnc, hBtnDec, hEditLog,
hStaticIn, hStaticOut;
static WCHAR inPath[MAX_PATH] = L"";
static WCHAR outPath[MAX_PATH] = L"";

```

```

// Вспомогательная функция: char* → wchar_t*
static wchar_t* ansi_to_wide(const char* src) {
    static wchar_t buf[512];

```

```

    if (!src) return L"";

```

```

    int len = MultiByteToWideChar(CP_UTF8, 0, src, -1, NULL,
0);
    if (len > 512) len = 512;

```

```

    MultiByteToWideChar(CP_UTF8, 0, src, -1, buf, len);
    return buf;
}

```

```

static void append_log(const wchar_t* fmt, ...) {
    wchar_t buf[1024];
    va_list ap;
    va_start(ap, fmt);
    vswprintf(buf, sizeof(buf) / sizeof(buf[0]), fmt, ap);
    va_end(ap);
    int len = GetWindowTextLengthW(hEditLog);
    SendMessageW(hEditLog, EM_SETSEL, (LPARAM)len,
(LPARAM)len);
    SendMessageW(hEditLog, EM_REPLACESEL, FALSE,
(LPARAM)buf);
    SendMessageW(hEditLog, EM_REPLACESEL, FALSE,
(LPARAM)L"\r\n");
}

```

```

static int hex_nibble(wchar_t c) {
    if (c >= L'0' && c <= L'9') return c - L'0';
    if (c >= L'A' && c <= L'F') return c - L'A' + 10;
    if (c >= L'a' && c <= L'f') return c - L'a' + 10;

```

```

    return -1;
}

```

```

static int hexw_to_bytes(const wchar_t* whex, uint8_t* out, size_t
expected_len) {
    if (!whex || !out) return 0;
    size_t byte_idx = 0;
    int hi = -1;

```

```

    for (size_t i = 0; whex[i] != L'\0' && byte_idx < expected_len;
i++) {

```

```

        wchar_t wc = whex[i];
        if (wc > 127) continue;

```

```

        int nib = hex_nibble(wc);

```

```

        if (nib == -1) continue;

```

```

        if (hi == -1) {
            hi = nib;

```

```

    }
    else {
        out[byte_idx++] = (uint8_t)((hi << 4) | nib);
        hi = -1;
    }
}

if (hi != -1 || byte_idx != expected_len) return 0;
return 1;
}

static size_t clean_hex_string(const wchar_t* src, wchar_t* dst,
size_t dst_size) {
    size_t j = 0;

    for (size_t i = 0; src[i] != L'\0' && j + 1 < dst_size; i++) {
        wchar_t c = src[i];

        if ((c >= L'0' && c <= L'9') ||
            (c >= L'A' && c <= L'F') ||
            (c >= L'a' && c <= L'f')) {
            dst[j++] = c;
        }
    }

    dst[j] = L'\0';
    return j;
}

static void choose_open_file(HWND owner, WCHAR* outbuf) {
    OPENFILENAMEW ofn = { 0 };
    WCHAR szFile[MAX_PATH] = L"";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = owner;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_PATHMUSTEXIST |
OFN_FILEMUSTEXIST;
    ofn.lpstrFilter = L"All Files\0*.*\0";

    if (GetOpenFileNameW(&ofn)) {
        wcscpy_s(outbuf, MAX_PATH, szFile);
    }
}

static void choose_save_file(HWND owner, WCHAR* outbuf) {
    OPENFILENAMEW ofn = { 0 };
    WCHAR szFile[MAX_PATH] = L"";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = owner;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_OVERWRITEPROMPT;
    ofn.lpstrFilter = L"All Files\0*.*\0";

    if (GetSaveFileNameW(&ofn)) {
        wcscpy_s(outbuf, MAX_PATH, szFile);
    }
}

typedef struct {
    magma_cfb_t* ctx;
} cb_ctx_t;

static void proc_cb_encrypt(void* ctxv, const uint8_t* in, uint8_t*
out, size_t n) {
    cb_ctx_t* c = (cb_ctx_t*)ctxv;
    magma_cfb_encrypt_stream(c->ctx, in, out, n);
}

static void proc_cb_decrypt(void* ctxv, const uint8_t* in, uint8_t*
out, size_t n) {
    cb_ctx_t* c = (cb_ctx_t*)ctxv;
    magma_cfb_decrypt_stream(c->ctx, in, out, n);
}

// ===== ОКХО "О СТУДЕНТЕ"
=====
static void show_student_info(HWND parent) {
    WCHAR msg[1024];

```

```

    swprintf_s(msg, _countof(msg),
        L"Студент: %ls\n"
        L"Группа: %ls\n"
        L"Вариант: %ls\n\n"
        L"Задание:\n%ls",
        STUDENT_FIO, STUDENT_GROUP,
        STUDENT_VARIANT, STUDENT_TASK);

    TASKDIALOGCONFIG tdf = { 0 };
    tdf.cbSize = sizeof(tdf);
    tdf.hwndParent = parent;
    tdf.dwFlags = TDF_ALLOW_DIALOG_CANCELLATION |
TDF_POSITION_RELATIVE_TO_WINDOW;
    tdf.pszWindowTitle = L"Курсовая работа — Магма (CFB)";
    tdf.pszMainIcon = TD_INFORMATION_ICON;
    tdf.pszMainInstruction = L"Сведения о студенте";
    tdf.pszContent = msg;
    tdf.pszExpandedInformation = L"Реализация
криптографического алгоритма «Магма» (ГОСТ Р 34.12-2015)
в режиме гаммирования с обратной связью.";
    tdf.pszFooter = L"ГОСТ Р 34.12-2015 | Учебное
приложение";
    TaskDialogIndirect(&tdf, NULL, NULL, NULL);
}

// ===== ОЧОБХОЕ ОКХО =====
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg,
WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            HFONT hFont = CreateFontW(16, 0, 0, 0, FW_NORMAL,
FALSE, FALSE, FALSE,
            DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
            CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
            DEFAULT_PITCH | FF_DONTCARE, L"Segoe UI");
            HFONT hBtnFont = CreateFontW(15, 0, 0, 0, 0, 0,
FW_MEDIUM, FALSE, FALSE, FALSE,
            DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
            CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
            DEFAULT_PITCH | FF_DONTCARE, L"Segoe UI");
            HFONT hLogFont = CreateFontW(14, 0, 0, 0, 0, 0,
FW_NORMAL, FALSE, FALSE, FALSE,
            DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
            CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
            FIXED_PITCH | FF_DONTCARE, L"Consolas");

            CreateWindowW(L"STATIC", L"Студент:", WS_VISIBLE |
WS_CHILD, 10, 10, 80, 24, hwnd, NULL, ghInstance, NULL);
            hEditStudent = CreateWindowW(L"EDIT", STUDENT_FIO,
WS_VISIBLE | WS_CHILD | WS_BORDER | ES_READONLY,
90, 10, 300, 24, hwnd, (HMENU)IDC_EDIT_STUDENT,
ghInstance, NULL);
            SendMessageW(hEditStudent, WM_SETFONT,
(WPARAM)hFont, TRUE);

            CreateWindowW(L"STATIC", L"Вариант:", WS_VISIBLE |
WS_CHILD, 10, 42, 80, 24, hwnd, NULL, ghInstance, NULL);
            hEditVariant = CreateWindowW(L"EDIT",
ansi_to_wide(STUDENT_VARIANT), WS_VISIBLE |
WS_CHILD | WS_BORDER | ES_READONLY, 90, 42, 160, 24,
hwnd, (HMENU)IDC_EDIT_VARIANT, ghInstance, NULL);
            SendMessageW(hEditVariant, WM_SETFONT,
(WPARAM)hFont, TRUE);

            CreateWindowW(L"STATIC", L"Ключ (hex или пароль):",
WS_VISIBLE | WS_CHILD, 10, 74, 200, 24, hwnd, NULL,
ghInstance, NULL);
            hEditKey = CreateWindowW(L"EDIT", L"", WS_VISIBLE |
WS_CHILD | WS_BORDER, 220, 74, 520, 24, hwnd,
(HMENU)IDC_EDIT_KEY, ghInstance, NULL);
            SendMessageW(hEditKey, WM_SETFONT,
(WPARAM)hFont, TRUE);

            hBtnKeyFile = CreateWindowW(L"BUTTON", L"Загрузить
ключ...", WS_VISIBLE | WS_CHILD, 770, 74, 150, 26, hwnd,
(HMENU)IDC_BTN_KEY_FILE, ghInstance, NULL);
            SendMessageW(hBtnKeyFile, WM_SETFONT,
(WPARAM)hBtnFont, TRUE);

```



```

        CreateWindowW(L"STATIC", L"IV (hex, 16 символов):",
WS_VISIBLE | WS_CHILD, 10, 106, 160, 24, hwnd, NULL,
ghInstance, NULL);
        hEditIV = CreateWindowW(L"EDIT",
L"0001020304050607", WS_VISIBLE | WS_CHILD |
WS_BORDER, 180, 106, 180, 24, hwnd,
(HMENU)IDC_EDIT_IV, ghInstance, NULL);
        SendMessageW(hEditIV, WM_SETFONT,
(WPARAM)hFont, TRUE);
        SendMessageW(hEditIV, EM_SETLIMITTEXT, 16, 0);

        hBtnIn = CreateWindowW(L"BUTTON", L"Входной
файл...", WS_VISIBLE | WS_CHILD, 10, 142, 160, 28, hwnd,
(HMENU)IDC_BTN_IN, ghInstance, NULL);
        SendMessageW(hBtnIn, WM_SETFONT,
(WPARAM)hBtnFont, TRUE);
        hBtnOut = CreateWindowW(L"BUTTON", L"Выходной
файл...", WS_VISIBLE | WS_CHILD, 180, 142, 160, 28, hwnd,
(HMENU)IDC_BTN_OUT, ghInstance, NULL);
        SendMessageW(hBtnOut, WM_SETFONT,
(WPARAM)hBtnFont, TRUE);

        hBtnEnc = CreateWindowW(L"BUTTON",
L"Зашифровать", WS_VISIBLE | WS_CHILD, 360, 142, 140, 32,
hwnd, (HMENU)IDC_BTN_ENC, ghInstance, NULL);
        SendMessageW(hBtnEnc, WM_SETFONT,
(WPARAM)hBtnFont, TRUE);
        hBtnDec = CreateWindowW(L"BUTTON",
L"Расшифровать", WS_VISIBLE | WS_CHILD, 510, 142, 140,
32, hwnd, (HMENU)IDC_BTN_DEC, ghInstance, NULL);
        SendMessageW(hBtnDec, WM_SETFONT,
(WPARAM)hBtnFont, TRUE);

        CreateWindowW(L"STATIC", L"Лог операций:",
WS_VISIBLE | WS_CHILD, 10, 182, 120, 24, hwnd, NULL,
ghInstance, NULL);
        hEditLog = CreateWindowW(L"EDIT", L"", WS_VISIBLE |
WS_CHILD | WS_BORDER | ES_LEFT | ES_MULTILINE |
ES_AUTOVSCROLL | ES_READONLY | WS_VSCROLL,
10, 208, 910, 260, hwnd, (HMENU)IDC_EDIT_LOG,
ghInstance, NULL);
        SendMessageW(hEditLog, WM_SETFONT,
(WPARAM)hLogFont, TRUE);

        append_log(L"Приложение готово к работе.");
        break;
    }
    case WM_COMMAND: {
        int id = LOWORD(wParam);

        if (id == IDC_BTN_KEY_FILE) {

            WCHAR path[MAX_PATH] = L"";
            choose_open_file(hwnd, path);
            if (path[0]) {

                size_t keylen;
                uint8_t* kbuf = read_file_w(path, &keylen);

                if (!kbuf) {
                    append_log(L"Ошибка: не удалось прочитать файл
ключа: %s", path);
                }
                else {
                    uint8_t final_key[32];
                    kdf_sha256(kbuf, keylen, final_key);
                    append_log(L"Ключ из файла нормируется через
SHA-256 для соответствия hex-ключ (64 символа): %s", path);

                    WCHAR whex[65];

                    for (int i = 0; i < 32; i++) {
                        wsprintfW(&whex[i * 2], L"%02X", final_key[i]);
                    }

                    whex[64] = L'\0';
                    SetWindowTextW(hEditKey, whex);
                    free(kbuf);
                }
            }
        }
    }

```

```

    }
}
else if (id == IDC_BTN_IN) {
    choose_open_file(hwnd, inPath);
    if (inPath[0]) append_log(L"Входной файл: %s", inPath);
}
else if (id == IDC_BTN_OUT) {
    choose_save_file(hwnd, outPath);
    if (outPath[0]) append_log(L"Выходной файл: %s",
outPath);
}
else if (id == IDC_BTN_ENC || id == IDC_BTN_DEC) {
    if (!inPath[0] || !outPath[0]) {
        append_log(L"Ошибка: выберите входной и
выходной файлы.");
        break;
    }

    WCHAR keyw[512];
    GetWindowTextW(hEditKey, keyw, _countof(keyw));
    uint8_t key_raw[32];

    WCHAR clean_key[128] = { 0 };
    size_t hex_count = clean_hex_string(keyw, clean_key,
_countof(clean_key));

    if (hex_count == 64) {

        if (hexw_to_bytes(clean_key, key_raw, 32)) {
            append_log(L"Используется введённый hex-ключ
(64 символа).");
        }
        else {
            append_log(L"Ошибка: не удалось распарсить hex-
ключ.");
            break;
        }
    }
    else {
        append_log(L"Введён не hex-ключ (%zu символов) →
хэшируем как пароль.", hex_count);
        int len_utf8 = WideCharToMultiByte(CP_UTF8, 0,
keyw, -1, NULL, 0, NULL, NULL);

        if (len_utf8 <= 1) {
            append_log(L"Ошибка: пустой ввод.");
            break;
        }

        char* key_utf8 = (char*)malloc(len_utf8);
        WideCharToMultiByte(CP_UTF8, 0, keyw, -1,
key_utf8, len_utf8, NULL, NULL);
        kdf_sha256((uint8_t*)key_utf8, len_utf8 - 1, key_raw);
        free(key_utf8);
    }

    WCHAR ivw[64];
    GetWindowTextW(hEditIV, ivw, _countof(ivw));
    WCHAR clean_iv[32] = { 0 };
    size_t iv_hex = clean_hex_string(ivw, clean_iv,
_countof(clean_iv));

    if (iv_hex != 16) {
        append_log(L"Ошибка: IV должен быть ровно 16 hex-
символов. Фактически: %zu", iv_hex);
        break;
    }

    uint8_t iv[8];

    if (!hexw_to_bytes(clean_iv, iv, 8)) {
        append_log(L"Ошибка: не удалось распарсить IV.");
        break;
    }

    magma_cfb_t ctx;
    magma_cfb_initialization(&ctx, key_raw, iv);
    cb_ctx_t cbctx = { .ctx = &ctx };

```

```

        int ok = process_file_stream_w(inPath, outPath, &cbctx,
        (id == IDC_BTN_ENC) ? proc_cb_encrypt :
        proc_cb_decrypt);

        if (ok) {
            append_log(L"Успешно: %s", outPath);
        }
        else {
            append_log(L"Ошибка при обработке файлов!");
        }
    }
    break;
}
case WM_DESTROY:
    PostQuitMessage(0);
    break;
default:
    return DefWindowProcW(hwnd, msg, wParam, lParam);
}
return 0;
}

int run_gui(HINSTANCE hInstance, int nCmdShow) {
    ghInstance = hInstance;
    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = L"MagmaCFBWin";
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = CreateSolidBrush(RGB(248, 248, 250));
    RegisterClassW(&wc);

    HWND hwnd = CreateWindowW(
        wc.lpszClassName,
        L"Магма (CFB) — Демонстрация шифрования",
        WS_OVERLAPPEDWINDOW & ~WS_THICKFRAME,
        CW_USEDEFAULT, CW_USEDEFAULT, 950, 520,
        NULL, NULL, hInstance, NULL
    );

    if (!hwnd) return -1;

    // Показываем информацию о студенте
    show_student_info(hwnd);

    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG msg;
    while (GetMessageW(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    return (int)msg.wParam;
}

```

cipher.h

```
#pragma once
```

```
#ifndef CIPHER_H
#define CIPHER_H
```

```
#include <stdint.h>
#include <stddef.h>
#include "magma.h"
```

```
// === Структура CFB-контекста (сохранение в регистр) === //
typedef struct {
    magma_key_t key;
    uint8_t reg[MAGMA_BLOCK_SIZE]; // 8 байт
    регистр (IV)
} magma_cfb_t;
```

```
// === Инициализация key_raw 32 байта, IV 8 байт === //
void magma_cfb_initialization(magma_cfb_t *ctx, const uint8_t
key_raw[32], const uint8_t iv[8]);
```

```
// === Шифрование/дешифрование потоков произвольной
длины === //
// decrypt и encrypt используют разные обновления регистра
void magma_cfb_encrypt_stream(magma_cfb_t *ctx, const uint8_t
*in, uint8_t *out, size_t len);
```

```
void magma_cfb_decrypt_stream(magma_cfb_t *ctx, const
uint8_t* in, uint8_t* out, size_t len);
#endif
```

fileIO.h

```
#pragma once
```

```
#ifndef FILE_IO_H
#define FILE_IO_H
```

```
#include <stddef.h>
#include <stdint.h>
```

```
/* Простая функция чтения файла в динамический буфер.
Возвращает указатель длину через outlen. Возвращает NULL
при ошибке. */
uint8_t* read_file(const char *path, size_t *outlen);
```

```
uint8_t* read_file_w(const wchar_t* path, size_t* outlen);
```

```
// Запись буфера в файл. Возвращает 1 при успехе, 0 при
ошибке
int write_file(const char *path, const uint8_t *buf, size_t len);
```

```
/* Поблочная обработка (считывание входа и запись
результат с вызовом
для обработки блоков */
typedef void (*process_block_cb)(void *ctx, const uint8_t *in,
uint8_t *out, size_t n);
```

```
int process_file_stream_w(const wchar_t *inpath, const wchar_t
*outpath, void *ctx_cb, process_block_cb cb);
```

```
#endif
```

kdf.h

```
#pragma once
```

```
#include <stdint.h>
#include <stddef.h>
```

```
// Безопасное получение 32-байтного ключа через SHA-256
void kdf_sha256(const uint8_t* input, size_t input_len, uint8_t
out[32]);
```

magma.h

```
#pragma once
```

```
#ifndef MAGMA_H
#define MAGMA_H
```

```
#include <stdint.h>
#include <stddef.h>
```

```
#define MAGMA_BLOCK_SIZE 8U
```

```
typedef struct {
    // 8 * 32 = 256 бит
    uint32_t k[8];
} magma_key_t;
```

```
// === Инициализация ключа (32 байта) === //
void MagmaSetKey(magma_key_t* key, const uint8_t
user_key[32]);
```

```
// === Шифрование одного блока (8 байт) === //
void MagmaEncryptBlock(const magma_key_t* key, const uint8_t
in[8], uint8_t out[8]);
```

```
// == Расшифровка одного блока (8 байт) ==
void MagmaDecryptBlock(const magma_key_t* key, const uint8_t
in[8], uint8_t out[8]);

#endif
```

sha256.h

```
#pragma once
```

```
#include <stdint.h>
#include <stddef.h>
```

```
#define SHA256_DIGEST_SIZE 32
```

```
typedef struct {
    uint32_t state[8];
    uint64_t count;
    uint8_t buffer[64];
} SHA256_CTX;
```

```
void sha256_init(SHA256_CTX* ctx);
void sha256_update(SHA256_CTX* ctx, const uint8_t* data,
size_t len);
void sha256_final(SHA256_CTX* ctx, uint8_t
digest[SHA256_DIGEST_SIZE]);
```

```
// Удобная обёртка: хэширует всё за один вызов
void sha256_hash(const uint8_t* data, size_t len, uint8_t
out[SHA256_DIGEST_SIZE]);
```

student_info.h

```
#pragma once
```

```
#ifndef STUDENT_INFO_H
#define STUDENT_INFO_H
```

```
#define STUDENT_FIO L"ФИО: Белянин Никита Николаевич"
#define STUDENT_GROUP L"Группа: ПИбд-41"
#define STUDENT_VARIANT "6"
#define STUDENT_TASK L"Лабораторная работа 3:
Реализация шифра Магма (ГОСТ 28147) - режим CFB
(гаммирование с обратной связью)"
```

```
#endif
```

ui.h

```
#pragma once
```

```
#ifndef UI_H
#define UI_H
```

```
#include <Windows.h>
#include <stdint.h>
```

```
// Запуск GUI
int run_gui(HINSTANCE hInstance, int nCmdShow);
```

```
// Функция логирования для GUI
void append_log(const wchar_t* fmt, ...);
```

```
#endif
```

Реализация на Java

Main.java

```
package org.encrypting;
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.io.File;
import java.io.PrintStream;
```

```
import java.nio.charset.StandardCharsets;
```

```
public class Main {
    private JFrame mainFrame;
    private JTextField keyField;
    private JTextField ivField;
    private File inputFile, outputFile;
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
Main().showStudentInfo());
    }
```

```
    private void showStudentInfo() {
        JOptionPane.showMessageDialog(
            null,
            "<html><b>Студент:</b> " + StudentInfo.FIO + "<br>"
+
            "<b>Группа:</b> " + StudentInfo.GROUP +
"<br>" +
            "<b>Вариант:</b> " + StudentInfo.VARIANT +
"<br><br>" +
            "<b>Задание:</b><br>" + StudentInfo.TASK +
"</html>",
            "Сведения о студенте",
            JOptionPane.INFORMATION_MESSAGE
        );
        createAndShowGUI();
    }
```

```
    private void createAndShowGUI() {
        mainFrame = new JFrame("Магма (CFB) — Шифрование");
```

```
mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
;
    mainFrame.setLayout(new BorderLayout());
```

```
    // Верхняя панель
    JPanel topPanel = new JPanel(new GridBagLayout());
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.anchor = GridBagConstraints.WEST;
```

```
    gbc.gridx = 0; gbc.gridy = 0;
    topPanel.add(new JLabel("Ключ (hex или текст:)", gbc);
    gbc.gridx = 1;
    keyField = new JTextField(40);
    topPanel.add(keyField, gbc);
    gbc.gridx = 2;
    JButton loadKeyBtn = new JButton("Загрузить ключ...");
    loadKeyBtn.addActionListener(this::loadKeyFromFile);
    topPanel.add(loadKeyBtn, gbc);
```

```
    gbc.gridx = 0; gbc.gridy = 1;
    topPanel.add(new JLabel("IV (16 hex символов:)", gbc);
    gbc.gridx = 1;
    ivField = new JTextField("1234567890ABCDEF");
    ivField.setColumns(20);
    topPanel.add(ivField, gbc);
```

```
mainFrame.add(topPanel, BorderLayout.NORTH);
```

```
    // Центр: кнопки
    JPanel centerPanel = new JPanel(new FlowLayout());
    JButton selectInput = new JButton("Выбрать входной
файл");
    selectInput.addActionListener(e -> selectFile(true));
    JButton selectOutput = new JButton("Выбрать выходной
файл");
    selectOutput.addActionListener(e -> selectFile(false));
    JButton encryptBtn = new JButton("Зашифровать");
    encryptBtn.addActionListener(e -> processFile(true));
    JButton decryptBtn = new JButton("Расшифровать");
    decryptBtn.addActionListener(e -> processFile(false));
```

```
centerPanel.add(selectInput);
centerPanel.add(selectOutput);
centerPanel.add(encryptBtn);
centerPanel.add(decryptBtn);
```

```

mainFrame.add(centerPanel, BorderLayout.CENTER);

// Лог
JTextArea logArea = new JTextArea(10, 50);
logArea.setEditable(false);
logArea.setFont(new Font(Font.MONOSPACED,
Font.PLAIN, 12));
JScrollPane scroll = new JScrollPane(logArea);
mainFrame.add(scroll, BorderLayout.SOUTH);

// Перенаправление System.out в лог
System.setOut(new PrintStream(new
TextAreaOutputStream(logArea)));

mainFrame.setSize(800, 500);
mainFrame.setLocationRelativeTo(null);
mainFrame.setVisible(true);
}

private void loadKeyFromFile(ActionEvent e) {
    JFileChooser chooser = new JFileChooser();
    if (chooser.showOpenDialog(mainFrame) ==
JFileChooser.APPROVE_OPTION) {
        try {
            String keyText =
FileUtils.readFile(chooser.getSelectedFile());
            keyField.setText(keyText);
            System.out.println("Ключ из файла хэширован через
SHA-256");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(mainFrame, "Ошибка
чтения ключа: " + ex.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

private void selectFile(boolean input) {
    JFileChooser chooser = new JFileChooser();
    if (chooser.showOpenDialog(mainFrame) ==
JFileChooser.APPROVE_OPTION) {
        if (input) {
            inputFile = chooser.getSelectedFile();
            System.out.println("Входной файл: " +
inputFile.getAbsolutePath());
        } else {
            outputFile = chooser.getSelectedFile();
            System.out.println("Выходной файл: " +
outputFile.getAbsolutePath());
        }
    }
}

private void processFile(boolean encrypt) {
    if (inputFile == null || outputFile == null) {
        JOptionPane.showMessageDialog(mainFrame, "Выберите
входной и выходной файлы!", "Ошибка",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    try {
        // Обработка ключа
        byte[] keyBytes = parseKey(keyField.getText());
        // Обработка IV
        byte[] ivBytes = parseIV(ivField.getText());

        MagmaCFB cipher = new MagmaCFB(keyBytes, ivBytes);
        byte[] data = FileUtils.readFile(inputFile);
        byte[] result = encrypt ? cipher.encrypt(data) :
cipher.decrypt(data);
        FileUtils.writeFile(outputFile, result);

        String op = encrypt ? "Зашифровано" : "Расшифровано";
        System.out.println(op + ": " +
outputFile.getAbsolutePath());
        JOptionPane.showMessageDialog(mainFrame, op + "
успешно!", "Успех",
JOptionPane.INFORMATION_MESSAGE);
    }
}

```

```

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка: "
+ ex.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
        ex.printStackTrace();
    }
}

private byte[] parseKey(String keyInput) {
    // BCEГДА трактуем как пароль
    System.out.println("Ключ обработан как пароль → SHA-
256");
    return
HashUtils.sha256(keyInput.getBytes(StandardCharsets.UTF_8));
}

private byte[] parseIV(String ivInput) {
    ivInput = ivInput.replaceAll("[^0-9A-Fa-f]", "");
    if (ivInput.length() != 16) {
        throw new IllegalArgumentException("IV должен
содержать 16 hex-символов");
    }
    return hexToBytes(ivInput);
}

private static byte[] hexToBytes(String hex) {
    byte[] bytes = new byte[hex.length() / 2];
    for (int i = 0; i < bytes.length; i++) {
        bytes[i] = (byte) Integer.parseInt(hex.substring(2 * i, 2 * i +
2), 16);
    }
    return bytes;
}

private static String bytesToHex(byte[] bytes) {
    StringBuilder sb = new StringBuilder();
    for (byte b : bytes) {
        sb.append(String.format("%02X", b));
    }
    return sb.toString();
}

// Вспомогательный класс для перенаправления вывода в
JTextArea
private static class TextAreaOutputStream extends
java.io.OutputStream {
    private final JTextArea textArea;

    public TextAreaOutputStream(JTextArea textArea) {
        this.textArea = textArea;
    }

    @Override
    public void write(int b) {
        textArea.append(String.valueOf((char) b));
    }

    textArea.setCaretPosition(textArea.getDocument().getLength());
}

@Override
public void write(byte[] b, int off, int len) {
    textArea.append(new String(b, off, len));
}

textArea.setCaretPosition(textArea.getDocument().getLength());
}
}
}

```

FileUtils.java

```

package org.encrypting;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;

public class FileUtils {

```

```

public static byte[] readFile(File file) throws IOException {
    return Files.readAllBytes(file.toPath());
}

// Чтение файла как текста в UTF-8 (только для ключей!)
public static String readTextFile(File file) throws IOException {
    return Files.readString(file.toPath(),
StandardCharsets.UTF_8);
}

public static void writeFile(File file, byte[] data) throws
IOException {
    Files.write(file.toPath(), data);
}
}

```

StudentInfo.java

```
package org.encrypting;
```

```

public class StudentInfo {

    public static final String FIO = "Беянин Никита
Николаевич";

    public static final String GROUP = "ПИбд-41";

    public static final String VARIANT = "6";

    public static final String TASK = "Реализация шифра Магма
(ГОСТ Р 34.12-2015) в режиме гаммирования с обратной
связью (CFB)";
}

```

MagmaCFB.java

```
package org.encrypting;
```

```

import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.util.Arrays;

// Реализация блочного шифра "Магма" (ГОСТ Р 34.12-2015) в
режиме гаммирования с обратной связью (CFB).
/* Параметры алгоритма: Длина блока 64 бита (8 байт), длина
ключа 256 бит (32 байта). Количество раундов: 32
Режим работы: CFB (Cipher Feedback) — потоковый режим,
не требует дополнения (padding) */
public class MagmaCFB {

    /* S-блоки (таблицы замен) — фиксированная часть
стандарта.
    Используются 8 таблиц по 16 значений (4-битная замена).
Значения официально заданы в ГОСТ */
    private static final int[][] S = {
        {12, 8, 2, 10, 6, 4, 14, 5, 11, 13, 9, 7, 15, 3, 1, 0},
        {8, 10, 12, 6, 2, 11, 0, 7, 4, 13, 15, 1, 9, 5, 3, 14},
        {12, 14, 1, 13, 10, 8, 9, 15, 2, 0, 6, 4, 11, 3, 7, 5},
        {13, 7, 5, 12, 10, 9, 0, 6, 1, 11, 14, 8, 3, 15, 2, 4},
        {10, 14, 13, 8, 2, 11, 7, 15, 12, 6, 3, 9, 0, 5, 4, 1},
        {7, 9, 11, 15, 12, 2, 14, 13, 10, 3, 8, 4, 1, 5, 6, 0},
        {14, 6, 3, 5, 10, 15, 8, 12, 11, 1, 4, 7, 9, 13, 0, 2},
        {5, 0, 15, 10, 3, 12, 9, 6, 8, 13, 2, 11, 14, 4, 1, 7}
    };

    // Массив раундовых ключей
    private final int[] roundKeys = new int[32];

    // Вектор инициализации (IV) — 64 бита (8 байт)
обеспечивает уникальность гаммы при одинаковом ключе.
    private byte[] iv;

    public MagmaCFB(byte[] userKey, byte[] iv) {
        if (userKey.length != 32) throw new
IllegalArgumentExcepion("Ключ должен быть 32 байта");
        if (iv.length != 8) throw new IllegalArgumentExcepion("IV
должен быть 8 байт");

```

```

        this.iv = iv.clone();
        expandKey(userKey);
    }

    // Расширение 256-битного ключа до 32 раундовых 32-
битных ключей.
    private void expandKey(byte[] key) {
        // Интерпретируем байты ключа как 8 32-битных слов
        ByteBuffer bb =
ByteBuffer.wrap(key).order(ByteOrder.BIG_ENDIAN);
        int[] k = new int[8];
        for (int i = 0; i < 8; i++) k[i] = bb.getInt();

        // Формируем 32 раундовых ключа K0..K7, K0..K7,
K0..K7, K7..K0
        for (int i = 0; i < 24; i++) roundKeys[i] = k[i % 8];
        for (int i = 0; i < 8; i++) roundKeys[24 + i] = k[7 - i];
    }

    // F-функция (раундовая функция сети Фейстеля).
    private int F(int x) {
        int y = 0;

        // Обрабатываем 8 4-битных фрагментов слова (слева
направо)
        for (int i = 0; i < 8; i++) {
            int nibble = (x >> (28 - 4 * i)) & 0xF;

            // Применяем замену через i-й S-блок
            int s = S[i][nibble];
            y |= (s << (28 - 4 * i));
        }

        // Циклический сдвиг влево на 11 бит
        return Integer.rotateLeft(y, 11);
    }

    // Шифрование одного 64-битного блока (основная функция
сети Фейстеля).
    private void encryptBlock(byte[] in, int inOff, byte[] out, int
outOff) {
        ByteBuffer bb = ByteBuffer.wrap(in, inOff,
8).order(ByteOrder.BIG_ENDIAN);
        int n1 = bb.getInt();
        int n2 = bb.getInt();

        // 32 раунда сети Фейстеля
        for (int i = 0; i < 32; i++) {
            int t = n1;
            n1 = n2 ^ F(n1 ^ roundKeys[i]);
            n2 = t;
        }

        bb = ByteBuffer.wrap(out, outOff,
8).order(ByteOrder.BIG_ENDIAN);
        bb.putInt(n1);
        bb.putInt(n2);
    }

    // Шифрование данных в режиме CFB (гаммирование с
обратной связью).
    /* Гамма генерируется шифрованием регистра (начинается с
IV)
    Каждый байт открытого текста XOR-ится с первым байтом
гаммы
    Регистр сдвигается, и в конец добавляется байт
шифротекста */
    public byte[] encrypt(byte[] data) {
        if (data == null || data.length == 0) return new byte[0];

        byte[] result = new byte[data.length];

        // Инифиализация регистра IV
        byte[] reg = iv.clone();

        for (int i = 0; i < data.length; i++) {

            // Генерация 8-байтной гаммы
            byte[] gamma = new byte[8];

```

```

        encryptBlock(reg, 0, gamma, 0);

        // XOR открытого текста с первым байтом гаммы
        result[i] = (byte) (data[i] ^ gamma[0]);

        // Обновление регистра
        System.arraycopy(reg, 1, reg, 0, 7);
        reg[7] = result[i];
    }
    return result;
}

// Расшифрование данных в режиме CFB.
public byte[] decrypt(byte[] data) {
    if (data == null || data.length == 0) return new byte[0];
    byte[] result = new byte[data.length];

    // Инициализируем регистр IV
    byte[] reg = iv.clone();

    for (int i = 0; i < data.length; i++) {

        // Генерация гаммы (аналогично шифрованию)
        byte[] gamma = new byte[8];
        encryptBlock(reg, 0, gamma, 0);

        // XOR шифротекста с первым байтом гаммы
        result[i] = (byte) (data[i] ^ gamma[0]);

        // Обновление регистра
        System.arraycopy(reg, 1, reg, 0, 7);
        reg[7] = data[i];
    }
    return result;
}
}

```

HashUtils.java

```
package org.encrypting;
```

```
public class HashUtils {
```

```

    // Начальные хеш-значения (первые 32 бита дробных частей
    // квадратных корней первых 8 простых чисел)
    private static final int[] H = {
        0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
        0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
    };

```

```

    // Константы (первые 32 бита дробных частей кубических
    // корней первых 64 простых чисел)
    private static final int[] K = {
        0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
        0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
        0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
        0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
        0xe49b69c1, 0xeffbe4786, 0x0fc19dc6, 0x240ca1cc,
        0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
        0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
        0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
        0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
        0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
        0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
        0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
        0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
        0x391c0cb3, 0x4ed8aa4a, 0x5b99cca4f, 0x682e6ff3,
        0x748f82ee, 0x78a5636f, 0x84c87814, 0x8ecc70208,
        0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
    };

```

```

    public static byte[] sha256(byte[] input) {
        // 1. Предварительная обработка
        int length = input.length;
        // Длина в битах
        long bitLength = (long) length << 3;

```

```
        // Добавляем бит '1'
```

```

        int paddingLength = (56 - (length + 1) % 64 + 64) % 64;
        byte[] padded = new byte[length + 1 + paddingLength + 8];
        System.arraycopy(input, 0, padded, 0, length);
        padded[length] = (byte) 0x80;

```

```

        // Добавляем длину в битах ( 64-bit)
        for (int i = 0; i < 8; i++) {
            padded[padded.length - 8 + i] = (byte) (bitLength >>> (56 -
                i * 8));
        }

```

```

        // 2. Инициализация хеш-значений
        int[] h = H.clone();

```

```

        // 3. Обработка блоков по 512 бит (64 байта)
        for (int i = 0; i < padded.length; i += 64) {
            int[] w = new int[64];
            // Копируем 16 слов
            for (int j = 0; j < 16; j++) {
                w[j] = ((padded[i + j * 4] & 0xFF) << 24) |
                    ((padded[i + j * 4 + 1] & 0xFF) << 16) |
                    ((padded[i + j * 4 + 2] & 0xFF) << 8) |
                    (padded[i + j * 4 + 3] & 0xFF);
            }

```

```

            // Расширяем до 64 слов
            for (int j = 16; j < 64; j++) {
                int s0 = Integer.rotateRight(w[j-15], 7) ^
                    Integer.rotateRight(w[j-15], 18) ^
                    (w[j-15] >>> 3);
                int s1 = Integer.rotateRight(w[j-2], 17) ^
                    Integer.rotateRight(w[j-2], 19) ^
                    (w[j-2] >>> 10);
                w[j] = w[j-16] + s0 + w[j-7] + s1;
            }

```

```

            // Инициализация рабочих переменных
            int a = h[0], b = h[1], c = h[2], d = h[3],
                e = h[4], f = h[5], g = h[6], h7 = h[7];

```

```

            // Основной цикл
            for (int j = 0; j < 64; j++) {
                int s1 = Integer.rotateRight(e, 6) ^
                    Integer.rotateRight(e, 11) ^
                    Integer.rotateRight(e, 25);
                int ch = (e & f) ^ ((~e) & g);
                int temp1 = h7 + s1 + ch + K[j] + w[j];
                int s0 = Integer.rotateRight(a, 2) ^
                    Integer.rotateRight(a, 13) ^
                    Integer.rotateRight(a, 22);
                int maj = (a & b) ^ (a & c) ^ (b & c);
                int temp2 = s0 + maj;

```

```

                h7 = g;
                g = f;
                f = e;
                e = d + temp1;
                d = c;
                c = b;
                b = a;
                a = temp1 + temp2;
            }

```

```

            // Добавляем результат блока
            h[0] += a;
            h[1] += b;
            h[2] += c;
            h[3] += d;
            h[4] += e;
            h[5] += f;
            h[6] += g;
            h[7] += h7;
        }

```

```

        // 4. Формируем хеш
        byte[] hash = new byte[32];
        for (int i = 0; i < 8; i++) {
            hash[i * 4] = (byte) (h[i] >>> 24);
            hash[i * 4 + 1] = (byte) (h[i] >>> 16);

```

```
        hash[i * 4 + 2] = (byte) (h[i] >>> 8);
        hash[i * 4 + 3] = (byte) h[i];
    }
    return hash;
}
```